



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

txKernel

内核赛初赛技术报告

参赛队名 老被炸

队伍成员 杨岩琰、刘佳硕、孟书培

指导老师 夏文、仇洁婷

2026 年 6 月

摘要

txKernel 是一个用 Rust 语言实现，支持 RISC-V64 与 LoongArch64 架构，面向 Linux ABI 兼容构建的基于异步无栈协程执行模型的宏内核。

目前 TxKernel 实现进度如下：

- **用户态程序**：能引导并运行 BusyBox 和 Alpine Linux 用户空间，支持 shell、vi、gcc/tcc 编译、sqlite3（含并发 WAL 写、ext4 持久化存储），以及 iptables/nft NAT、veth/bridge/netns 网络。
- **Linux 兼容性**：实现 200+ 系统调用，涵盖文件系统、进程/信号、内存管理、futex、epoll、io_uring、eventfd、timerfd、AIO、IPC、网络等子系统，并支持 glibc 与 musl C 库。
- **异步无栈协程**：线程不持有内核栈，运行状态通过 future 保存。在线程用 future 实现的基础上支持用户态定时抢占与内核协作式调度，并且构建了我们的异步内核观测框架 `tx-observe`，支持批量追踪系统调用内部的执行时长，并可以导出到 perfetto/parquet 格式进行分析。

截止初赛完成时，TxKernel 已经通过绝大部分初赛测例，LTP 得分约 6000 分，在排行榜位列 11 名。

9	T2026104869911460	最小可运行	武汉大学	54	2026-06-24 12:52:42	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.C
10	T2026104869910509	letsgooooo	武汉大学	474	2026-06-25 14:07:07	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.C
11	T2026181239910991	老被炸	哈尔滨工业大学 (深圳)	45	2026-06-18 13:29:08	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.C
12	T2026104869910452	Falcores	武汉大学	119	2026-06-25 15:43:31	102.0	98.0	98.0	99.0	55.0	55.0	54.0	54.C

txKernel 各个模块的完成情况如下表：

表 0-1：模块完成情况

模块	完成情况
进程管理	异步无栈协程执行、多核调度与负载均衡；进程/线程身份与负载分离的对象模型，支持 fork/clone/exec/exit/wait 与线程组语义
内存管理	按需分页、写时复制、缺页处理；内核空间动态映射、用户指针检查；匿名与文件 mmap、共享内存映射；分级内存分配器
文件系统	类 Linux 虚拟文件系统（RNode 与 backing 拆分）、路径解析缓存、页缓存读写；支持挂载；异步 Ext4 磁盘文件系统，以及 tmpfs、procfs、devfs
信号机制	标准信号与实时信号的不同等待与排队机制、用户自定义信号处理、进程级与线程级信号投递
进程间通信	管道、System V 信号量/消息队列/共享内存、eventfd/signalfd、futex

模块	完成情况
设备驱动	硬件中断、MMIO 与 virtio 驱动、串口驱动；设备树（FDT）解析
网络模块	TCP/UDP/ICMP、NetDeviceOps trait 统一抽象 6 种设备、IPv4 协议
架构管理	自研硬件抽象层、支持 RISC-V 与 LoongArch 双架构

在架构设计上，我们希望借用 Rust 的类型系统建模开发规范，用编译器、linter 检查保证内核使用的语言一致（特别是约束 AI），并通过异步协作式调度的特性简化系统调用路径上被异步事件打断的处理。txKernel 设计的核心概念如下：

- **内核主体围绕系统调用设计。** 系统调用是由一系列同步 step 组成的 future 状态机，由线程的 future 嵌套持有。系统调用的订阅—阻塞—等待—唤醒循环由调度器统一处理，子系统专注于实现系统调用每一步 step 的语义，不再各自实现自己的等待原语。这一设计在进程、虚拟文件系统和虚拟内存子系统都有体现。
- **存储对象身份和数据载荷分离。** 内核通过一系列索引（pid、fd、路径，甚至虚拟地址）将内核内部的对象暴露给用户态程序。内核对象的身份（可索引性）和内核数据（可操作性）实际上是独立的。因此，我们将内核中具有类似特性的重量对象分离为 identity 和 payload，前者维护对象之间的拓扑关系，后者维护对象的数据，采用不同的生命周期模型管理，用四类智能指针建模其生命周期语义。
- **通过协作式调度简化系统调用回滚设计。** 内核必须保证系统调用作出的修改在异步事件的打断下的原子性，否则会产生处于不一致状态的对象。我们通过系统调用分步设计，规定阻塞和失败只能在资源准备和权限鉴定阶段发生，一旦进入修改状态系统调用不允许阻塞和失败，一旦发起必须成功。这极大简化了同步内核中系统调用失败路径的回滚逻辑。
- **系统调用不依赖隐式全局上下文（Linux `current()`）。** 执行时所需上下文（执行权限、oscomp、资源）全部在固定的路径构造传入。

repo 的主要内容如下：

```

.
├─ boards/           # 板级HAL与内核二进制crate (RISC-V/LoongArch·qemu-virt)
├─ crates/           # 架构无关的内核crate
│   ├── tx-drivers/   # virtio、串口/TTY驱动
│   ├── tx-ext4/       # ext4文件系统
│   ├── tx-ext4-format/ # ext4镜像格式化
│   ├── tx-fat/        # FAT文件系统
│   ├── tx-fat-format/ # FAT镜像格式化
│   ├── tx-fs/         # 文件系统框架与tmpfs/procfs/devfs/devpts
│   ├── tx-hal/        # 硬件抽象层
│   ├── tx-kernel/     # 内核装配：启动、trap分发、初始化
│   ├── tx-observe/    # 内核侧观测/追踪运行时
│   ├── tx-observe-types/ # 观测数据类型定义
│   ├── tx-platform-adapter/ # 平台边界适配宏（#[platform_adapter]）
│   ├── tx-policy/     # 调度与cgroup策略
│   └── tx-reactor/    # 无栈协程reactor与调度

```

		tx-scripts/	# 操作流程编排 (drive、进程、挂载、路由)
		tx-services/	# 内核服务: 随机数、凭证、rlimit、时间、trace
		tx-shims/	# Linux系统调用语义与ABI适配
		tx-substrate/	# 基座: 对象池 (zone)、EBR、索引、发布总线、帧、预留
		tx-subsystems/	# 语义子系统: 进程、内存、文件系统、IPC、网络、设备、信号
		tx-test-support/	# 测试支撑: step引擎、宿主驱动
		tx-vdso/	# 编译期vDSO ELF镜像
		docs/	# 设计文档与开发记录
		external/	# 第三方依赖与测试套件 (musl、lmbench、oscomp-autotest等)
		tools/	# 评测、构建与调试脚本 (oscomp-judge、LTP运行器等)
		xtask/	# 统一开发命令 (构建/QEMU/OSComp)

目录

摘要	1
第 1 章 前言：从 Linux 用户态契约到 txKernel substrate	1
1.1 执行模型：future、reactor 与 step outcome	2
1.2 存储模型和并发控制	5
1.3 消息通知	7
1.4 协程下的系统调用组织方式	8
1.5 拆解 Inode	10
1.6 总结：内核的核心抽象	12
第 2 章 Reactor 设计	13
2.1 异步模型简介	13
2.2 Rust 异步接口介绍	13
2.3 reactor 设计	15
2.4 异步线程执行流设计	18
2.5 异步系统调用调度模型	22
2.6 异步系统的调试框架：tx-observe	28
第 3 章 线程、进程管理	29
3.1 概述	29
3.2 进程与线程	30
3.3 进程生命周期	32
第 4 章 文件系统：虚拟文件系统与实例	33
4.1 虚拟文件系统	33
4.2 磁盘文件系统	37
4.3 伪文件系统	38
4.4 页缓存	39
第 5 章 虚拟地址空间管理	40
5.1 概述	40
5.2 虚拟内存区域(recipe)	40
5.3 进程地址空间(AddressSpace)	41
5.4 缺页处理	43
第 6 章 内存分配	45
6.1 概述	45
6.2 页帧分配器	45
6.3 对象池 (Zone)	48
6.4 延迟回收 (EBR)	51
第 7 章 进程间通信	52
7.1 概述	52
7.2 信号	52
7.3 管道与 futex	53
7.4 System V IPC	53
7.5 事件机制	54

第 8 章	设备驱动	54
8.1	概述	54
8.2	设备模型	55
8.3	设备发现	55
8.4	块设备	55
8.5	网络设备	56
8.6	字符设备	56
第 9 章	硬件抽象层	57
9.1	启动	57
9.2	页表管理	59
9.3	中断与异常处理	61
9.4	时钟与定时器	65
9.5	用户态内存管理	65
第 10 章	网络栈	66
10.1	概述	66
10.2	Socket 模型	67
10.3	传输层	69
10.4	网络层	72
10.5	链路层	74
10.6	设备	76
10.7	异步网络 I/O 模型	77
第 11 章	开发流程与团队分工	78
第 12 章	AI 使用声明	79
第 13 章	未来展望	80
第 14 章	参考设计	80

第1章 前言:从Linux用户态契约到txKernel substrate

内核是运行在处理器保护模式下，管理硬件、并负责用户进程访问硬件的软件。此外，它还把硬件能力、并发执行和持久状态，组织成一组用户程序可以稳定依赖的系统接口。设计内核时，我们将内核的架构分为两层：

- 上层是直接面向用户态程序的接口与对象：POSIX 标准 / Linux manpage 规定的进程、文件系统、进程间通信等接口，规定了子系统必须实现的语义。鉴于比赛以 LTP 作为主要测例，子系统在实现自己的语义时将以 Linux 为主要参考。
- 下层提供上层子系统反复依赖的原语——对象的存储与生命周期、执行的推进与等待、等待/通知等通用逻辑。下层的设计具有更大的自由度，并且其提供的基础设施决定上层语义的实现方式。我们的设计主要在这一层展开。

txKernel 是用 rust 语言编写的基于异步、无栈协程的内核。每一个执行流都被组织成一个无栈协程，由 reactor 统一驱动与调度；协程只在显式的挂起点让出，使得内核工作的每一步都在两次挂起之间完整执行。

我们利用异步无栈协程的调度模型和 rust 的类型系统实现了**系统调用执行流和语义的分解**：系统调用提供控制流框架，组合子系统提供的承载具体语义的协程，统一交给 reactor 调度。这给出了一种新的内核设计模型，我们的设计文档也据此展开：

- 前言会介绍我们设计内核时抽象出的核心概念和类型，以及据此构建出的系统调用模型。所有子系统的设计围绕为系统调用提供支持的主轴展开。
- 随后按照同步内核的概念进入各子系统，介绍其在这一框架下的实现：进程、线程、虚拟内存、文件系统、设备、IPC、信号，以及贯穿两层的并发控制与验证。

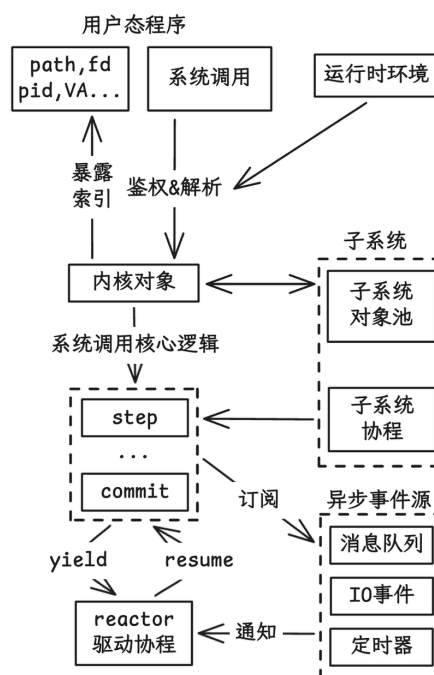


图2 txKernel 的主要架构。左侧是系统调用主轴，规定所有系统调用的通用形状；子系统设计位于右侧，按系统调用规定的接口提供协程和对象访问接口。

需要说明的是，这一套抽象是我们在比赛学习和实现过程中逐步设计的，txKernel 当前实现仍处在迁移过程中。若干路径已经采用了统一结构，也有不少地方仍使用较粗的锁、局部表和传统状态机。本文把这套两层模型描述为架构目标与正在落地的主线，而非声称所有子系统都已完成同等程度的改造。

1.1 执行模型：future、reactor 与 step outcome

txKernel 的执行模型建立在 Rust 标准的异步模式之上。本节先把这套标准模式讲清——它是 Rust 语言与 async 生态共有的东西，与内核无关；再转入 txKernel 在其上构建的执行模型。

1.1.1 Rust 的异步模式

1.1.1.1 执行流：future、poll 与 reactor

异步模式的基本数据对象是 **future**：一段被打包成对象、可以暂停的计算。推进一个 future 的操作称为 **poll**。std::future::Future 只有一个关联类型与一个方法：

```
pub trait Future {
    type Output;
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
}

pub enum Poll<T> {
    Ready(T),    // 完成—结果在此
    Pending,     // 未完成—状态已存入 future, 稍后再 poll
}
```

poll 一个 future，要么返回 Ready(v) 给出最终值，要么返回 Pending 表示尚不能推进，且在未就绪时**不阻塞**。cx: &mut Context<'_> 携带一个 Waker，即 future 可再次推进时用以唤醒当前任务的句柄；返回 Pending 前，future 存下这个 Waker 的克隆，待条件具备，别处对它调用 wake，当前任务即被重新标记为可调度。

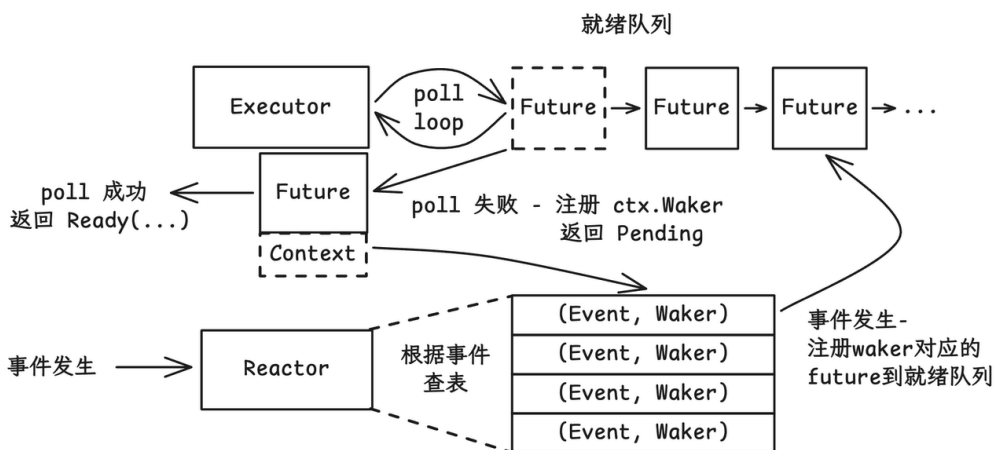


图3 future / reactor / waker 的协作。executor 的 poll 循环逐个推进就绪队列中的 future；poll 成功返回 **Ready**，poll 未就绪则注册 **ctx** 中的 **Waker** 并返回 **Pending**；reactor 收到事件后据其查 **(Event, Waker)** 表，将对应 future 重新注册回就绪队列。

1.1.1.2 无栈模式：future 如何安排暂停的状态

future 取代了同步执行所依赖的调用栈。一个带挂起点的 `async` 函数会被编译器改写成一台状态机——一个 enum，其变体对应各个挂起点，其字段恰好是那些必须跨越挂起点存活的局部变量。考察一段三层嵌套的调用：

```
async fn read_db() -> String {
    // I/O
}

async fn process_user() {
    let user_id = 1001;           // 局部变量 1
    let data = read_db().await;   // 挂起点 1
    let status = true;           // 不跨 await
}

async fn main_task() {
    let start_time = 0;          // 局部变量 A
    process_user().await;        // 挂起点 A
    foo(start_time);
}
```

在同步世界里，`main_task` 调用 `process_user`、`process_user` 调用 `read_db`，是运行时压入的三层栈帧；每一层的局部变量随调用入栈、随返回出栈，栈的深度到运行时才确定。异步世界则相反：编译器在编译期就把这条 `.await` 调用链铺成一个层层内嵌的结构体（图 2-2），`readDbFuture` 内嵌于 `procUserFuture`、后者又内嵌于 `mainTaskFuture`。每一层各带一个 `EnumTag` 标记自己停在哪个挂起点，并把跨挂起点存活的局部变量——`start_time`、`user_id`——作为字段平铺进这块连续内存。

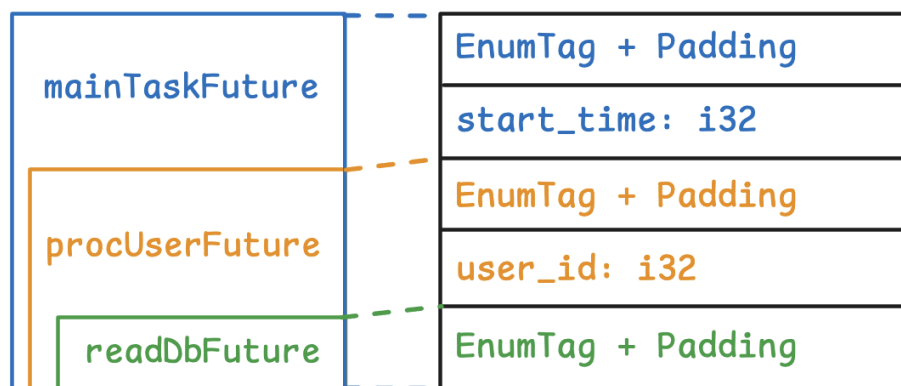


图 4 `main_task` 的 future 结构体布局。三层 `async` 函数生成的 future 层层内嵌为一块连续内存；每层各有一个 `EnumTag` 记录当前挂起点，跨 `.await` 存活的局部变量（`start_time`、`user_id`）成为该层的字段。Future 结构体内只有标记和局部数据，状态记账在 `Executor` 侧完成。

运行时栈帧被编译成了编译期大小已知、布局固定的对象。一个任务暂停时所需保存的全部状态，就是这块内存；poll 它即按 `EnumTag` 推进到下一个变体，暂停它即停在某个变体、连同存活字段一并保留。没有运行时栈增长，也不需要为每个任务预留一段独立的栈。在这套模型下，一个任务的暂停状态收束在一个固定大小的 future 对象里，由 reactor 统一驱动——无栈协程因此不需要为每任务分配固

定大小的内核栈，只需要给每个处理器分配一块运行时栈即可。future 运行时所需要的局部变量会被紧凑的加载到缓存里。

1.1.2 txKernel 的执行模型

Rust 标准库和编译器只提供了 Future trait、async / await 语法、async 函数到状态机的编译，以及 Poll 类型，允许用户根据标准接口实现自己的 reactor 实现调度。txKernel 在 no_std 环境下按语言要求实现了自己的运行时。（生态惯用法把 poll 循环称为 executor、把对接事件源、重新调度的部分称为 reactor；本文不作此区分，将两块统称为 reactor。）

实现运行时只是填上语言留白的一侧。真正属于 txKernel 的，是在这套标准模式之上为内核工作建立的一层执行词汇——它扩展了 Poll 的表达力，并据此把子系统、系统调用与驱动循环分工开来。

1.1.2.1 yield 类型化

Poll 的两种结果对内核工作而言过于粗糙：它仅区分“完成”与“未完成”，而内核中一段工作的“未完成”可有数种不同含义。考察系统调用的实际行为，可见它们在时间上呈现几种典型形状：

- open、close、fork 等对象生命周期调用是**瞬时的**，一步完成或失败，无中间态；
- getdents、sendfile 等批量搬运是**渐进的**，分步完成，但不等待任何条件；
- epoll_wait、futex 等是**纯等待**，无进展可言，其存在意义即挂起至某外部条件成立；
- 管道、socket 上的 read、write 则是**复合的**，先取走现成的字节（这点进展 POSIX 要求如实上报），取完不足再转入等待——进展与等待于一次调用中同时发生。

我们以一个封闭的和类型 StepOutcome 表示这几种形状：

```
enum StepOutcome<T, P> {
    Done(T),                // 终态成功—结果在此
    Err(Errno),             // 终态失败
    Continue { progress: P }, // 有进展、未完成、需再次调用
    Yield {
        progress: P,        // 有进展（亦可能没有），且必须等待
        shape: YieldShape
    },
}
```

在协程两次 yield 之间的同步逻辑，我们用 StepOp trait 表示，类型签名表示协程的执行返回进展和本次执行的结果。Step 之间不允许调用 .await，是一次执行的最小单元。在这个模型下，系统调用的函数调用栈被组织成几段同步 step 的集合，它们返回的 Outcome 由一个 drive 函数集中解释。

```
trait StepOp {
    type Output;
    type Progress: StepProgress;

    fn step(&mut self, ctx: &mut ScriptCtx)
        -> StepOutcome<Self::Output, Self::Progress>;
}

enum StepOutcome<T, P> {
```

```

Continue { progress: P },
Yield { progress: P, shape: YieldShape },
Done(T),
Err(Errno),
}

```

```

async fn drive<S: StepOp>(
    mut op: S,
    ctx: &mut ScriptCtx,
    mode: DriveMode,
) -> Result<S::Output, Errno> {
    let mut acc = S::Progress::EMPTY;

    loop {
        match op.step(ctx) {
            Continue { progress } => {
                acc.extend(progress);
            }
            Yield { progress, shape } => {
                acc.extend(progress);
                match mode.classify(&shape, acc.is_empty()) {
                    Translate(EAGAIN) => return Err(EAGAIN),
                    Translate(PartialReturn) => return Ok(acc.into_output()),
                    Translate(UnsupportedShape) => return Err(EOPNOTSUPP),
                    Resolve => {
                        let resume = resolve_yield(shape).await;
                        op.apply_resume(resume)?;
                    }
                }
            }
            Done(v) => return Ok(v),
            Err(e) => return Err(e),
        }
    }
}

```

1.2 存储模型和并发控制

1.2.1 身份/资源二分

Linux/POSIX 契约要求许多对象具有“部分存活”的状态。一个进程退出后，地址空间、线程和文件表已经释放，但父进程仍然必须能通过 `waitpid` 读取它的退出状态；此时它作为可运行/可发信号目标已经死亡，但作为可回收的子进程身份仍然存在。一个文件被 `unlink` 后，路径名消失，但已经打开的 `fd` 仍然可以读写，直到最后一个引用关闭。类似现象说明：对象的身份和它的资源具有可分离的生命周期。

txKernel 将这一点作为对象模型的核心。我们把对象分成身份和资源两个部分，一个对象可以

- 只保留身份，处在不可操作的状态，只能访问或者 reap
- 或者二者都在，此时处于正常的可操作状态。

我们以进程的拆分为例：进程的 Pid、父子指针、退出状态等被提取为 `processIdentity`，一直存活直到被 reap。剩余的重量资源指针保留在 `processPayload` 里，在进程退出时就被释放。

```
pub struct ProcessIdentity {
    pub pid: Pid,
    pub(crate) parent: ProcessSpinMutex<Option<Weak<ProcessIdentity>>>,
    pub(crate) children: ProcessChildren,
    pub(crate) pgrp: ProcessSpinMutex<Cap<ProcessGroup>>,
    pub(crate) exit_status: ProcessSpinMutex<Option<ExitStatus>>,
    pub(crate) adopted_by_init: AtomicBool,
    pub(crate) payload: ProcessSpinMutex<Option<PayloadCap<ProcessPayload>>>,
}

pub struct ProcessPayload {
    pub vm: AtomicSlot<Cap<AddressSpace>>,
    pub sig_actions: Arc<SigActionTable>,
    pub(crate) threads: ProcessThreads,
    pub(crate) group_pending: PendingSignalQueue,
    pub(crate) siginfo_slots: SigInfoSlots,
    // ...
}
```

拆分后，POSIX 的进程语义就可以由进程身份和资源的分步回收表示，无需专门的字段表示状态。

```
running / runnable / sleeping
    ProcessIdentity.payload = Some(ProcessPayload)
    ProcessPayload.threads 中至少存在 live thread

zombie
    ProcessIdentity.payload = None
    ProcessIdentity.exit_status = Some(...)
    parent.children 仍然保留该 identity

reaped
    parent.children 删除该 identity
    identity 被回收
```

拆分后的内核对象具有不同的访问模式。独立出来的身份侧读多写少，我们设计时统一采用无锁的基于 epoch 的回收 (Epoch-based Reclamation) 的方法管理 Identity 的回收，从而减少只读路径的并发占用。高频的查询路径不会直接增加资源侧的原子引用计数，这也减少了并发访问场景下的开销。

1.2.2 不同强度的 index

为了避免并发常见的 TOCTOU bug，我们规定所有**线程被唤醒后必须重新检查或解析自己所具有的资源**，避免被唤醒后访问到垃圾数据。由此我们在二分的基础上设计四种类型，表达线程对资源的不同占有情况：

并发类（由 EBR 支持，不保留任何东西，是“看”的引用）：

- `Weak<T>` —— 只承诺“它曾经指向某个实体”。它所指的实体可能已经被回收。要使用它，必须在一个 epoch guard 下**观察**它，观察要么产出有效索引，要么报告该槽位已不再指向同一实体。
- `IdentRef<'g, T>` —— 观察所产出的东西：一个 guard 作用域内的指针（'g 生命周期把它绑在所取的 guard 上），可安全解引用，因为 guard 在其持有期间推迟了物理回收；不能被跨 yield 持有。

保留类（引用计数，`'static`，可存储、可跨线程、可跨 await）：

- `Cap<T>` —— 占用 Identity，在自己被回收前不允许 Identity 被回收。
- `T::OperationalEvidence` —— 进一步占用 Payload，跨 await 保存两个部分。

todo：详细描述这里的内容

四个类型由弱到强，通过类型系统强制子系统执行我们的并发纪律。在一次 `StepOp` 内部，task 必须重新检测自己持有的 `Weak` 索引，得到世纪的

1.3 消息通知

1.3.1 事件通知的需求

在现代操作系统内核中，无论是处理网络 Socket 数据、文件系统状态变化，还是进程间通信（如管道），其核心都面临一个基础的 M:N 同步问题：

- **事件源 (M)**：随时可能发生状态转变（如“有数据可读”、“缓冲区可写”）。
- **监听者 (N)**：多个用户态进程或内核 Task 需要挂起等待这些状态转变，并在条件满足时被唤醒。

Linux 通常依靠 **等待队列 (Wait Queues)** 结合 `epoll` 来实现这一语义。对象（如 Inode、Socket）内部维护等待队列头，Task 将自己挂载上去后进入睡眠。然而，这种模式在现代高并发和异步场景下存在一定局限：

1. **生命周期管理**：等待队列的节点通常直接与进程调度实体绑定，生命周期管理复杂。如果 Task 意外被 Kill，必须非常小心地清理其挂载在所有对象上的等待节点，否则会导致内核 Panic。
2. **唤醒管理**：在 `epoll` 中，为了实现多对多的监听分发，内核需要维护复杂的内部红黑树和双向链表，以防止惊群效应和事件丢失。

1.3.2 参考模型：BSD knote

当用户态程序调用 `kevent()` 注册一个监控事件时，内核会动态分配一个 `struct knote` 结构体。这个结构体是 M:N 订阅关系中的最小单元，它同时挂载在两个地方：

- **挂载点 A（面向事件源 M）**：`klist` 在 BSD 内核中，几乎所有可以被监控的对象（比如 Socket 的接收缓冲区 `so_rcv`、代表文件的 `vnode`、代表进程的 `proc` 结构），内部都有一个名为 `klist` 的链表头。这个链表上挂着所有对其感兴趣的 `knote`。
- **挂载点 B（面向订阅者 N）**：`kqueue` 每个 `kqueue` 实例内部维护着自己拥有的所有 `knote`（通常是一个哈希表或者线性数组），以及一个**活跃队列 (Active Queue)**。

每个 `knote` 在创建时，都会绑定一个特定的**过滤器（Filter/mask）**，比如 `EVFILT_READ`（读就绪）、`EVFILT_WRITE`（写就绪）、`EVFILT_VNODE`（文件系统变化）。当事件发生时，内核遍历对象上的 `klist` 链表，调用其过滤器函数，如果条件为真就通知订阅者事件就绪。当订阅者被销毁时，相关的 `knote` 会一同被销毁。

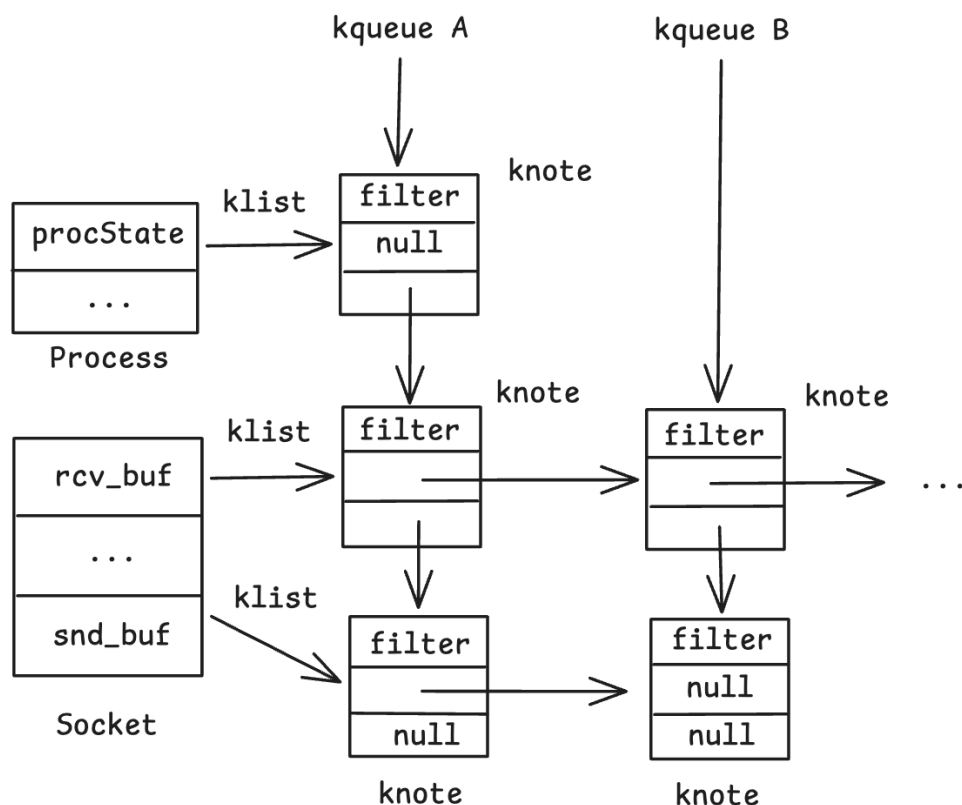


图5 BSD knote/kqueue 架构。knote 实际上表达了 M:N 图中的连接节点。knote 维护横向和纵向两个链表，纵向表示 kqueue A 订阅 socket 的数据可用性和缓冲区可用性，以及进程的状态；横向表示同一个 socket 的数据可用性被多个订阅者订阅。摘自《FreeBSD 操作系统设计与实现（第二版）》中译本 P269，有简化。

1.3.3 TxKernel 的实现

1.4 协程下的系统调用组织方式

1.4.1 TxKernel 的系统调用模型

系统调用是用户态修改内核内部数据结构的唯一入口。以修改内核结构的系统调用为例，用户传入的资源索引被解析成实际的对象，随后内核进行一系列可能阻塞的操作（获取资源，发起 IO 填充缓存），然后完成对象的修改，返回结果。结合我们的对象模型和执行模型，我们系统调用的 step 可以划为清晰的五步：

1. 获取 Epoch guard，根据用户传入的索引寻找对象
2. 增加对象的引用计数，升级智能指针的类型
3. 等待资源锁、申请内存资源、发起 IO，期间可能导致失败、阻塞。如果阻塞，注册对资源的需求后返回 `StepOp` 重新调度。

4. 在修改所需的资源就绪后，内核全部的操作都可以在内存里完成，或者一次性的发起 IO 请求。此时所有的修改操作都集中在一次 `Step` 里完成，不允许调度到其他 task 和失败，必须完成修改。修改完成后，返回结果。

协程的调度模型允许我们简化系统调用的控制流：两次 `yield` 之间的代码类似数据库中事务，通过前置阻塞和可能的失败点，我们避免在修改的逻辑中插入大量清理的代码。发起 IO 后内存资源的生命周期由对应的子系统管理，修改步骤只负责发起 IO。

在同步抢占模型里，处理同步调用和异步信号(`SIGKILL`)之间的竞态是相当复杂的问题。Linux 通过在代码里插入百余处 `fatal_signal_pending()` 检查类似信号，返回 `EINTR`。我们的设计简化了信号处理的逻辑：当 `SIGKILL` 到来时，协程要么只持有资源的引用计数，要么已经完成修改，Reactor 可以直接回收 task 的资源，无需沿着调用栈重新调度，清理不同栈帧上的资源。当 future 被回收时，RAII 会自动处理局部资源的回收。当 `SIGKILL` 到来时，调度器只需在外部边界直接销毁该状态机，利用 RAII 完成资源析构。系统调用内部的 `StepOp` 永远不需要编写用于响应外部信号的探测与回滚代码。

以 `fork` 为例，系统调用的几步大致如下：从进程获取资源，增加引用计数，申请内存资源，然后提交。在提交区域内的调用不允许出错返回(?)。这样的拆解也有助于系统调用专注于语义，让子系统处理内部的调用

```
fn step_fork(parent: &ThreadContext, flags: CloneFlags) -> StepOutcome<Pid> {
    let guard = epoch::guard();

    // observe
    let proc_w = proc::checks::require_fork_permitted(parent, flags, &guard)?;
    let vm_w    = vm::checks::require_address_space_clonable(parent, flags,
&guard)?;
    let fd_w    = fd::checks::require_fd_table_clonable(parent, flags, &guard)?;

    // upgrade
    let proc_cap = proc_w.upgrade()?;
    let vm_cap   = vm_w.upgrade()?;
    let fd_cap   = fd_w.upgrade()?;

    // reserve
    let proc_r = proc::execution::fork_reserve(proc_cap, flags)?;
    let vm_r   = vm::execution::fork_reserve(vm_cap, flags)?;
    let fd_r   = fd::execution::fork_reserve(fd_cap, flags)?;

    // 后面提交依赖的所有变量存储在此，释放时统一RAII
    let bundle = ForkReservations { proc_r, vm_r, fd_r };

    // commit
    vm::execution::fork_commit(bundle.vm_r);
    fd::execution::fork_commit(bundle.fd_r);
    let pid = proc::execution::fork_commit(bundle.proc_r);

    // publish
```

```
publish_process_birth(pid);

StepOutcome::Done(pid)
}
```

并非所有系统调用都涉及复杂的异步逻辑。我们针对一次完成和只读的系统调用设计了专门的路径：

```
dispatch()
├─ ImmediateSyscall
│   纯粹查询，无需StepOp，以同步函数的形式实现
│   getpid/getuid/umask/times/uname/rt_sigreturn...
│
├─ OneShotStepOp
│   修改内核对象，但只涉及内存操作，不会阻塞
│   drive_oneshot(&mut op, &mut script_ctx)
│   setuid/sigaction/setuid/chdir/close...
│
└─ Full async script
    可能分多次完成（部分read），或者阻塞多次(writev)
    drive(op, &mut script_ctx, mode).await
    read/write/open/futex_wait/poll/execve...
```

1.5 拆解 Inode

1.5.1 通用前端：Rnode

在一切皆文件的哲学下，Linux 有大量的内部结构体需要以路径的形式暴露给用户态程序。inode 作为路径的实际承担者，随着内核的演进承担了许多额外的职责，演化成了尴尬的重型对象。Linux 的 inode 结构体定义长达 100 行(v7.1.1)，至少承担了以下功能：

- 提供路径解析节点(RCU lookup 的主要部分)
- 用于块设备的页缓存容器
- 供伪文件系统使用的专有字段
- 支持动态行为延迟绑定的函数指针表等。

字段冗余非常大。多数现代接口需要的只是通过路径暴露自身的能力，不需要 inode 承担额外的记账逻辑。基于第三部分**身份和数据分离**的思路，我们把 inode 拆分成 Rnode 和 RnodeBacking 两部分，前者承担 VFS pathlookup 所需的元数据，**作为子系统暴露数据结构的通用前端接口**。

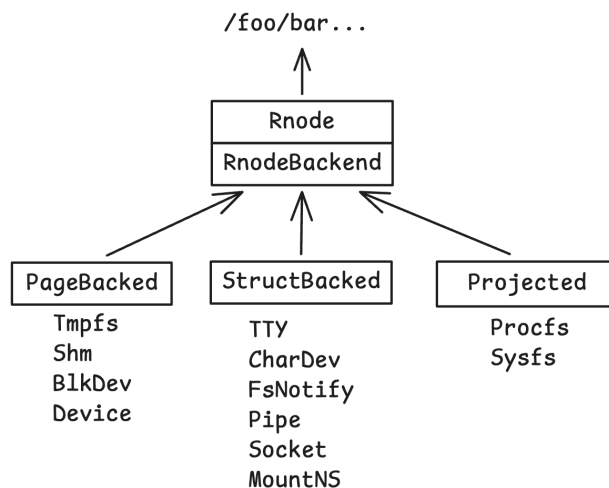


图6 Rnode 的三大后端和支持的子系统。只有左侧的子系统需要用到 inode 作为页缓存容器的功能。

1.5.2 Rnode 的三类后端

前文提到 `AddressSpace` 的页缓存对于伪文件系统和基于结构体的文件完全是多余的。我们用一个和类型 `enum RnodeBacking` 表示所有可能通过路径暴露的对象：

- `PageBacked { pc: Cap<PageContainer> }` 表示所有以 Offset 寻址、以页粒度管理缓存的对象，包含：

- 磁盘上的持久化文件
- 匿名页（匿名 mmap, tmpfs, memfd）
- 设备 MMIO 区域

它们共享着完全一致的、统一的 `step_read`、`step_write` 和 `step_mmap` 步进函数，统一由一个名为 `PageContainer` 的实体负责页缓存的生命周期管理。

- `StructBacked { payload: StructPayload }` 表示一系列不通过 offset 寻址，也不维持页缓存，但仍然通过路径暴露的对象。这类对象大多是不能 `lseek` 的 FIFO 数据流/事件通知对象，并且维护自己的缓冲区：

- 管道
- 行规程(Line discipline)
- 套接字
- `signalfd`，`eventfd` 等现代 poll 接口。

这类对象由对应的子系统结构体（如 `PipeData`，`SocketIdentity`）接管，无需实现页面读写接口。

- `Projected { schema, key }` 为 procfs 等伪文件系统准备，例如
 - `/proc/<pid>/status`
 - `/sys/`

等由子系统对象字段直接生成的文件。它们只持有一套根据对象字段生成元数据的 schema。

考虑到我们实现的功能是 Linux 的有限子集，不需要支持 inode 的延迟绑定实现功能拓展，我们选择用模式匹配建模出所有 `read()` 请求的处理方法，放弃 Rnode 上的函数指针字段。这样我们就不必依赖 `FileOps` 的虚函数表做动态分派，并且利于编译器生成静态的 future 结构体，更不用为每一个文件后端实现几十个方法的 `FileOps` 接口（其中一多半只能填入 `return -EINVAL`）。

```
// 对和类型的match穷尽了所有情况
match &rnode.backing {
  RNodeBacking::PageBacked { pc } => {
    // 统一处理所有基于页缓存的读写，底层再分发到具体存储
    page_backed::step_read(pc, buf, len)
  }
  RNodeBacking::StructBacked { payload } => {
    // 路由给具体的子系统结构（管道、Socket等）处理
    match payload { /* 根据struct类型分派到子系统函数 */ }
  }
  RNodeBacking::Projected { schema, key } => {
    // 动态执行 Schema 生成视图数据
    projection::step_read(schema, key, buf, len)
  }
}
```

在 Linux 将多数内核资源整合为 fd 的大趋势下，提取出 Rnode 作为通用前端不仅简化了文件系统的语义，也减轻了后端子系统的实现负担。Rnode 作为路径适配器，体现了我们对 Linux VFS 接口的思考和设计。VFS 章节会详细介绍 Rnode 和 File 类型的具体实现。

1.6 总结：内核的核心抽象

上面几个部分呈现了我们内核的总体设计，成为每个子系统设计所依赖的通用抽象。整个内核上层围绕内核如何暴露资源和操作接口给用户，以及内核如何解析用户请求所涉及，内核底层围绕异步无栈协程调度模型对异步事件友好、状态确定(future)的特性所设计，并结合类型系统强制子系统遵守内核整体的并发纪律。我们把整个内核的核心设计呈现如图 7：

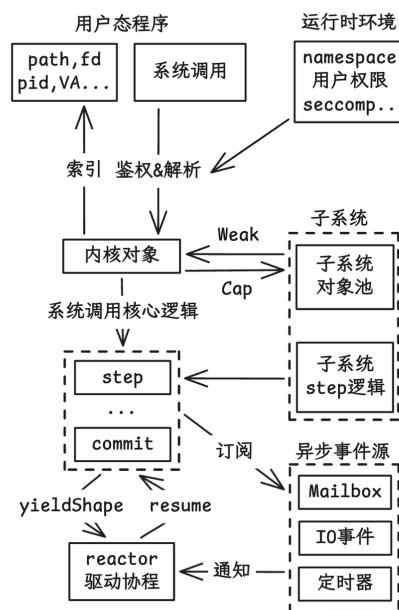


图 7 txKernel 的核心设计。左侧是系统调用的主轴，以对象解析和协程执行为核心；右侧是内核完成 POSIX/Linux 语义的子系统。整个子系统的设计围绕系统调用的工作流程和核心抽象所展开。

对象身份和数据分离、子系统提供异步函数由调度器驱动、对象通过四类索引访问、内核对象和用户态表示解耦构成了我们内核的主要特点，这四个抽象贯穿我们所有内核子系统的设计。我们将在后面的章节介绍具体的子系统实现。

第 2 章 Reactor 设计

2.1 异步模型简介

在同步内核里，线程在内核态和用户态之间交替执行。用户态发生中断后，处理器特权级提升，保存用户态上下文并在内核栈上运行中断处理程序直至完成。当进程执行阻塞时，其内核栈停在 `schedule()` 调用点，直到某个 `wake_up()` 调用唤醒。

TxKernel 保留了系统调用执行的基本流程，但使用无栈协程的原语重新实现了进程的阻塞机制。我们借用 Rust 提供的异步生态，用 `Future` 保存线程阻塞时的上下文，给出了同步执行模型的异步实现。

2.2 Rust 异步接口介绍

下面给出 Rust 标准库和编译器提供的 Future 接口的简单介绍。熟悉 future/waker 的读者可以跳过至下一节。

2.2.1 基本接口：Future 和 Poll

在同步线程模型里，一旦线程开始执行便不会停下，直到处理器被中断打断，此时内核将它的运行时上下文压栈，以供将来继续执行。在 Rust 的 future 模型里，一个 future 不会主动开始执行，只有被调用 `Poll` 后才会开始执行，并且在下一次 `yield` 点让出。标准库中接口如下：

```
pub trait Future {
    type Output;
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
}

pub enum Poll<T> { Ready(T), Pending }

pub struct Waker(/* opaque */)

// Waker使用伪代码
fn poll(self: Pin<&mut Self>, cx: &mut Context) -> Poll<T> {
    if self.ready() {
        Poll::Ready(self.take())
    } else {
        self.stash_waker(cx.waker().clone()); // "call me when ready"
        Poll::Pending
    }
}
```

两个类型的性质如下：

1. `poll` 方法立即返回，不会导致调用者阻塞。它要么立即返回计算完成的结果 (`Ready(T)`)，要么返回表示计算尚未完成的子类型(`Pending`)。
2. `Future` 自己管理自己的上下文。每次 `poll` 都让它内部记录的 `yield` 处继续执行，而非从头开始。
3. `Future` 的就绪状态由 `Waker.wake` 方法标记。`Waker` 由 `Context` 携带，如果 `poll` 返回 `pending`，`poll` 会把它的 `waker` 注册到 `reactor` 的唤醒表上。当 `future` 阻塞等待的事件完成后，调用者通过 `Waker.wake` 通知 `reactor` 将 `future` 加入自己的就绪队列。

2.2.2 异步函数状态机： `async fn`

编译器会把异步函数编译为实现了 `future trait` 的结构体。每一处 `await` 的调用点会被编译为一处 `yield` 的状态。结构体里保存跨 `await` 存活的局部变量。以下面在两个管道之间拷贝数据的函数为例，函数的局部状态完全被存在和类型编译后生成的结构体里，不用显示分配栈来动态保存函数执行的上下文。

```
async fn copy_one(src: &Pipe, dst: &Pipe) -> usize {
    let byte = src.read_one().await;    // await A
    dst.write_one(byte).await;          // await B
    1
}

//编译器大致生成下面的结构体:
enum CopyOne {
    Start { src, dst },
    AfterRead { dst, read_fut: ReadOne },    // 在A挂起
    AfterWrite { write_fut: WriteOne },      // 在B挂起
    Done,
}

impl Future for CopyOne {
    type Output = usize;
    fn poll(self: Pin<&mut Self>, cx: &mut Context) -> Poll<usize> {
        loop {
            match self.state {
                Start { .. }    => { /* 开始执行，到src读取完毕 */ }
                AfterRead { .. } => match read_fut.poll(cx) {
                    Pending    => return Poll::Pending,    // 还在A处
                    Ready(byte) => { /* 开始写入，至dst写入完毕 */ }
                },
                AfterWrite { .. } => match write_fut.poll(cx) {
                    Pending => return Poll::Pending,        // 还在B处
                    Ready(()) => { self.state = Done; return Poll::Ready(1); }
                },
                Done => unreachable!(),
            }
        }
    }
}
```

`Reactor` 的职责便是不断取出就绪的 `future`，调用其 `poll` 方法直至任务完成。

2.3 reactor 设计

2.3.1 Task：最小调度单元

我们将 future 和调度相关的数据打包成 Task，作为调度的最小单元：

```
// crates/tx-reactor/src/task.rs:21,90,113
type TaskFuture = Pin<Box<dyn Future<Output = ()> + Send + 'static>>;

struct Task {
    future: Option<TaskFuture>,          // 线程
    status: TaskStatus,
    wake_state: Arc<TaskWakeState>,
    mailbox: Arc<TaskMailbox>,          // 线程事件队列
    // ...
}

enum TaskStatus { Runnable, Polling, Parked, Completed, Cancelled }
```

TaskStatus 和 Linux 线程的四类状态是对应的关系：

Linux	txKernel
加入某核心 Runqueue	Runnable
在 CPU 上运行	Polling
TASK_INTERRUPTIBLE / 在 Waitqueue 里挂起	Parked
TASK_DEAD / zombie	Completed / Cancelled

Task 是面向 reactor 的接口，reactor 并不指定 future 的形状。在用户态线程外，网络栈以更加原生的形式创建和使用内核协程，统一包装在 Task 内由 reactor 调度。虽然用户态线程也是用协程实现的，但它在用户态运行时会在时间片耗尽后被抢占，重新调度，下文会详细介绍。

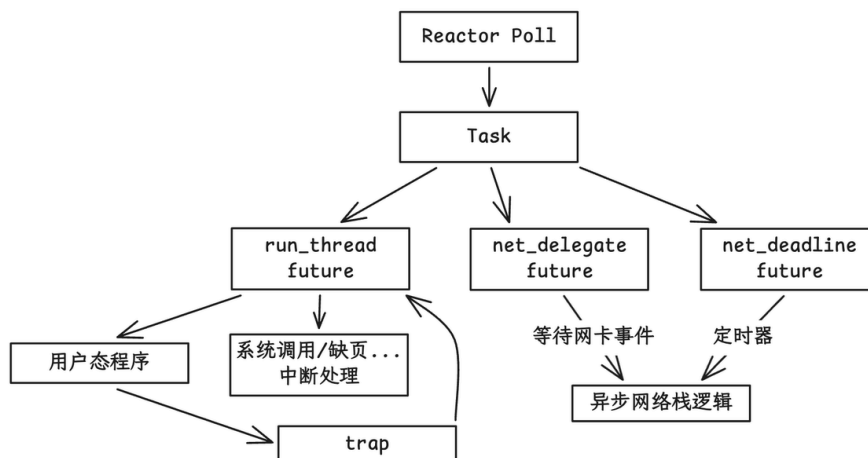


图 8 Task 处理流程图

2.3.2 通知 Task 的消息总线

每一个 future 内部都由我们设计的 Mailbox 完成通用消息订阅。当 Task 执行异步 I/O 并需要等待 (Pending) 时, 它调用 `object.subscribe()`。此时,

1. 外层 future 提取该 Task 的 `Arc<Mailbox>`, 降级生成一个 `Weak<Mailbox>`, 并结合期望的 `interest` (如可读事件) 封装成一个 `Subscriber`, 推入对象的订阅列表中。
2. 该 Task 将控制权交还给 Reactor。

当底层资源状态改变时 (例如硬件中断到达, 数据拷贝入资源缓冲区) 时:

1. 对象遍历其 `Vec<Subscriber>`。
2. 对比 `InterestMask`, 如果匹配, 则尝试升级 `Weak<Mailbox>`。
3. 如果升级成功, 调用 `mailbox.post(SourceFired)`, 将事件推入该 Mailbox 的 `Vec<Events>` 中。
4. Mailbox 接收到事件后, 更新其内部的 `TaskWakeState`, 并通过 `wake_queue.push_back(task_id)` 将所属的 Task 重新投递到 Reactor 的就绪队列中等待轮询 (Poll)。

2.3.3 Task 的调度逻辑

目前 TxKernel 已经支持多核启动。每个处理器核心都维护自己的运行队列, 新 task 被创建时被分配到处理器 `RunQueue` 上。目前我们对 task 的调度处在比较简单的启发式阶段: 将内核 task 和线程 task 分开, 在线程 task 内部按优先级和时间片剩余情况分类。新创建的线程和时间片耗尽的线程会被分到 `new_queue` 内, 有剩余时间片的线程被分配到 `preempted_queue` 内。

调度的优先级如 `pick_next_from_local` 所示。我们判断交互式任务的逻辑是, 一个任务被抢占 (有剩余时间片), 并且被内核事件唤醒。交互式任务和被抢占后等待时间超过阈值的 task 都会在拥有完整时间片的进程之前被调度。

```
// crates/tx-reactor/src/scheduler.rs:357
struct HartRunQueues {
    kernel_queue:    VecDeque<TaskId>,    // 纯内核Task
    boosted_queue:   VecDeque<TaskId>,    // 被高优先级 wake hint 唤醒的 task, 比如
signal/lifecycle
    new_queue:       VecDeque<TaskId>,    // 新提交的用户 task, 或 budget 用完后重新开始的
task
    preempted_queue: VecDeque<TaskId>,    // trap/yield/slice/wake 后仍有时间片的 task
}
//scheduler.rs:1271
fn pick_next_from_local(hart) -> Option<(TaskHandle, SliceConfig)> {
    pop(kernel_queue)
        .or_else(|| pop(boosted_queue))
        .or_else(|| pop_fair_aged_preempted())    // 等待时间超过阈值的被抢占task
        .or_else(|| pop_latency_wake_preempted()) // 交互式task先于正常耗尽时间片的task
调度
        .or_else(|| pop(new_queue))
        .or_else(|| pop(preempted_queue))
}
```

```
// crates/tx-reactor/src/runtime.rs:182,262
struct Reactor { shared: Arc<ReactorShared>, /* per-hart locals */ }

struct ReactorShared {
    tasks: SpinLock<TaskTable>,          // 所有reactor共享的内核task队列
    scheduler: Phase1Scheduler,          // 调度逻辑, 采用策略模式存储
    timers: SpinLock<TimerQueue>,        // deadline wakes (nanosleep, futex timeout)
    // ...
}
```

2.3.4 Reactor 的核心循环

Rust 异步生态里, Reactor 的核心逻辑可以概括为:

```
loop:
    从队列里取出就绪的future
    构建 context 及 waker
    match future.poll(Context::from(waker)):
        Ready(_) -> 计算完毕, drop(futtrue)
        Pending -> 注册waker, future转入等待队列
```

我们的 reactor 代码里的关键路径如下:

```
loop {
    // (1) 唤醒当前处理器上可被唤醒的task
    drain_wakes_for_hart(hart); // runtime.rs:704

    // (2) 尝试运行可运行的task, 没有则窃取; 无法窃取则当前处理器挂起
    let (handle, slice) = match pick_next_or_steal_local(hart) {
        Some(x) => x,
        None    => break,
    };

    // (3) 将task中的future取出, 标记task为polling
    let (key, mut future, wake_state) =
        tasks.lock().take_runnable_future_by_id(handle.id());
    set_current_mailbox(hart, task.mailbox);

    // (4) 构建task下次被唤醒的waker
    let waker = task_waker(wake_state); // waker.rs:49
    let mut cx = Context::from_waker(&waker);

    // (5) 运行task; 这一步就是同步内核的 switch_to() 调用, 运行至下一次yield
    let result = future.as_mut().poll(&mut cx); // runtime.rs:849

    // (6) future返回运行结果
    match result {
```

```

Poll::Ready(()) => {
    tasks.lock().finish_polled_complete(key, future);    // 计算完成，销毁task
    scheduler.task_dropped(key.id());
}
Poll::Pending => match tasks.lock().finish_polled_pending(key, future) {
    Woken { hint } => mark_runnable_from_hart(key, hint, hart), // 重新检查，
防止第五步后唤醒被忽略
    Parked          => task_stopped_local(key.id(), Blocked),    // 将future
所有权归还task
},
}
}

```

2.4 异步线程执行流设计

本章介绍了我们对标准 future 模型的拓展：经典 future 模型是在用户态执行的，并不会触及上下文切换等底层调用。可以说，上面整个章节都是我们为异步模型“函数染色”问题被迫支付的代价：一旦某个函数变成 `async`，那么调用它的所有上层函数都要变成 `async`，一路传染到顶层。这就要求异步系统必须将系统核心循环围绕 future 来组织，从而满足编译器的约束。在完成了上面嵌套 future 的组织后，我们现在可以自由的使用异步函数实现系统调用了。

2.4.1 从内核态进入用户态

本节讲解在 reactor 调度下，我们的线程完整的调度流程，以及内核态-用户态之间的切换是怎么实现的。线程调度设计多个 future，整体架构如图 9：

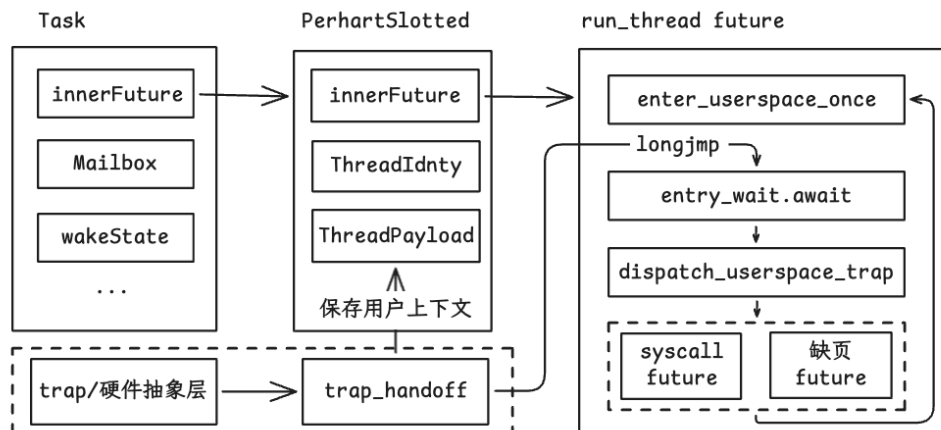


图 9 用户态-内核态切换

图中从左到右分别是 reactor 正在运行的 task，当前处理器上正在运行的线程记账结构体，以及实际的 future 状态机。前两个对象在上面的章节已经介绍，本章讲解 `PerhartSlotted` 和 `run_thread future`。

在内核中，线程主要有三类状态需要维持：

- 线程的执行状态：在内核态/用户态的执行进度，以及在内核态/用户态跳转时的上下文恢复。同步内核用内核栈表示。

- 线程的用户态资源：在发起系统调用时，线程的信号掩码、寄存器快照等数据。当线程运行结束后，这一部分资源不可再被操作。
- 线程的身份：当前线程所属的进程，pthread_join 时的状态。当线程运行结束后，这一部分资源仍可被访问，用于 join 收集退出时状态。

我们把这三类状态分别用 `run_thread` future, `thread_payload` 和 `thread_identity` 表示。选择将后两者拆开是因为他们具有不同的生命周期，并且将线程身份和重量级的资源分开能避免进程遍历线程时 TCB 的无关数据污染缓存。三个结构体设计如下：

```
// crates/tx-kernel/src/thread_future.rs:256
// async会被状态机编译为实现了future的struct
async fn run_thread<P: TxPlatform>(
    thread: Cap<ThreadIdentity>,
    payload: PayloadCap<ThreadPayload>,
) {
    // 伪代码
    loop {
        let entry_wait = payload.userspace_slot().start_request();
        handle_thread_signals(); // 进入用户态前处理信号

        let ctx = payload.build_userspace_context(); // 修改用户态寄存器等内容
        TrapIf::enter_userspace_with_context(&ctx); // 上下文切换至用户态

        // 用户态中断后，trap handler跳转至此，此时payload已保存用户态数据
        // 中断请求被提交给payload
        let trap = entry_wait.await();

        // 根据保存的用户态trap数据做dispatch
        payload.dispatch_userspace_trap(trap).await();
    }
    // 线程终止则退出循环
}

fn enter_userspace_with_context(ctx: &UserTrapContext, root: &PmapRoot) {
    // 启用用户上下文
    let mut frame = Rv64TrapFrame { ... };
    frame.restore_user_context(ctx);

    Platform::activate_user_pmap(root);

    // 保存内核上下文
    let cpu = Platform::current_cpu_id();
    let resume_ctx = current_kernel_resume_ctx_ptr();
    let stack_top = trap_stack_top_for_cpu(cpu);
    tx_rv64_enter_userspace_save_resume(resume_ctx, &frame, stack_top);
}
```

在中断发生后，中断处理函数通过当前处理器 `PerhartSlotted` 的信息找到当前线程的 `payload` 保存数据，其功能类似 Linux 的 `current()` 宏。`PerhartSlotted future` 只是 `run_thread` 的简单包装，作用是在 reactor poll 时将线程信息注册到处理器上，随后便进入 `run_thread` 逻辑中，检查信息后跳转进入用户态。

```
// crates/tx-kernel/src/thread_future.rs:192
struct PerHartSlotted<P, F> {
    thread: Cap<ThreadIdentity>,
    payload: PayloadCap<ThreadPayload>,
    inner: F, // run_thread future
}

impl<P, F: Future> Future for PerHartSlotted<P, F> {
    type Output = F::Output;
    fn poll(self, cx) -> Poll<F::Output> {
        let hart = current_cpu_id();

        // 在当前处理器上登记当前线程payload和idenitiy.
        set_current_thread_identity(hart, self.thread.clone());
        set_current_thread_payload(hart, self.payload.clone());
        if let Some(mailbox) = current_task_mailbox(hart) {
            self.payload.bind_mailbox(downgrade(mailbox)); // wire wakes -> this
task
        }

        let out = self.inner.poll(cx); // run_thread 被调度

        // 当前处理器上的线程运行完毕后，把槽位清空
        clear_current_thread_payload(hart);
        clear_current_thread_identity(hart);
        out
    }
}

// crates/tx-subsystems/src/thread_runtime/structure.rs:140
struct ThreadPayload {
    task: SpinMutex<Option<TaskKey>>, // 自己在task的队列id

    userspace_slot: UserspaceRunSlot, // 当前处理器运行的数据
    active_request: SpinMutex<Option<UserspaceRunRequest>>, // debug校验位

    saved_user_context: SpinMutex<Option<UserTrapContext>>, // 用户态寄存器
    pending_syscall_return: SpinMutex<Option<Result<i64, i32>>>, // 返回时写入a0的内容

    mailbox: SpinMutex<Option<Weak<TaskMailbox>>>, // 唤醒当前future的消息队列
    stopped: AtomicBool, // SIGSTOP?
}
```

```
// ...
// signal fields (signal_mask, thread_pending, signal_summary,
//   saved_signal_context, saved_signal_mask, alt_stack) – see §9
}
```

2.4.2 从用户态返回内核态

本章流程以 RISC-V 架构为例，中断处理部分的详细内容参见硬件抽象层一节。

HAL 层在 trap 处理中会用汇编代码保存用户寄存器，根据 `scause` 对中断进行分类，根据中断类型分派对应的处理函数，保存上下文信息。这里以系统调用处理代码做例子。

```
// crates/tx-kernel/src/trap_handoff.rs:202
fn hand_off_syscall(hart, view, req) -> HandoffOutcome {
    // 1. 取出当前处理器运行线程的thread payload
    let payload = current_payload_for_hart(hart)?;           // None -> NoActivePayload
    let active = payload.active_userspace_request()?;        // 并发debug 校验

    // 2. 保存用户寄存器快照，修改pc跳过ecall
    let mut ctx = view.capture_user_context();
    ctx.pc = ctx.pc + RV64_ECALL_INSN_BYTES;
    payload.store_saved_user_context(Some(ctx));

    // 3. 标记entry_wait为已完成，并把trap的请求打包好提交给wait_entry
    let slot = payload.userspace_slot().clone();
    slot.complete_interesting_trap(active, UserspaceTrapInfo::Syscall(req))?;
    HandoffOutcome::Resolved
}
```

成功后，内核取出在进入用户态前保存的 `kernelResumeCtx`，调用 `tx_rv64_resume_kernel_after_reschedule(ctx)`，通过汇编实现的 `longjmp` 回到 `run_thread` 内部继续执行，中断处理函数结束，从中断处理函数的运行栈回到处理器栈上继续执行 `future`。

```
tx_rv64_resume_kernel_after_reschedule:
    .option push
    .option norelax
    la gp, __global_pointer$
    .option pop

    ld sp,    TX_RV64_RCTX_SP(a0)
    ld ra,    TX_RV64_RCTX_RA(a0)
    ld s0,    (TX_RV64_RCTX_S0 + 0)(a0)
    ld s1,    (TX_RV64_RCTX_S0 + 8)(a0)
    ...
    ld s11,   (TX_RV64_RCTX_S0 + 88)(a0)
    ret
```

```

async fn run_thread<P>(...) {

    TrapIf::enter_userspace_with_context(&ctx);

    // longjmp 回到这里重新继续执行
    let trap = entry_wait.await;
    payload.dispatch_user_trap(trap);
    // loop...
}

```

在返回 future 后，线程根据用户态 trap 保存的内容分派不同的函数处理。用户态线程运行在用户栈上，运行的时间片用完后在内核完成 yield 让出处理器；内核态线程执行在自己的 future 上，采用协作式模型调度。除非用户态运行出现错误，否则不会同步返回，而是将控制交给内核的异步逻辑去处理。

```

// async fn dispatch_user_trap
match trap {
    // 系统调用
    UserspaceTrapInfo::Syscall(req) => {
        // 系统调用同样由异步函数实现，后文介绍
        let result: SyscallResult = dispatch::<P>(req, &ctx).await;
        match result { /* ... */ }
    }

    // 缺页异常，见虚拟内存管理章节
    UserspaceTrapInfo::PageFault(info) => {
        match aspace.fault_script(info.into()).await {
            Ok(()) => { /* 重新运行出错指令 */ }
            Err(_) => { deliver_synchronous_fault(SIGSEGV); return; }
        }
    }

    // 用户时间片耗尽，让reactor调度另一个线程。
    UserspaceTrapInfo::TimerPreempt => { yield_now().await; }

    // 出错，杀死线程
    UserspaceTrapInfo::Fatal(_)      => { deliver_synchronous_fault(SIGSEGV);
return; }
}

```

2.5 异步系统调用调度模型

前面章节介绍了，系统调用被纳入线程的异步执行循环：用户态触发 trap 后，trap shell 只负责保存用户寄存器上下文并完成一次 userspace-run wait；真正的系统调用解析、分派、阻塞等待和结果写回，都发生在 run_thread 这个 thread future 中。

可能阻塞的系统调用不直接操作 reactor，而是把自身表示成一个 StepOp，由 drive() 推进。一次 step() 要么完成，要么失败，要么继续推进，要么返回一个 YieldShape 描述当前等待的对象。

`drive()` 再把这个等待根据需求注册到 `TaskMailbox`、`WaitSource`、`TimerWheel` 上。系统调用流程如图 10:

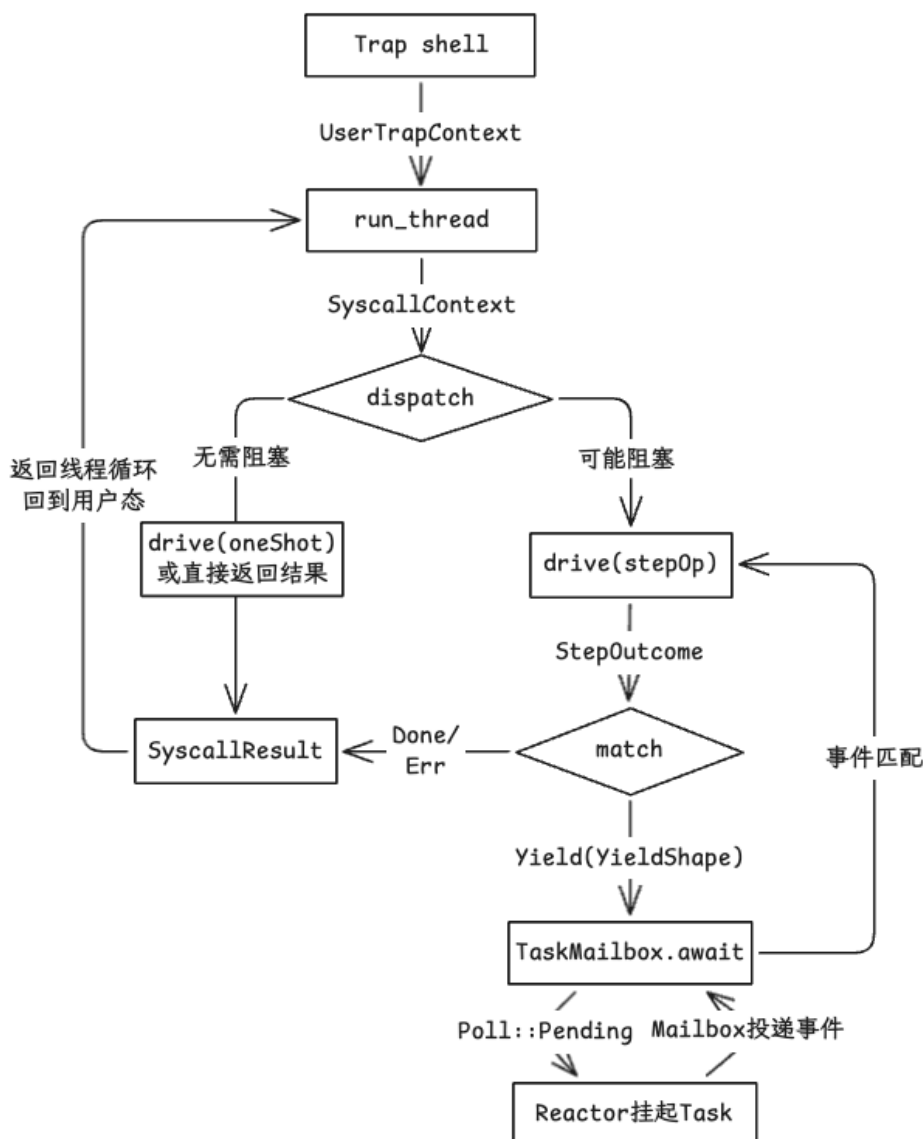


图 10 异步系统调用流程图

2.5.1 系统调用的分类

上一节介绍到 `run_thread` future 的循环里最后一步是 `dispatch_syscall_trap`，线程从这里进入系统调用分派。

```

async fn dispatch_full_syscall<P: TxPlatform>(
    req: SyscallRequest,
    ctx: &SyscallCtx<'_>,
) -> SyscallResult {
    if let Some(result) = dispatch_pthread_hot_oneshot(req, ctx) {
        result
    } else if let Some(result) = dispatch_writev_pagebacked_oneshot(&req, ctx) {

```

```

        result
    } else if req.nr == NR_WRITEV {
        dispatch_writev_hot(req, ctx).await.unwrap()
    } else if matches!(req.nr, NR_MMAP | NR_MPROTECT | NR_MUNMAP) {
        dispatch_vm_hot(req, ctx).await.unwrap()
    } else {
        dispatch::<P>(req, ctx).await
    }
}

```

构造分层的目的，是避免所有系统调用都走最重的 async wait 路径。getpid、getuid、uname、times 等纯查询类系统调用可以立即返回；不会阻塞，只修改内存状态的系统调用可以走 oneshot step；真正可能等待外部事件的系统调用才进入 full async drive()。系统调用框架支持 async，但不是强迫所有系统调用都异步化。

直接返回的同步系统调用模式如下，直接构造系统调用结果返回 run_thread future。

```

fn sys_getpid(ctx: &ImmediateCtx) -> SyscallResult {
    SyscallResult::Return(ctx.process.pid().raw() as i64)
}

```

2.5.2 异步系统调用的调度

我们将可能阻塞的系统调用看成一系列步骤(step)的组合。系统调用在一个步骤执行完后协作式让出。子系统只负责提供每一步运行的纯同步函数，系统调用层负责设置阻塞模式和解析用户传入的 pid, fd 得到实际要修改的内核对象作为上下文，和子系统提供的函数打包交给 drive 函数完成返回值解释、唤醒注册等。

2.5.2.1 四个核心组件：StepOp, StepOutcome, yieldShape, drive

StepOp 是 syscall 内部语义推进的基本单位。它本身不是 async future；它的 step() 是同步函数：

```

pub trait StepOp<I: SubjectIdentity = ProcessIdentity> {
    type Output;
    type Progress: StepProgress;

    fn step(
        &mut self,
        ctx: &mut ScriptCtx<I>,
    ) -> StepOutcome<Self::Output, Self::Progress>;

    fn apply_resume(&mut self, resume: ResumeOutcome) -> Result<(), Errno> {
        ...
    }
}

```

这个设计有两个重要约束。

第一，`step()` 是纯同步的。它只同步观察当前对象状态、尝试推进一次语义转换。如果不能继续，就返回 `Yield`。

第二，`step()` 不持有跨 `yield` 的 `guard`。比如 `OpenFileReadOp` 每次 `step()` 内部重新获取 `epoch guard`：对象状态观察和修改发生在一次同步 `step` 内；一旦要等待，就释放本轮 `guard`，把等待交给 `reactor`。下一次被唤醒后重新 `step()`，重新观察状态。

di

```
fn step(&mut self, _ctx: &mut ScriptCtx<I>) -> StepOutcome<usize, ByteProgress> {
    let guard = step_engine::guard();
    let result = self.file.step_read_with_netns(..., &guard);
    ...
    result
}
```

`StepOutcome` 是 `StepOp::step()` 的返回值，四类结果的含义如下：

- `Continue`：本次 `step` 有进展，但系统调用尚未完成。`drive()` 累积 `progress` 后立刻再次 `step`。
- `Yield`：当前不能继续，需要等待外部事件。`progress` 表示本次 `yield` 前已经完成的部分工作，`shape` 描述等待对象。
- `Done`：系统调用语义完成，返回最终结果。
- `Err`：`syscall` 语义失败，返回 `errno`。

```
pub enum StepOutcome<T, P> {
    Continue { progress: P },
    Yield { progress: P, shape: YieldShape },
    Done(T),
    Err(Errno),
}
```

`YieldShape` 表示系统调用在阻塞时等待的事件：

```
pub enum YieldShape {
    // 等待对象就绪，例如 pipe readable、socket writable、futex wake。
    OnWaitSource {
        source: WaitSourceId,
        interests: InterestMask,
    },
    // 等待另一task以自己的权限执行，为ptrace预留的
    OnAgent {
        endpoint: DelegateEndpoint,
        request: DelegateRequest,
        token: DelegateToken,
        deadline: Deadline,
        cancel: AgentCancelPolicy,
    },
    // 等待计时器
```

```

OnTimer {
    token: TimerId,
    deadline: Deadline,
},
// 等待epoll子系统实现的通知
OnEdge {
    source: WaitSourceId,
    interests: InterestMask,
},
}

```

这是我们异步模型简化阻塞等待的关键：阻塞对象只需要拥有一个 wait source，并在状态变化时发布 readiness。syscall driver 看到 `YieldShape` 后负责注册等待和恢复执行。

drive 是每一个异步系统调用调用的通用接口，负责解释 StepOp 产生的所有控制流请求，核心功能是根据 yieldOutcome 的类型完成订阅。

2.5.2.2 简化的系统调用设计

在四个原语的支持下，我们的 syscall 的控制流逻辑被解耦出来，成为一套统一的模板：系统调用被分为上下两层：上层准备系统调用所需的全部参数，下层用上层准备好的参数逐步执行。

```

pub async fn sys_xxx(args: [u64; 6], ctx: &SyscallCtx<'_>) -> SyscallResult {
    // 1. 解析 Linux ABI 参数
    let parsed = parse_args(args)?;

    // 2. 从 SyscallCtx 构造 StepOp 执行上下文
    let mut script_ctx = build_subject_script_ctx(ctx);

    // 3. 根据 fd flags / syscall flags 选择 drive 模式
    let mode = if nonblocking {
        DriveMode::Nonblocking
    } else {
        DriveMode::Waiting
    };

    // 4. 构造语义操作
    let op = XxxOp { ... };

    // 5. 交给 drive() 推进
    match drive(
        op,
        &mut script_ctx,
        mode,
        script_ctx.mailbox(),
        script_ctx.delegate_registry(),
        script_ctx.timer_wheel(),
    ).await {

```



```

    Ok(value) => SyscallResult::Return(value),
    Err(errno) => SyscallResult::error_from(errno),
}
}

```

我们以 `sys_read` 为例展示这一套流程:

```

pub(super) async fn sys_read<'a, P: tx_hal::TimeIf>(
    args: [u64; 6],
    ctx: &SyscallCtx<'a>,
) -> SyscallResult {
    let fd = args[0] as i32;
    let buf_ptr = args[1] as usize;
    let len = args[2] as usize;

    if fd < 0 {
        return SyscallResult::Error(EBADF_VALUE);
    }

    // 上半: fd → Cap<OpenFile>
    let file = match resolve_fd(&ctx.process, fd as u32) {
        Some(file) => file,
        None => return SyscallResult::Error(EBADF_VALUE),
    };

    // 随后按 backing 分派(io.rs:2385-2489):
    //  posix_mq / ufd / signalfd / eventfd / timerfd / socket /
    //  socketpair / zero 字符设备 / pipe / pagebacked / tty ...
    // 各自走专门的 read 语义。

    let op = OpenFileReadOp {
        file: &file,
        out: &mut staging,
        caller_netns: ctx.process.net_namespace(),
        cursor: 0,
    };
    match drive(
        op,
        &mut script_ctx,
        mode, // Waiting / Nonblocking – 阻塞与否只是一个参数
        mailbox_arc.as_ref(),
        delegate_registry_arc.as_deref(),
        timer_wheel_arc.as_ref(),
    ).await {
        Ok(total) => {
            if total > 0 {

```

```

// 累积完成后,一次性 copy 回用户空间
if let Err(errno) =
    bootstrap_copy_to_user(&ctx.aspace, buf_ptr as u64,
&staging[..total])
{
    return SyscallResult::error_from(errno);
}
}
SyscallResult::Return(total as i64)
}
Err(v3errno) => SyscallResult::error_from(v3errno.into()),
}

```

2.6 异步系统的调试框架：tx-observe

异步系统的追踪和调试是公认的复杂问题。利用我们调度架构里 future 嵌套 poll 的链条，我们实现了一套在不打断内核执行、不引入锁与分配的日志调试框架，并适配 perfetto 框架，支持可视化查看不同 future 执行的时长以及跳转关系。

这套观测的核心想法是，不同级别 future 的 poll 点正好是天然的日志输出点。我们在系统调用的统一入口 dispatch 的不同嵌套深处、poll 轮次发出不同的日志，实现系统调用的自动跟踪。

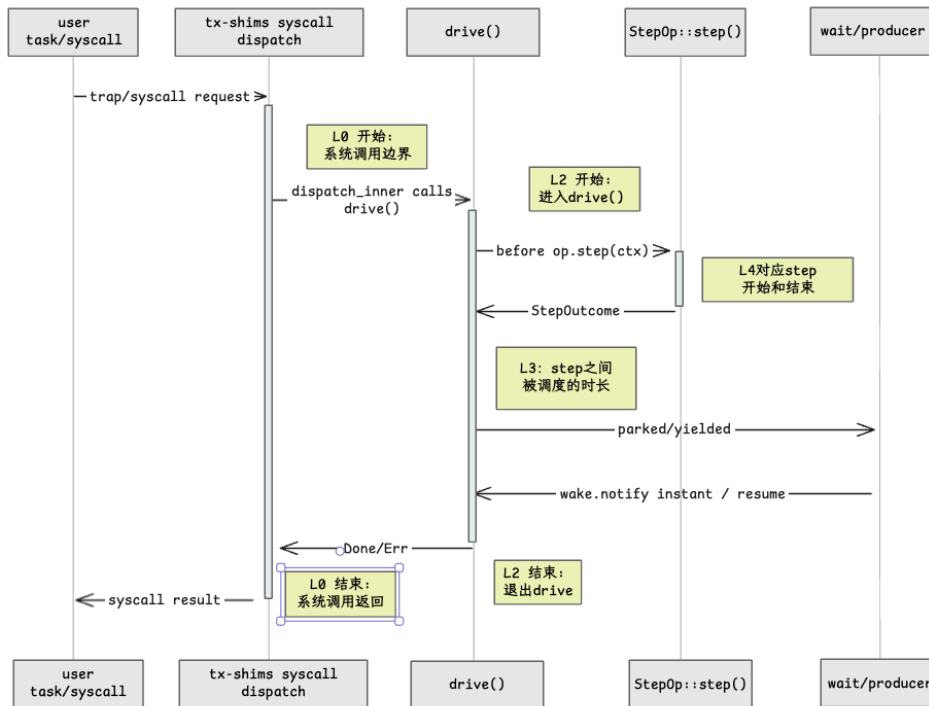


图 11 异步系统调试框架

目前这套框架已经在 QEMU 上成功落地，当内核发送 log 时，将 80 字节的信息写入 QEMU ivshmem 划出的 ring buffer 里，由宿主机读取后解码，转换为 perfetto 或 parquet 格式，也支持用 sql 做信息聚合，做火焰图或百分位查询。在 perfetto 框架生成的 trace 如下图 12：



图 12 调试生成示例

第 3 章 线程、进程管理

3.1 概述

txKernel 的进程管理实现了进程与线程的创建(fork / clone)、程序加载(exec)、退出与回收(exit / wait)、以及多核上的调度执行。

它在对象模型上与多数 rCore 系内核明显不同。Chronix、Del0n1x 都沿用 Linux 风格,用一个统一的 `TaskControlBlock` 同时表示进程和线程,靠共享字段区分二者;txKernel 则把进程和线程各自拆成“身份(identity)”与“负载(payload)”两个独立对象——`ProcessIdentity / ProcessPayload` 与 `ThreadIdentity / ThreadPayload`。这样做是为了贴合 Linux 里普遍存在的“半存活”语义(见下),让僵尸、回收、exec 这些状态自然落出来,而不必在一个大结构上用标志位硬凑。

上一节已经介绍了线程的执行模型, 本节介绍进程的 pthread 接口和生命周期等内容。

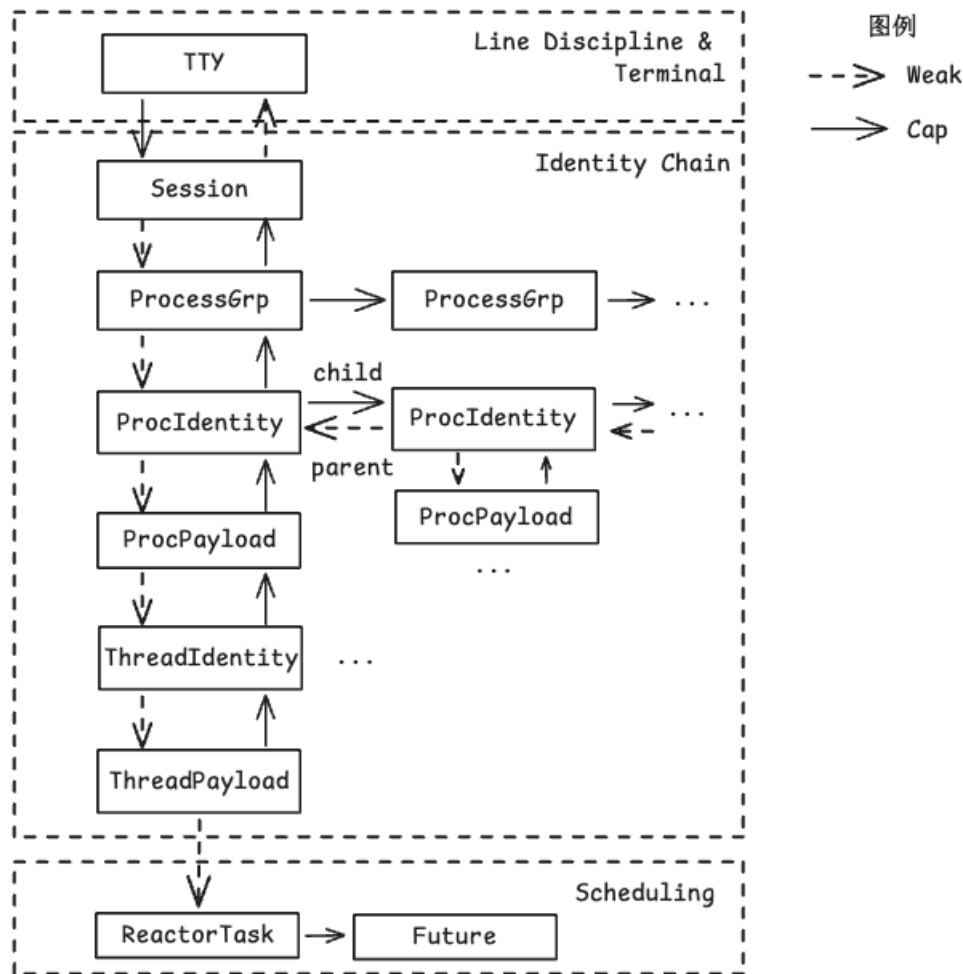


图 13 进程管理整体架构图

3.2 进程与线程

进程是资源分配的单位,线程是调度的单位。一个进程含一个或多个线程,同进程的线程共享该进程负载里的资源(地址空间、文件表、信号处理表等)。txKernel 用四个对象表示它们,核心是 identity / payload 的拆分。

3.2.1 身份与负载的拆分

Linux 语义里,一个对象常常“名字还在、实体已没”:进程退出成僵尸后,它的地址空间和线程都已释放,但父进程仍要能 `wait` 到它的退出码;线程退出后,在被回收前它的 tid 仍可被引用。也就是说,“可寻址的身份”和“可操作的实体”有各自独立的生命周期。

txKernel 据此把进程拆成两个对象: `ProcessIdentity` 承载身份——pid、父子与进程组关系、退出码; `ProcessPayload` 承载实体——地址空间、文件表、凭证等可操作资源。身份通过一个 `PayloadCap` 指向负载;进程退出时先丢弃负载(释放资源),身份则留在父进程的 `children` 列表里,直到 `wait` 回收——僵尸语义就此自然成立。

```
pub struct ProcessIdentity {
    pid: Pid,
```

```

parent:    Mutex<Option<Weak<ProcessIdentity>>>, *// 父进程(弱引用,防环)*
children:  ProcessChildren,                      *// 子进程(保留僵尸)*
pgrp:      Mutex<Cap<ProcessGroup>>,             *// 进程组*
exit_status: Mutex<Option<ExitStatus>>,          *// 退出码(exit 时写,wait 时读)*
payload:   Mutex<Option<PayloadCap<ProcessPayload>>>, *// 负载;exit 时 Some→None*
}

pub struct ProcessPayload {
    frame:    Frame,          *// 地址空间(vm) + 信号处理表(sig_actions)*
    threads:  ProcessThreads, *// 本进程的线程*
    fds:      Mutex<BTreeMap<u32, Cap<OpenFile>>>>, *// 文件描述符表*
    cwd:      Mutex<Option<Cap<DEntry>>>>,          *// 当前工作目录*
    cred:     AtomicSlot<Cap<Cred>>,                *// 凭证(uid/gid...)*
    nsproxy:  AtomicSlot<Cap<NsProxy>>>,            *// 命名空间视图// brk / umask /
    group_pending(进程级信号) / 退出协调 ...*
}

```

线程同样拆成两半: `ThreadIdentity` 是可寻址的身份(tid、所属进程、退出码), `ThreadPayload` 是可操作的实体(reactor 任务句柄、信号状态、用户态上下文)。线程退出时丢弃 `payload`, `identity` 留到被回收。

```

pub struct ThreadIdentity {
    tid: Tid,
    owner_proc: Weak<ProcessIdentity>,          *// 所属进程(弱引用)*
    exit_status: Mutex<Option<i32>>>,
    payload: Mutex<Option<PayloadCap<ThreadPayload>>>>,
}

pub struct ThreadPayload {
    task: Mutex<Option<TaskKey>>>,              *// reactor 里的任务句柄*
    signal_mask: AtomicU64,                     *// 阻塞信号掩码*
    thread_pending: PendingSignalQueue,         *// 线程级待处理信号*
    userspace_slot: UserspaceRunSlot,           *// 用户态运行请求*
    saved_user_context: Mutex<Option<UserTrapContext>>>, *// 保存的用户陷阱上下文// 信号摘要 / 待写回的 syscall 返回值 ...*
}

```

这四个对象都由对象池(Zone)分配、用 `Cap` 引用,身份与负载各自独立回收——正是前面对象 `substrate` 那套 `identity/payload` 模型在进程上的落地。

3.2.2 进程与线程的关系

每个进程的第一个线程是主线程,它的 `tid` 即进程的 `tgid`。线程通过 `owner_proc` 弱引用回指所属进程,进程通过 `payload.threads` 持有它的线程集合。同一进程的所有线程共享进程负载里的资源(地址空间、文件表、信号处理表),因此“创建线程”只是新增一个 `ThreadIdentity / ThreadPayload`、挂进同一个进程负载,而不新建进程对象。

3.3 进程生命周期

txKernel 的进程生命周期实现了 fork / clone(创建)、exec(加载新映像)、exit(退出)、wait(回收)。这些操作都要改动全局状态——pid 表、父子与进程组关系、地址空间——为了在多核下安全、且失败可回滚,它们统一遵循一套五阶段纪律:观察(observe)→升级(upgrade,取强引用)→预留(reserve,占好新对象与名字)→提交(commit,原子生效)→发布(publish,唤醒等待者)。提交之前不改任何对外可见状态,中途失败就丢弃预留、干净退回。

3.3.1 fork 与 clone

fork 创建一个新进程,clone 按标志决定共享什么: CLONE_THREAD 在现有进程里新建一个线程(只增 ThreadIdentity / Payload,不建进程对象),其余标志(CLONE_VM / FILES / SIGHAND 等)决定父子共享哪些资源。以 fork 为例:预留阶段占好新的 ProcessIdentity / Payload、ThreadIdentity / Payload 以及 pid/tid 名字;提交阶段一次性完成——签发这四个对象、在 pid 命名空间登记号码、把子进程挂入父进程的 children、继承父进程的进程组。子进程的地址空间与父进程按写时复制共享(见虚拟内存一章),首次写才分裂。

3.3.2 exec

exec 用新程序映像替换当前进程的执行体,分步进行,并把“不可回滚”的改动尽量推后:

1. 解析与授权:VFS 解析路径、加载 ELF、做权限检查——此阶段可失败、可安全返回错误;
2. 线程组收敛:若进程当前不止一个线程,先让其余线程退出(exec 语义只保留调用线程);
3. 不可回滚替换:构建新地址空间并原子替换 frame.vm,扫描文件表关闭带 FD_CLOEXEC 的 fd,把信号处理表恢复成默认,重置 PC/SP 指向新映像入口;
4. 发布:返回用户态,从新入口开始执行。

3.3.3 exit 与回收

线程退出时写入退出码、(若设置了 clear_child_tid)向用户地址写 0 并 futex 唤醒、置终止标志、丢弃自己的 payload、向 reactor 发任务排空信号。整组退出(exit_group,如 exit 系统调用或致命信号)由一个组退出状态协调:标记其余线程终止并唤醒,待它们都退出后,最后一个线程触发进程退出——丢弃进程 payload(释放地址空间、文件表等),但 ProcessIdentity 连同退出码留在父进程的 children 里。

父进程的 wait / waitpid 遍历 children、找到匹配的僵尸,读出退出码,把它从 children 与进程组里撤销,身份对象至此才被回收;阻塞版的 wait 挂在父进程的“子进程退出”等待源上,子进程僵尸化时被唤醒。若父进程先退出,它存活的子进程被重新挂到 init 名下(reparent),由 init 负责最终回收。这一整套僵尸语义之所以自然,正是因为身份(可寻址、留到被 wait)和负载(实体、退出即释放)从一开始就拆成了两个独立对象。

3.3.4 线程创建 (clone)

创建线程走 clone 的 CLONE_THREAD 分支(step_clone_thread)。与 fork 最大的不同是它全程不新建进程对象:新线程与同伴共享同一个进程负载(地址空间、文件表、信号处理表),只做线程侧的几步:

1. 分配 tid,签发 ThreadIdentity / ThreadPayload,登记 tid;

2. 拷一份父线程的用户上下文,换上调用方指定的新栈与 TLS (RISC-V 用 `tp(x4)`、LoongArch 用 `r2`), 继承父线程的信号掩码;
3. 若带 `CLONE_CHILD_SETTID / CLEAR_TID`,把用户传入的 `ctid` 指针存进 `payload` 的 `clear_child_tid` (运行期也可用 `set_tid_address` 改写);
4. 挂入进程的 `threads` 集合,线程计数加一。

3.3.5 线程退出 (`exit/exit_group`)

单个线程退出(`exit`)走 `step_thread_exit`,按固定顺序收尾:

1. 快照:先取出 `clear_child_tid` 与 `robust list`——紧接着要丢弃 `payload`,这两样必须提前留存;
2. 置僵尸:写入退出码、丢弃线程 `payload`、保留 `ThreadIdentity`,再从 `threads` 集合摘除、线程计数减一;
3. 兑现用户态约定:
 - `clear_child_tid` 协议——向用户地址 `*ctid` 写 0,再对该地址发一次 `futex` 唤醒;这正是 `pthread_join` 能在子线程退出后醒来的底层机制;
 - `robust list`——遍历健壮 `futex` 链表,把每个标记为 `FUTEX_OWNER_DIED` 并唤醒等待者,避免持锁线程猝死引发死锁;
4. 回收 `tid`:主线程的 `tid` 等于 `pid`、留到进程被 `wait` 时随身份一并回收,因此这里只回收非主线程。

整组退出(`exit_group`,或致命信号)走 `step_exit_group`:标记其余线程终止并唤醒,待它们逐个退出后,最后一个退出的线程级联触发进程退出、进程随之僵尸化——接回前面“`exit`与回收”那条路径。

第 4 章 文件系统：虚拟文件系统与实例

4.1 虚拟文件系统

4.1.1 概述

txKernel 的虚拟文件系统(VFS)让一套系统调用(`open / read / write / stat / unlink`等)能驱动多种文件系统。它保留了传统 Unix 的几类对象——文件身份 `RNode`、路径里的名字 `DEntry`、一次打开 `OpenFile`、一处挂载 `MountPayload`——只重构其内部:把“全能 `inode`”拆成 `RNode` 身份加内容后端 `backing` (`RNode` 与三类 `backing` 见前言)。具体文件系统只需实现 `FsOps` (命名空间操作:`lookup / create / unlink / rename / readdir`等)与 `FsPageBacking` (按页读写:`fetch_page / flush_page / truncate / fsync`)两组接口,即可接入 VFS。

txKernel 文件系统自上而下分为四层(图 2-1-1):进程打开文件(`fd` 表与 `OpenFile`)、核心对象(`DEntry` 与 `RNode`,含 `backing` 内容去向)、挂载实例(`MountPayload`)、具体文件系统(`ext4 / tmpfs / procfs / devfs`)。

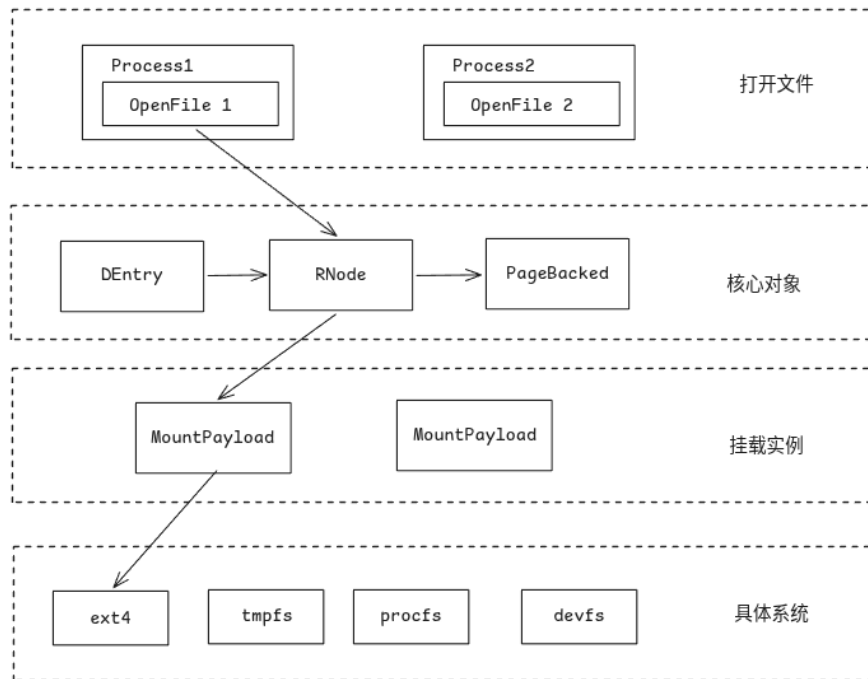


图 14 虚拟文件系统

图 14 简单描述了文件系统的整体工作流程,自底向上看:

- 具体文件系统经挂载接入。一个具体 fs(如根文件系统 ext4)被挂载时,内核为它建一个 MountPayload ——把该 fs 实现的 fs_ops / fs_page_backing 与底层块设备装进去——并建立该 fs 的根 RNode 与根 DEntry,接到目录树的挂载点上。此后这个 fs 就以统一接口对 VFS 可见;系统里每挂一处就多一个 MountPayload (图中挂载实例层的多个框)。
- 路径经 DEntry 树解析到 RNode。给出一个路径后,VFS 从根(或 cwd)的 DEntry 出发逐段下行:每段先查 dcache,未命中则用当前目录的 inode 号调 FsOps::lookup 问对应 fs,拿到子项再建 / 复用 DEntry。DEnter.rnode 指向该名字对应的 RNode (文件身份),走到末段就得到目标文件的 RNode。
- open 把 RNode 变成 fd 上的 OpenFile。解析拿到 RNode 后,open 新建一个 OpenFile,令其 backing = Rnode { rnode } 直接握住这个 RNode,再装进进程 fd 表的一个空 fd。此后该文件既能按路径(DEntry → RNode)、也能按 fd(OpenFile → RNode)到达同一个身份;后续 read / write 走 fd,经 OpenFile 直达 RNode、不再过 DEntry。
- 读写经 RNode 的 backing 落到内容与底层 fs。真正取数据时看 RNode.backing:普通文件是 PageBacked,指向一个页缓存 PageContainer ——命中就直接返回物理页;未命中则经所属 MountPayload 的 fs_page_backing (FsPageBacking::fetch_page)从具体 fs、再到块设备把页读上来、填进缓存。这条 RNode → PageBacked → fs_page_backing → 具体 fs → 块设备 就是一次文件读的完整下行路径(StructBacked / Projected 则分别转给内核对象、读时生成,不走页缓存)。

图中各层对应的关键结构体如下,逐一说明(RNode 与 backing 的拆分思路见前言,此处只给落地结构)。

4.1.2 对象设计

文件身份 `RNode` 与它的内容后端 `backing` 是这套设计的核心,下面先详讲这两个,再简述其余几个对象。

4.1.2.1 `RNode` 与 `backing`

把“全能 `inode`”拆成 `RNode` 身份加内容后端 `backing`, 是 txKernel 文件系统的核心设计(其取舍已在前言论证,这里给落地)。Linux 的 `inode` 随内核演进背了太多职责——路径解析节点、块设备的页缓存容器、伪文件系统专有字段、支持动态扩展的函数指针表等,结构臃肿,而多数接口真正需要的只是“通过路径暴露自身”的能力。txKernel 因此把它拆开: `RNode` 只承担 VFS 路径查找所需的元数据,作为各子系统对外暴露的通用前端;真正的内容交给 `backing`。

```
pub struct RNode {
    fs_object_id: FsObjectId,           // inode 号
    meta: InodeMeta,                   // 不可变的 stat 快照
    backing: RNodeBacking,              // 内容后端:PageBacked /
    StructBacked / Projected
    containing_mount: Option<Weak<MountPayload>>, // 所属挂载
    // wait_points:惰性分配的就绪端点
}
```

`meta` 是不可变快照(元数据变更回后端、再重载); `containing_mount` 关联到它所属的挂载。一个 `RNode` 可被多个 `DEntry` 指向(硬链接),也可被多个 `OpenFile` 指向(同一文件多次打开)。

`backing` 是一个和类型 `RNodeBacking`, 用一个枚举囊括所有“能通过路径暴露的对象”,分三类:

```
pub enum RNodeBacking {
    PageBacked { pc: Cap<PageContainer> }, // 以偏移寻址、按页缓存:磁盘文件 / 匿名页 / 设备 MMIO
    StructBacked { payload: StructPayload }, // 不按偏移、不维持页缓存:管道 / tty / socket / eventfd...
    Projected { schema, key },              // 读取时动态生成:procfs / sysfs
}
```

- `PageBacked`:以偏移寻址、按页管理缓存的对象——磁盘文件、匿名页(`tmpfs`、匿名 `mmap`、`memfd`)、设备 MMIO 区。它们共享同一套 `step_read` / `step_write` / `step_mmap`, 由一个 `PageContainer` 统一管页缓存。
- `StructBacked`:不按偏移寻址、也不维持页缓存、但仍通过路径暴露的对象——多是不能 `lseek` 的 FIFO 数据流或事件对象,如管道、行规程(`tty`)、`socket`、`signalfd`/`eventfd`,由各自子系统的结构接管、自带缓冲。
- `Projected`:为 `procfs`/`sysfs` 准备,内容在读取时按 `schema` 即时生成,本身不存字节。

读写时用一次 `match` 对 `backing` 穷尽分派,放弃了 Linux 那种 `FileOps` 函数指针虚表:这既利于编译器为每条读写路径生成确定的 `future`(无栈协程要求状态机静态可知),也省去了为每个后端实现几十个、其中一多半只能返回 `EINVAL` 的空方法。

4.1.2.2 其他核心对象

- **OpenFile**: 一次 `open` 的结果 (对应 Linux 的 `file`), 由进程 `fd` 表 `ProcessPayload.fds`: `BTreeMap<u32, Cap<OpenFile>>` 持有, 记 `offset`、`flags`, 经 `backing` 直接指向 `RNode` (也可是 `eventfd`/`signalfd` 等非 `inode` 对象)。 `dup` / `fork` 共享同一个 `OpenFile` (同一份偏移, 即 Linux 的同一份 `open file description`), 带 `FD_CLOEXEC` 的 `fd` 在 `exec` 时统一关闭, `RLIMIT_NOFILE` 限定 `fd` 号上界。
- **DEntry**: 路径里的一个名字, 持 `name` / `parent` / `rnode` 与子项缓存 `children` (`dcache`)。它非持久、无单一持有者, 本质是查找用的目录树; 父对子弱引用、子对父强引用, 故缓存不会单独把子树续命, 而任何活节点都能沿强父链回溯到根。 `DEntry.rnode` 即 `name edge`, 与 `OpenFile.backing` 两条边都汇到同一个 `RNode`。
- **MountPayload**: 一个挂载的运行时实例 (相当于超级块), 持 `fs_ops` / `fs_page_backing`、底层块设备与 `fstype`, 直接握着具体 `fs` 的实现; `RNode` 经 `containing_mount` 关联到它, `PageBacked` 文件的页经 `fs_page_backing` 在此取/写回。

4.1.3 路径解析

txKernel 用 `DEntry` 把“路径里的一个名字”缓存成“一个文件身份”, 路径解析就是从根 (或当前工作目录) 出发、按 / 分段沿目录树逐级下行, 直到报错或找到目标 `DEntry`。

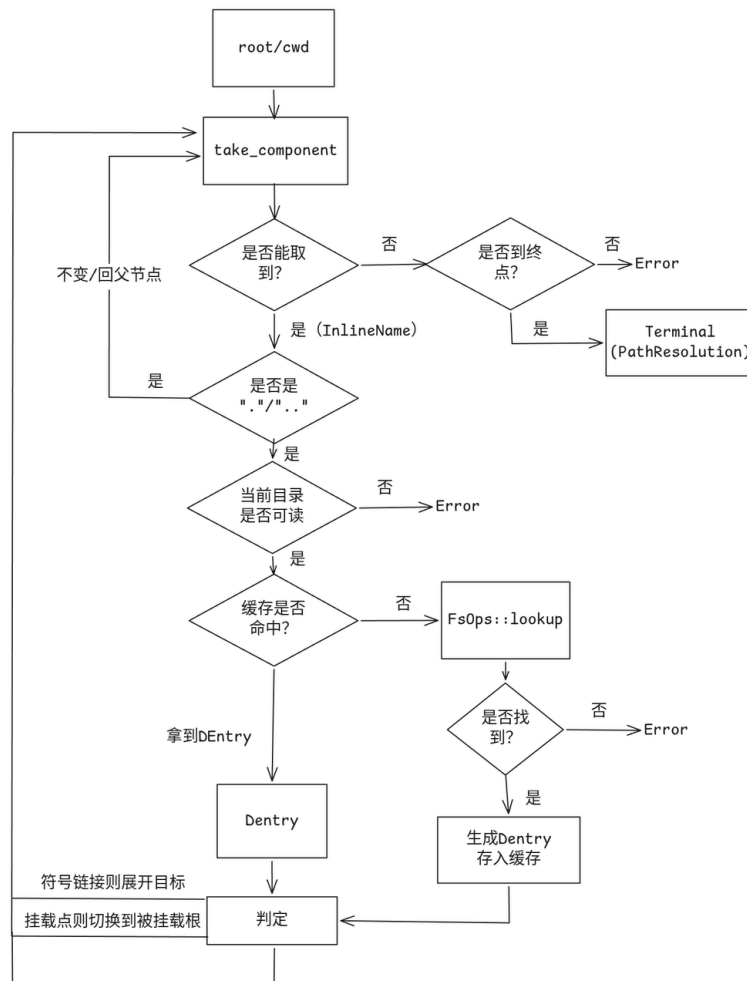


图 15 路径解析状态机

图 15 描述了路径解析过程中一次 `step_walk` 流程。路径解析从/(绝对路径)或 `cwd` (相对路径) 开始, 随后进入一个“取一段、查一段、判定、再取下一段”的循环, 每一轮处理路径中的一段名字。

每一轮先由 `take_component` 从剩余路径取下一段名字。若已取不到(路径走完了), 当前节点便是解析目标, 按调用方式(`WalkMode`, 如要整个文件、或只要父目录加末段名)校验它是否合格: 合格则到达终点 `Terminal`、产出 `PathResolution`, 否则报 `Error`。若取到一段, 先看它是不是 `.` 或 `..` —— `.` 原地不动、`..` 回退到父节点(以本次解析的根为界、不越过)——处理完回到循环开头取下一段; 否则继续往下。

接着在当前目录里查这一段名字: 先确认当前目录可搜索, 否则报 `Error`; 再查目录项缓存 `dcache`, 命中即直接拿到子 `DEntry`, 未命中则向文件系统 `lookup`, 找不到报 `Error` (`ENOENT` 路径错误), 找到则在内存生成子 `DEntry`、存入缓存。

拿到这一段的 `DEntry` 后进入判定, 决定下一步往哪走: 若是符号链接, 则展开其目标; 若是挂载点, 则切换到被挂载文件系统的根; 否则普通前进, 把它作为新的当前节点。三种情形都回到循环开头取下一段。由于解析在调用进程各自的 `mount namespace` 下进行, 同一路径在不同进程可能落到不同的文件系统。最终解析的终点是一个 `RNode` 身份。

4.1.3.1 解析的让出与重入

txKernel 的路径解析与传统 `namei` 最根本的区别在于它如何应对“查一段需要读盘”。传统内核在递归的调用栈上逐级下行, 某一级要从磁盘读目录项时, 整条内核栈就阻塞睡在那里。txKernel 是无栈协程内核、没有每线程内核栈, 做不到也不应这样阻塞。

因此当 `FsOps::lookup` 需要读盘时, `step_walk` 不阻塞, 而是把“当前走到哪一步”——完整的 `WalkingState` ——序列化成一个可保存的档(`ResumeToken`), 让出 CPU 交还调度器; 待磁盘读完, 再据这个档从原地继续往下走。于是整个路径解析是一个可存档、可重建的值, 而不是一条睡在调用栈上的递归——这正是无栈协程内核处理可阻塞路径查找的方式。

```
pub struct WalkingState {
    current: Cap<DEntry>,      // 当前 DEntry(下一段查找的父目录, 或终点)
    remaining: Vec<u8>,        // 尚未消费的剩余路径字节
    hop_count: u32,            // 符号链接跳数(超上限报 ELOOP, 防成环)
    mount_root: Cap<DEntry>,   // 本次解析的根(限制 `..`、绝对符号链接从此重启)
    must_be_directory: bool,   // 原路径是否以 `/` 结尾
}
```

4.2 磁盘文件系统

`ext4`(Fourth Extended Filesystem)是 Linux 上广泛使用的日志型文件系统, 在 `ext3` 之上引入 `extent` 映射、延迟分配与更大容量支持。txKernel 把它作为根文件系统所在的磁盘格式。

与常见的绑定 `lwext4` 这类 C 库不同的是, txKernel 的 `ext4` 是纯 Rust、全程异步的自研实现: 按 `ext4` 公开的 on-disk 格式编解码, 并适配到 txKernel 的 VFS 与页缓存。代码分两层:

- `tx-ext4-format`: 纯磁盘格式层, 不依赖内核——`superblock`、`inode`、`extent` 树、目录项、位图、JBD2 日志的编解码都在这里;
- `tx-ext4`: 内核侧后端, 把上层格式接进 VFS; 挂载产出一个 `MountedExt4`, 对外暴露 `fs_ops()` (命名空间操作)与 `fs_page_backing()` (按页读写)两组接口。

目前支持 ext4 的主要特性: extent 块映射(inode 内联根存 4 个 extent,溢出则下沉到一个 extent 块、最多 340 项)、4KiB 块、inode 与块位图分配、目录的线性与 htree 索引,以及 JBD2 ordered 日志(挂载时回放未落盘的事务)。

在接入 VFS 的方式上,有两条 txKernel 特有的设计约束:

- 每 inode 无状态。后端不缓存任何已解析的 ext4 字段(如 extent 树根):需要时直接从页缓存里的元数据块重新解析。这样页缓存一旦失效便自动重解析,省去了一份“解析结果是否与磁盘一致”的缓存维护;只有通用 POSIX 元数据(InodeMeta :大小、权限、时间戳等)常驻在 RNode 上。
- 不同步阻塞。所有磁盘 I/O 都不就地等待,而是返回“让出”,待块设备就绪再由 reactor 重新推进(契合无栈协程模型,见前言)。于是读一个文件的链路是:VFS 解析到 fs_object_id → 页缓存未命中 → fetch_page 按 extent 算出磁盘块号 → 块设备读出、填入页缓存 → 拷给用户。

块设备接入。ext4 经 MountPayload.backing (Arc<dyn BlockDevice>)读写底层块设备;此外 bdev-fs 把块设备本身暴露成 /dev/vda 这样的 PageBacked 文件,使其可被直接打开读写。块设备驱动(virtio-blk)与统一的 BlockDeviceOps 接口见设备一章。

4.3 伪文件系统

并非所有文件系统都对应磁盘上的数据。txKernel 还实现了一组伪文件系统(pseudo filesystem):它们没有磁盘载体,内容或常驻内存、或在读取时即时合成,主要用于内存存储、进程与系统信息暴露、设备访问。三者与磁盘文件系统走同一套 VFS 接口(FsOps ,需要时加 FsPageBacking),差别只在 RNode 的 backing 与内容来源:tmpfs 的内容在内存、backing 是匿名页缓存;procfs 的内容读取时渲染、backing 是投影(Projected);devfs 的内容来自设备、backing 是结构对象(StructBacked)。

4.3.1 tmpfs

tmpfs 把文件直接放在内存里,没有磁盘 I/O,适合临时文件与缓存数据;共享内存文件系统(shmfs)目前同等看待。每个节点持有元数据与一种载荷,普通文件的载荷就是一个匿名 PageContainer ,因此读写直接走页缓存:

```
enum TmpfsPayload {
    Directory(BTreeMap<InlineName, TmpfsDirEntry>),
    RegularFile { container: Cap<PageContainer>, size: u64 }, *// 匿名页缓存
    *Symlink(Arc<[u8]>),
}
```

tmpfs 的创建很简单:节点只活在内存、且由 DEntry 持有,创建就是新建一个 DEntry 与 RNode 。删除则要支持 unlink-open 语义——文件被 unlink 后,若仍有已打开的 fd,必须存活到最后一个 fd 关闭。做法是把被删名字的 DEntry 置为 Negative(用户从目录里再也查不到它),但 RNode 仍被该 DEntry 持有,于是持有 fd 的进程照样读写,直到最后一个引用消失才回收。以 0_TMPFILE 打开的匿名文件同理:它不挂在任何目录下,只被进程的 fd 表唯一持有。

4.3.2 procfs

procfs 把内核与进程的运行时信息以文件形式暴露在 /proc 下,是系统监控与调试的常用接口;它不占磁盘,内容读取时动态生成、不持久存储。它的 RNode 后备是一个投影

Projected { schema, key } :每次 read 都按 schema 与 key 即时观察当前内核状态、渲染成文本返回 (走 FsOps::step_read_projected),节点以固定 id 编址。目前支持:

- /proc/<pid>/stat :进程状态(running/sleeping/zombie 等)与调度、内存信息;
- /proc/self :指向当前进程的 /proc/<pid> , /proc/self/exe 为其可执行文件的符号链接;
- /proc/meminfo 、 /proc/mounts 、 /proc/cpuinfo 、 /proc/uptime :内存、挂载点、CPU 与运行时长;
- /proc/sys/* 、 /proc/net/* :内核参数与网络信息。

4.3.3 devfs

devfs 在 /dev 下提供设备节点,让用户程序用普通的 open/read/write 与设备交互。它的文件树本身是一张静态注册表, RNode 后备多是结构对象(StructBacked),读写直接转交给对应设备(如 /dev/tty 经 TTY 的行规程处理输入)。目前提供: /dev/null (丢弃写入、读到 EOF)、 /dev/zero (读出全 0)、 /dev/tty (控制终端)、 /dev/rtc (实时时钟)、 /dev/urandom 与 /dev/random (随机数)、 /dev/console 等;此外 bdev-fs 把块设备暴露为 /dev/vda ,使磁盘本身可被直接打开读写。

4.4 页缓存

页缓存把文件内容按页缓存在内存里,读命中省去一次磁盘 I/O,写则先落缓存、标脏、延迟回写。txKernel 没有给每种文件单设缓存,而是用一个 PageContainer 统一三类内容来源——匿名页(tmpfs、匿名 mmap)、磁盘文件、设备 MMIO,由 kind 区分;它按页号把内容索引到一个个 PageCacheEntry ,并用 size_bytes 记录当前文件大小、为读写定界:

```
pub struct PageContainer {
    kind: PageContainerKind,          *// Anon{swap_policy} / File{mount,
fs_object_id} / Device{base_ppn,...}*
    page_count: u64,
    size_bytes: AtomicU64,            *// 当前可见大小*
    state: PageContainerStateCell, *// 页号 → PageCacheEntry 的索引*
}

struct PageCacheEntry {
    ppn: Ppn,                         *// 该页物理页号*
    pin: PageCachePin,                *// 钉住对应物理帧*
    marks: PageMarks,                 *// dirty / writeback / referenced / no_reclaim*
}
```

每个 PageCacheEntry 记一页: ppn 是物理页号, pin 在缓存期间钉住该物理帧不被回收, marks 是脏、回写中、被引用、不可回收等标记。读一页时命中就直接返回 ppn ,缺失则文件页经 FsPageBacking::fetch_page 从磁盘读入、匿名页分配零页后装入缓存;写命中后标脏,脏页不立即落盘,延迟到 fsync 或内存回收时经 flush_page 写回。回收只挑非脏、非回写中的干净页,脏页须先写回——磁盘文件系统靠这套脏页管理维持与磁盘的一致,而 tmpfs 这类只活在内存的匿名页无需落盘。

页缓存同时是文件系统与虚拟内存的交汇点: mmap 一个文件时, VmEntry 指向同一个 PageContainer ,缺页时取到的与 read/write 是同一物理页。因此 MAP_SHARED 下两侧改动互相可见、内存也不重复占用,而 MAP_PRIVATE 在写时才复制出私有页(详见虚拟内存一章)。

第 5 章 虚拟地址空间管理

5.1 概述

txKernel 的虚拟内存管理实现了进程地址空间的建立与切换、`mmap` / `munmap` / `mprotect`、按需分配和写时复制,并让这些操作在多核下并发执行。其核心设计是一条原则:把映射的 `recipes`(VMA 集合)和已落实的 `pmap`(页表)分开。(我们把一个 VMA 称为 `recipe` (`Vmentry`))

每段虚拟地址该映射成什么——地址范围、读写执行权限、内容取自匿名页还是某个文件——都记录在一条 `recipe` 里,它是地址空间的基准;而硬件页表里的每一个页表项都是由 `recipe` 衍生得到的:它之所以存在,是因为有一条 `recipe` 是它的依据。页表并不预先建好,而是在缺页时按 `recipe` 逐页建立(`materialize`);`recipe` 一旦被改动或撤销,由 `recipe` 作为依据的页表项也随之失效。简单说,`recipe` 描述“应当映射什么”,页表只是 `recipe` 在硬件上、按需生成的投影。

据此,一个进程的地址空间 `AddressSpace` 由三部分组成: `recipes` 保存全部映射描述, `pmap` 保存已落实的页表, `range_lock` 协调两者之间的并发——查 `recipe` 无锁(经 EBR),改 `recipe` 与填页表则按地址区间加锁,互不相交的操作可以并行。三者关系如图 16。

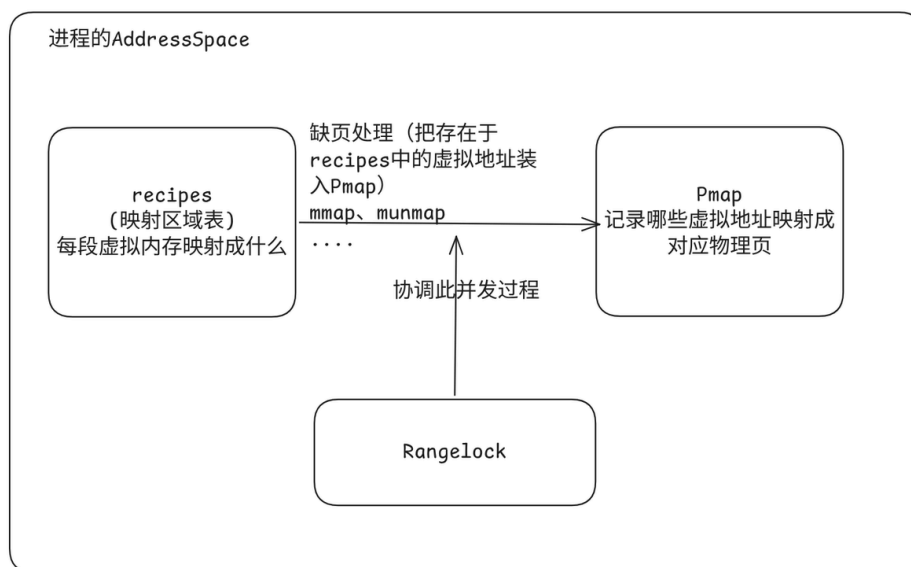


图 16 进程地址空间

地址变换的硬件那一层(页表结构、`PmapIf` 把单页装入或撤出、DMW 与 TLB 重填)属于架构适配部分,本节建立在其上,只讲 VM 层如何组织映射、如何应对缺页;涉及落实到硬件页表处,不再展开。

5.2 虚拟内存区域(recipe)

txKernel 把进程地址空间里的每一段映射描述成一条 `recipe`(对应传统内核的 VMA / 虚拟内存区域):一段连续地址,配上统一的权限和内容来源。一条 `recipe` 只记录这段地址应当映射成什么——内容从哪来、有什么权限——本身不持有物理页。可执行代码、全局数据、栈、堆、`mmap` 的文件、共享内存,各是一条 `recipe`;有了这份描述,缺页时内核才知道该往这一页填什么、`fork` 时才知道该复制什么、栈才能按需增长。

一条 recipe 记录一段连续区域的:地址范围、读写执行权限、若干标志(共享还是私有、是否向下增长等),以及最关键的后备 (backing) 来源——这段地址里的页内容该从哪来。后备 (backing) 分三种: `None` 是纯地址预留(占住地址、访问即非法,用于 `PROT_NONE` 守护页); `PrivateAnon` 是匿名映射,首次写入时才分配一页零页; `Page` 是文件映射,页取自文件的页缓存(`PageContainer`)、带一个文件内偏移。此外,私有映射(`MAP_PRIVATE`)会挂一个私有页集,记录哪些页已因写时复制而私有化。这些后备相关的引用(页缓存容器、私有页集)都放在 `owners` 里。代码里,一条 recipe 就是一个 `VmEntry` :

```
pub struct VmEntry {
    pub range: UserRange,          *// 映射的虚拟地址范围*
    pub prot: Prot,                *// 读 / 写 / 执行权限*
    pub flags: VmEntryFlags,        *// 共享 / 私有 / 向下增长等标志*
    backing: VmEntryBacking,        *// 后备种类(见下)*
    owners: Arc<VmEntryOwners>,    *// 文件页缓存容器 Cap + 私有 CoW 页集 Cap//
    ufd_registration ...*
}

*// 一条 recipe 的后备来源分三种*
pub enum VmBacking {
    None,                          *// 纯地址预留(PROT_NONE 占位)*
    PrivateAnon,                   *// 匿名:首次写时分配零页*
    Page { pc: VmCap<PageContainer>, offset }, *// 文件:页取自页缓存*
}
```

以上是一条 recipe。一个进程会有很多条(每段区域一条),它们作为一个整体由所属的地址空间统一管理——具体怎么组织成一棵可并发查找的范围树,属于地址空间的结构,在下一节介绍。

最后点出和许多内核的关键不同:它们的 VMA 直接持有这段区域的物理页(例如一张“页号 → 页”的表),而 txKernel 的 recipe 不直接持有任何物理页,只持有指向页来源的引用——一个页缓存容器、或一个私有页集。真正的页要么在页缓存里,要么在缺页时才分配、再装入页表。recipe 只是“应当映射什么”;正因为它不掺和具体的页,懒分配、写时复制、共享才能都统一在这一层描述上表达,而不必让区域结构自己管理页的生命周期。

5.3 进程地址空间(AddressSpace)

txKernel 用 `AddressSpace` 表示一个进程的整个地址空间,把三件事分给三个字段: `recipes` 是全部 recipe 的集合(上一节那些 `VmEntry`),记录应当映射什么; `pmap` 是已落实的页表,记录目前映射了什么; `range_lock` 负责多核并发改映射或缺页时的协调。

```
pub struct AddressSpace {
    recipes: RecipeIndex, *// 权威映射描述(应当映射什么)*
    pmap: VmPmap,          *// 已落实的页表(目前映射了什么)*
    range_lock: RangeLock, *// 二者之间的并发协调// mmap 搜索游标 / 统计信息 ...*
}
```

5.3.1 全部 recipe 的集合:无锁范围树

`recipes` 把一个进程的全部 recipe 收在一起,实现为 `RecipeIndex` ——本质是一棵“地址范围 → `VmEntry`”的范围树。它会被缺页路径极频繁地查询(“哪条 recipe 覆盖这个出错地址”),所以我们要把读做成无锁:树做成不可变的持久树,树根挂在一个 `AtomicPtr` 后面。

```
pub(in crate::vm) struct RecipeIndex {
    current: AtomicPtr<RecipeTree>, *// 当前发布的 recipe 树(地址范围 → VmEntry)*
    mutation: VmSpinMutex<()>, *// 仅写者使用的串行锁*
}
```

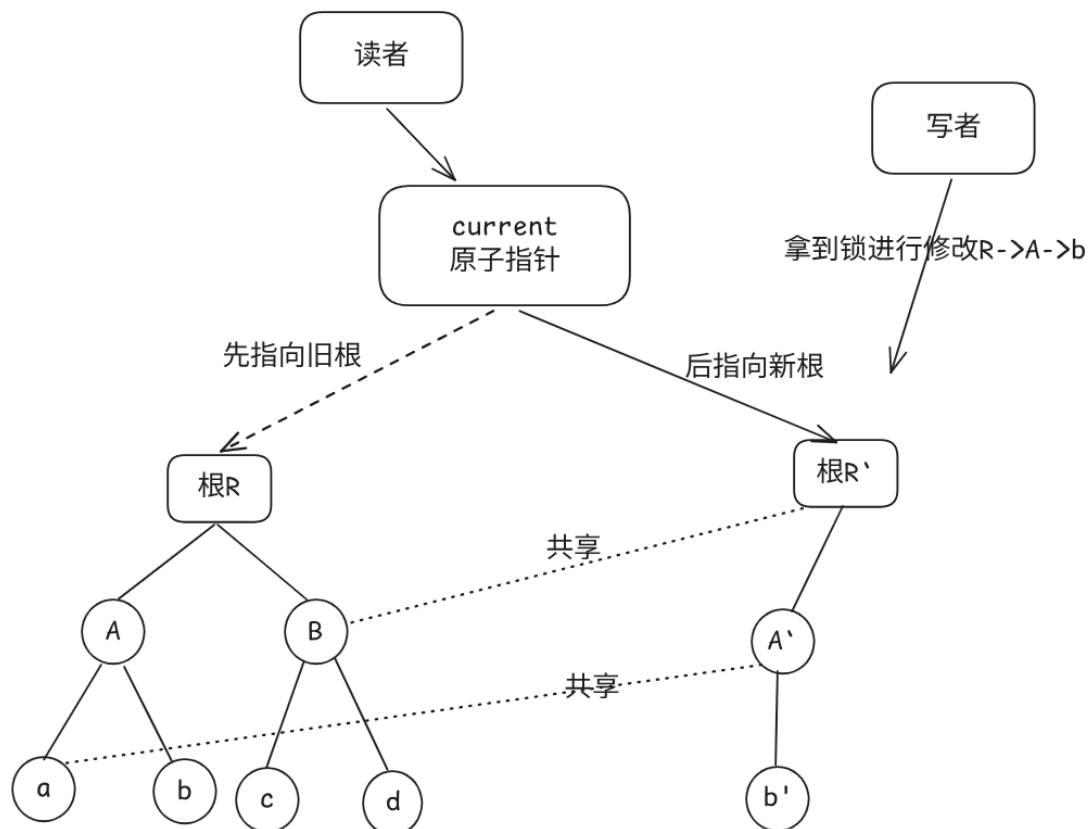


图 17 读者-写者问题处理流程

图 17 展示了我们是如何处理读者与写者问题,其中读者(缺页处理)取一个 EBR: Guard、对 `current` 做一次 `Acquire` 读,就拿到当时的整棵树,全程不加锁;树不可变,读期间绝不会被改写。写者(`mmap` 等)先拿 `mutation` 锁,把受影响的少数节点复制出来拼成一棵新树(其余结点共享),再原子地把树根换成新树,旧树经 `epoch::retire_raw` ——等所有仍在读旧树的读者都离开,EBR 才真正释放它。于是读者要么看到改动前、要么看到改动后的整棵树,绝不会看到改到一半的中间态(读到改动前的状态并不影响结果正确性,后面会再读根一次)。这套“读侧无锁 + EBR 回收”与对象池是同一套模式。

5.3.2 页表 Pmap

`pmap` 持有 HAL 的页表根,以及当前已安装的页表项集合——每一项记物理页号、权限,以及一个对应物理页的 `MapPin`:


```
pub struct VmPmap {
    root: Option<PmapRoot>,          *// HAL 的页表根*
    ops: VmPmapOps,                  *// 平台相关的装入 / 撤出操作*
    state: VmSpinMutex<VmPmapState>, *// 已安装的页表项*
}
```

这个是由 recipe 投影得到, recipe 一旦撤销,对应页表项就要拆除。真正把一页装入或撤出页表,走的是 HAL 的 `PmapIf` (预留一提交、TLB shutdown);页表根的创建与切换也属架构适配部分,这里只简单说明、不展开。

5.3.3 范围锁

为协调缺页时页表项的发布与 `mmap/munmap/mprotect` 对 recipe 的修改,每个地址空间配有一把 `range_lock`。它实现的是细粒度的区间锁:加锁粒度是操作声明的地址区间,而非整个地址空间。锁按操作类型分两种模式:

```
pub enum LockMode {
    ExclusiveWriter, // mmap/munmap/mprotect: 修改 recipe, 独占其声明的区间
    Materializer,    // 缺页: 发布一个页表项, 仅与重叠的 ExclusiveWriter 互斥
}
```

`ExclusiveWriter` 由修改 recipe 的操作持有,独占它声明的那段区间;`Materializer` 由缺页处理持有,只与区间重叠的 `ExclusiveWriter` 互斥。判定冲突只看两个操作声明的地址区间是否重叠,而非它们落在哪条 recipe 上——因此互不重叠的操作可以并发,尤其是任意两个 `Materializer` 永不冲突,不同地址上的缺页得以并行处理。这把锁也保证了:发布一个页表项时,它所依据的那条 recipe 不会被同时更改。

5.4 缺页处理

txKernel 的缺页处理实现了懒分配、写时复制、以及文件映射的按页加载:进程的栈、堆、`mmap` 区域都不预先分配物理页,首次访问才补上(懒分配);`fork` 出的父子进程先共享同一批页、谁写谁才各自复制(写时复制);映射进来的文件按页从页缓存调入。

一次缺页的处理分三步——查 recipe → materialize 一页 → publish 进页表,随后返回重执行出错指令——分别回答:这次访问合不合法、该往这一页填什么、怎么安全装入。

第一步先取一个 `Materializer` 许可,在 EBR 下无锁查找覆盖出错地址的 recipe,并对照它声明的权限——合法性就在这里判定。两种情况算非法、会向进程投递 `SIGSEGV`:没有任何 recipe 覆盖该地址(代码里是 `NoRecipe`),或访问越过了 recipe 的权限(`ProtectionViolation`,例如往只读区域写)。

合法的缺页进入第二步,按 recipe 的后备种类配合访问类型把这一页 materialize 出来——这一步正是懒分配与写时复制发生的地方(下面两小节展开):匿名区域读时给一张共享只读零页、写时分配一页清零的私有帧;文件区域从页缓存取对应页,私有文件映射被写时则复制出私有页。

第三步重新取 `Materializer` 许可、重读校验 recipe 未变之后,才把页表项装上。其中,发布前重校验,是为了挡住中途并发的 `munmap/mprotect`,不让一个已失效的映射被装上;而 materialize 需要等待时(例如读磁盘把文件页调进页缓存),会先放掉范围许可再让出,就绪后重新获取,以免一次缺页锁死整段区间。下面我们用工图 18 展示一下缺页处理的步骤:

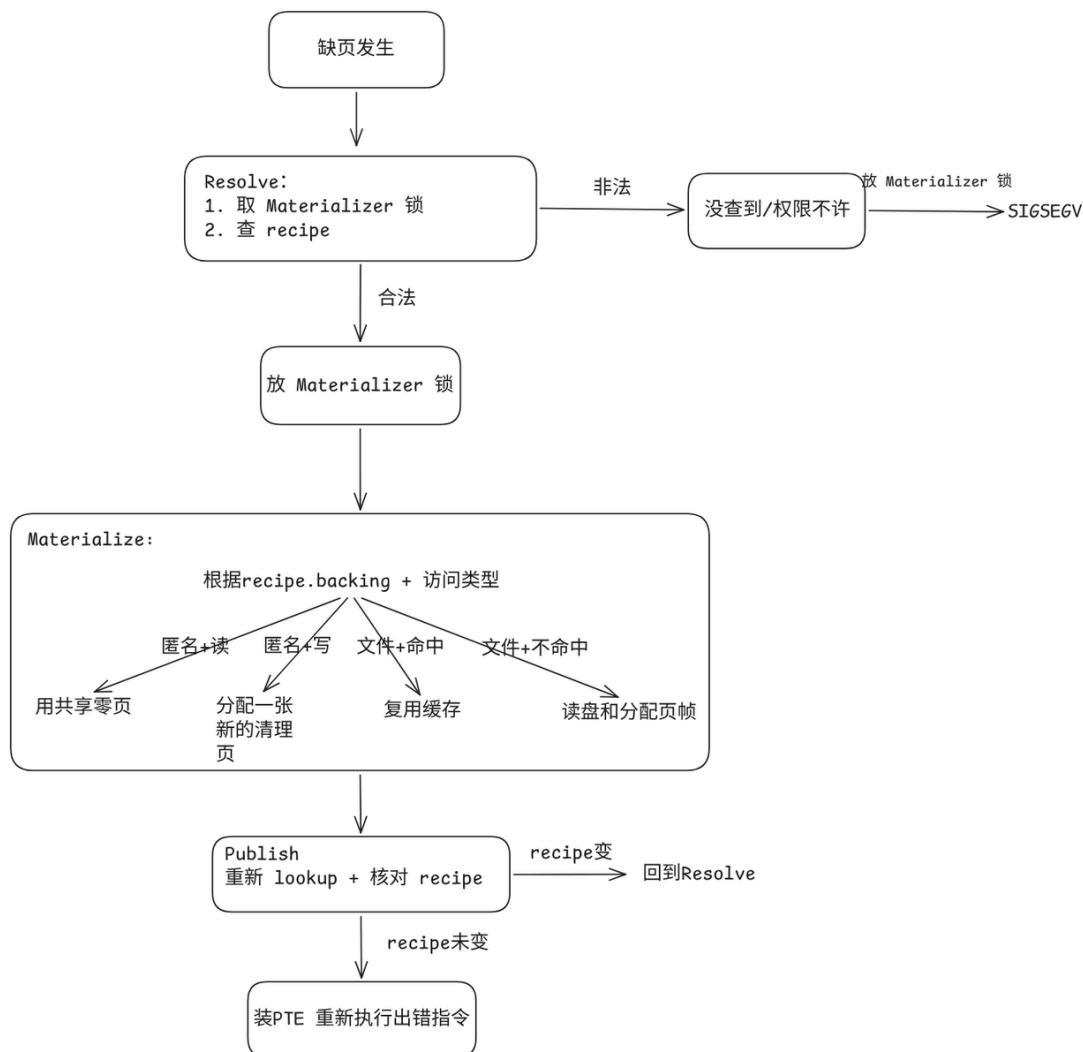


图 18 缺页处理流程图

5.4.1 按需 materialize(懒分配)

recipe 只声明映射、不预先分配物理页。某页被首次访问才触发缺页,这时才 materialize:匿名映射在读时用一张共享只读零页、在写时才分配一页清零的私有帧;文件映射则从页缓存(`PageContainer`)取对应页;随后装入页表。于是栈、堆、`mmap` 区域都只在真正用到时才占用物理内存。作为优化,私有匿名写缺页后会顺带预缺一批相邻页,减少随后的缺页次数。

5.4.2 写时复制(COW)

`fork` 时父子共享同一批物理页、统一降为只读并标记 CoW,不复制任何内容。之后任意一方写入,触发写缺页,内核复制出一页私有副本、把权限改回可写,再放行。每条私有 recipe 用一个私有页集(`PrivatePageSet`)记录哪些页已私有化,从而区分“仍在共享”与“已复制”,写缺页时据此决定直接放行还是先复制。

第 6 章 内存分配

6.1 概述

txKernel 的内存分配部分参考了往届作品和其他开源内核(FreeBSD 等):物理页分配沿用成熟内核的通用做法(位图分配);对象分配(Zone)则主要借鉴 FreeBSD 的 UMA(Universal Memory Allocator)对象池结构,并在其上自研了一套类型化引用与基于 epoch 的无锁回收机制。

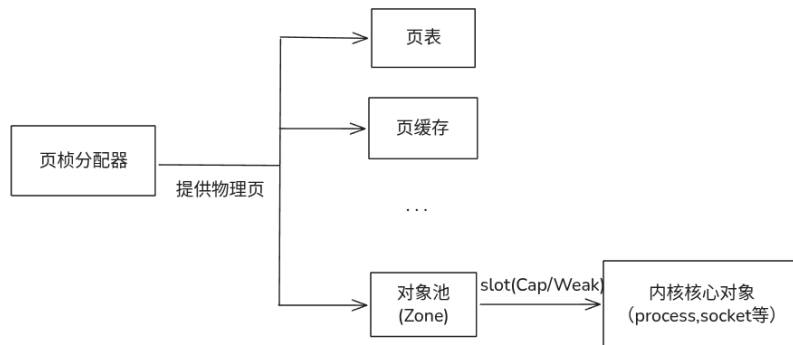


图 19 物理内存分配流程

图 19 是物理内存分配的简图,页帧分配器与 Zone 在下面小节中细讲。页帧分配器是所有物理页的唯一来源,页表映射、页缓存、缺页换页、DMA、内核堆、Zone 需要的页都由他提供。

txKernel 的 Zone 为每种对象类型建立了一个独立的对象池,借鉴了 FreeBSD 的“zone → slab → slot”结构,分层管理,细化每部分的责任。从 Zone 取出对象得到的不是裸内存,而是带强弱之分的类型化引用(Cap/Weak),且对象的身份与资源在槽内即分离存放;读多写少的查找走弱引用 + 无锁观察、不碰引用计数,Zone 是我们内核中的身份/资源二分设计的基础。

6.2 页帧分配器

txKernel 使用 `BitmapPageAllocator` 来管理物理页,它是上层一切分配的源头——页表映射、页缓存,以及内核堆、Zone 等。分配器自己不探测硬件:启动阶段由硬件抽象层(board 从设备树解析)传来两份信息: `BootInfo`, 含一张物理内存图(memory_regions,每段标 Usable/Reserved)以及内核镜像、initrd 等占用的物理区; `BootstrapPmapInfo`, 含内核直接映射的范围(供分配器清零、访问物理页)和启动期页表区。分配器据此划定覆盖区间、扣除保留区与内核镜像、并把元数据与启动页表分为永久页。据此,txKernel 为每个物理页建立初始的两条记录:

- `bitmap` 位标记该页能否分配
- `metas` 元数据记录该页当前被谁占用;可用 RAM 才进 `bitmap`,保留区、MMIO、内核镜像等只有 `metas` 行, `bitmap` 位恒为零、不分出。

`BitmapPageAllocator` 通过 `bitmap` 和 `metas` 来判断页面是否空闲以及页面持有者身份。

```

pub struct BitmapPageAllocator<'a> {
    metas: &'a [FrameMeta], // 每页一行: 被谁占用
    bitmap: &'a [AtomicU64], // 每页一位: 能否分配
    base_ppn: Ppn,           // 覆盖区间的起始物理页号
}
  
```

```

total: usize,           // 覆盖的页数
free: AtomicUsize,      // 当前空闲页数
hint: AtomicUsize,      // 下次从哪开始找空闲位
zero_frame: Option<FrameZeroer>,
}

```

6.2.1 页分配与回收

txKernel 的页分配与回收的流程如图 20:

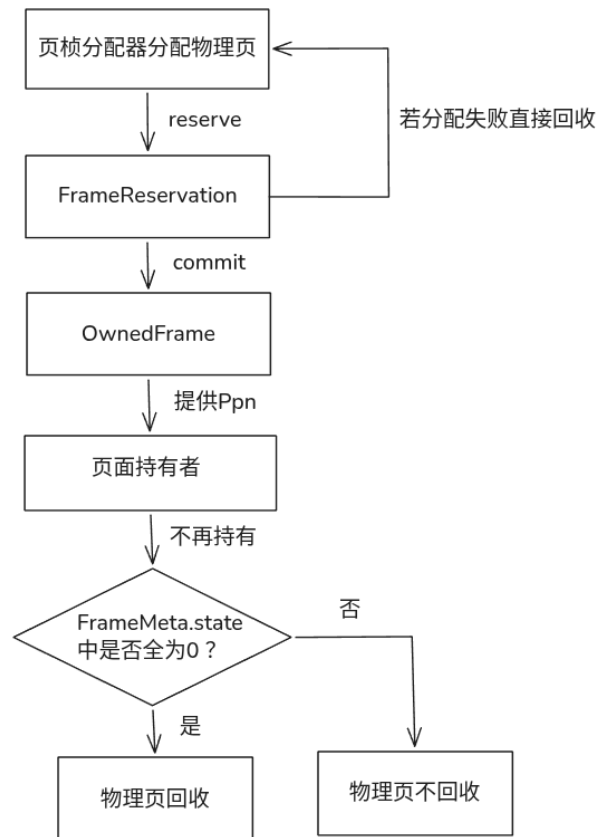


图 20 页分配与回收流程, 页分配整体采用预留(reserve)—提交(commit)两阶段的做法, 支持回滚; 页回收则采用显示回收的方法。

txKernel 的页提取采用了预留(reserve)—提交(commit)两阶段的做法, 中间留出一个可干净回滚的窗口。取页时, `BitmapPageAllocator` 调用 `reserve_frame`, 每次取页都从 `hint` (上次取页成功的位置) 起在 `bitmap` 里找到一个空闲位, 避免每次都从头扫一遍已被占满的低端内存。找到空闲页后, 先按需清零 (交给用户态的匿名页必须清零, 而马上会被整页覆盖的场景则跳过以省开销), 在 `bitmap` 中置位表示该页已被分配, 返回 `FrameReservation`, 但此时但 `metas` 行仍为零。待这一步所需的准备都完成, `commit` 才把 `metas` 的 `refcount` 置 1、将 `FrameReservation` 升成 `OwnedFrame`, 页到此才真正被分配。

两阶段取页主要是为了回滚安全: 一个操作往往需要很多资源, 这些资源需要一次分配完整, 否则就要干净退回, 两阶段的取页使得获取资源失败时可以安全回滚, 避免内存泄漏。

```

// reserve:只写bitmap位, 先占位, 还没有写入该页的占用情况 (metas)
fn reserve_frame(&self, policy: ZeroPolicy) -> Result<FrameReservation<'_, Self>,

```

```

AllocError> {
    let ppn = self.reserve_single_ppn()?; // 从hint起扫bitmap、CAS清位、摘走一页
    if let Err(err) = self.finish_zero_policy(ppn, policy) { // 按需清零;失败即回滚
        self.rollback_reserved(ppn);
        return Err(err);
    }
    Ok(FrameReservation::new(self, ppn))
}
// commit:提交则发布,丢弃则回滚
pub fn commit(mut self) -> OwnedFrame<'a, A> {
    self.allocator.commit_reserved(self.ppn); // 动metas:把refcount置1
    self.committed = true; // 标记已交出→下面Drop不再回滚
    OwnedFrame::new(self.allocator, self.ppn)
}

impl<A: PageAllocator> Drop for FrameReservation<'_, A> {
    fn drop(&mut self) {
        if !self.committed { self.allocator.rollback_reserved(self.ppn); } // 没提交就还回bitmap位
    }
}

```

分配完成后,对象手里拿到的并不是一个长期句柄,而是一个裸 Ppn (物理页号)—— commit 返回的 OwnedFrame 只是构造期的临时凭证,页一旦接进占有者自己的结构,它便被弃掉,占有者只留下这个 Ppn。

之所以不像 rCore 那样“一个句柄(FrameTracker)即一页、drop 即释放”,是因为 txKernel 的一页可被多个角色同时占用——被进程页表映射、被页缓存收纳、被 DMA 钉住,这些角色各自独立地出现和消失。这样的页没有单一所有者(谁先 drop 都不能释放,别人还占着),塞不进一个“drop 即释放”的句柄。于是 txKernel 把页的占用状态集中记进 FrameMeta ——按角色分成几类计数,对象侧只持有裸 Ppn:

```

pub struct FrameMeta {
    state: AtomicU32,
    // bits 0..10  refcount      通用拥有计数
    // bits 10..20  map count    被多少张页表映射
    // bits 20..28  cache count  被多少个页缓存收纳
    // bits 28..32  pin count    DMA / 长期钉住
}

```

因此回收由计数裁定:每个占用者不再占用时只把自己那一类计数减 1,所有 release_* 都进入统一处理程序——唯有四类计数全归零的那一次,才把该页的 bitmap 位放回、重新可分配:

```

fn release_when_zero(&self, ppn: Ppn, state_after: u32) {
    if state_after == 0 { // 四类计数全 0 = 最后一个松手的人
        self.return_to_free_pool(ppn); // 把 bitmap 位放回 (保留页除外)
    }
}

```

6.3 对象池 (Zone)

进程、socket、挂载点这类核心对象创建回收频繁,还要在多核上按身份无锁查找(按 pid 找进程、按路径找挂载点都在系统调用热路径上)。用 Arc 来管它们有两个问题:每查一次都要动一次原子引用计数,在热路径上太贵;而且 Arc 只有“有引用/无引用”两态,表达不了“进程退成僵尸、身份还在实体已没”这类分层的生命周期。

txKernel 参考 FreeBSD 的设计为这类对象设计了类型化对象池 (Zone), 同类对象密集地排在物理页里,分配回收就是占用或让出页里的一个槽;对外只给句柄、不给裸指针。一个对象池自上而下分四层:

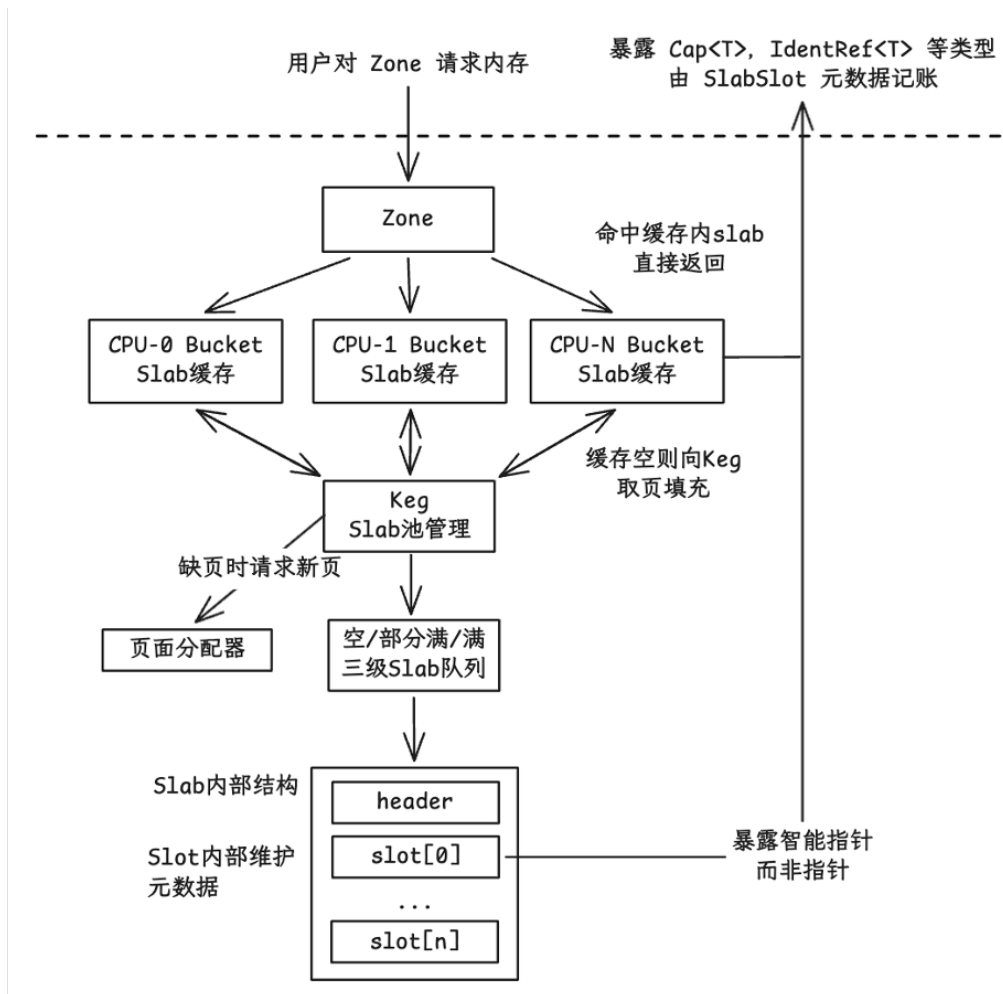


图 21 Zone 的基本结构

图 21 描述了 Zone 的基本结构, 从上到下基本分为四层:

Zone<T>	每种类型一个对象池
├ ZoneBucket	每CPU空槽缓存 (免锁分配)
└ Keg	中心slab管理器 (三链表 + 反查缓存)
├ ZoneSlab	一个物理页 (位图记空闲, 最多64槽)
└ Slot	单个对象槽 (状态字 + 对象)

- Zone 为每个 CPU 配一份空槽缓存 ZoneBucket。分配对象时先从当前 CPU 的 ZoneBucket 取空槽; ZoneBucket 为本 CPU 专属、且 CPU 绑定不迁移,该路径无需加锁。本地缓存耗尽时,再从中心的 Keg 批量补充一批。常态下分配命中本地缓存,锁不进入热路径。

- **Keg** 是某类型全部 slab 页的中心管理者,按占用情况将 slab 归入三条链表——全空、部分占用、全满,分配时优先取部分占用的 slab 以提高页填充率。三种类型的 slab 会随着槽的分配与回收相互转换,全空的 slab 数量超过 2 时回收。句柄到槽的反查在每次句柄解引用、克隆、释放时都会发生、频次很高, **Keg** 为此维护一张 64 项的直接映射缓存(按 slab 编号取模索引、以 slab 自身的单调 id 校验),使反查保持在接近 $O(1)$ 。
- **ZoneSlab** 是一个物理页:页头存放该页元数据(所属 zone、空闲计数等),其后排列一排等大的槽。槽的空闲情况由一个 64 位位图表示,每位对应一个槽,故一页最多容纳 64 个对象。
- **Slot** 由状态字与对象本体两部分组成。状态字是一个原子 **u64**,将槽的状态、强引用计数与代号(generation)合并在一个字中,使分配、发布、回收均可由一条 compare-exchange 完成条件校验与状态更新,无需对单个槽加锁。

Zone<T> 每种类型一个静态实例,持有该类型对象的全部存储,并通过 **ZoneAllocated** trait 将类型 **T** 绑定到唯一的 zone,使分配 **T** 的代码统一落到同一个对象池中。

```
pub struct Zone<T: 'static> {
    keg: Keg<T>,                                // 中心 slab 管理器
    buckets: [ZoneBucket<T>; MAX_ZONE_CPUS],    // 每 CPU 空槽缓存 (免锁)
}

pub struct Keg<T: 'static> {
    partial_head: *mut ZoneSlab<T>,             // 部分占用 (分配优先)
    full_head:    *mut ZoneSlab<T>,             // 全满
    empty_head:   *mut ZoneSlab<T>,             // 全空 (留一张复用)
    slab_cache:   [*mut ZoneSlab<T>; SLAB_CACHE_SIZE], // 句柄反查缓存 (64 项)
}

pub struct ZoneSlab<T: 'static> {
    backing_ppn: Ppn,                            // 支撑的物理帧
    free_bitmap: u64,                             // 每位一个槽,置位 = 空闲
    next: *mut ZoneSlab<T>,                       // 侵入式链表指针
    // id / zone / slot_count / free_count / list ...
}

pub struct Slot<T: 'static> {
    meta: SlotMeta,                               // 状态 + 引用计数 + 代号 (打包进一个原子字)
    value: UnsafeCell<MaybeUninit<T>>,           // 对象本体
}
```

6.3.1 slot 的生命周期

txKernel 的一个 slot 由两部分组成: **SlotMeta**, **value**。 **SlotMeta** 把 **state**、**retain** (强引用计数)、**generation** (代号)打包进一个 **AtomicU64**, 表示该 slot 的状态, 其中 **state** 包含五个状态, 五个状态之间的每一次转换都是一条 compare-exchange——同时校验旧态、写入新态, 无需对单槽加锁:

```
enum SlotState {
    Free,    // 空槽, 可分配
```



```

Reserved, // 已占位,尚未发布
Live,     // 已发布,可被 Cap 引用
Dead,     // 末个 Cap 已弃,挡新引用,待交 EBR
Retiring, // 已交 EBR,待宽限期后回收
}

```

Slot 的分配采用与页分配器相同的思路,走 reserve 和 commit 两阶段,以处理回滚问题。slot 分配时,取一个空槽(本 CPU ZoneBucket 免锁取 → 空了找 Keg → 没有可复用 slab 才向页分配器要新页),reserve 把 slot 置 Reserved (占住、尚未发布;后续准备失败时 ZoneReservation 的析构会把它退回 Free),写入对象后 sign 才发布为 Live、置 retain=1,表示该 slot 已分配完成。

当一个 Slot 的最后一个 Cap drop 时,表示改 slot 的资源需要被回收,进入 slot 的回收程序。当 slot 的 retain=1 并且其又有一个 Cap 要 drop 时,将 retain 设置为 RETAIN_SENTINEL_DEAD (哨兵),并将状态从 Live 转换到 Dead,此时所有对于该对象的 Weak::upgrade 都会失败。之所以将 retain 设置为哨兵,而不是直接归零,是因为给对象增加 Cap 的操作都需调用 inc_retain,直接用哨兵把 slot 待回收和 free 的状态分开,可以避免每次调用时都去查 slot 的 state。之后调用 try_retire 把 Dead 转化为 Retiring、交给 EBR。EBR 回收完之后, state 置 Free,回收完成。

对象在 Live 期间对外只以句柄出现:强引用 Cap (内部只是一个紧凑 key、不存代号)、弱引用 Weak (只“曾指向”,带 generation)、IdentRef (guard 作用域内的借用)。Weak 不能直接解引用,必须在 epoch guard 下 upgrade,按 generation 校验、且要求 state==Live,才升成 Cap。

generation 校验是为了防止 slot 回收后复用给另一个对象,而旧 Weak 还存着同一个 key 的情况。每次 slot 被重新分配时,generation 都要+1,表示这是该 slot 的版本号,Weak 解引用时,对比 Weak 的 generation 和 slot 的 generation,保证是同一个对象。

```

fn try_retire_slot(slot) {
    loop {
        let cur = meta.load(Acquire);
        if cur.retain() != RETAIN_SENTINEL_DEAD || cur.state() != Dead
        { return; } // 不是我标的 Dead → 别人已处理
        let new = cur.with_retain(0).with_state(Retiring);
        if meta.compare_exchange(cur, new, AcqRel, Acquire).is_ok() {
            epoch::retire_raw(slot, reclaim_slot::<T>); // 交 EBR,恰好一次
            return;
        }
    }
}

unsafe fn reclaim_slot<T>(ptr) {
    // EBR 已证明没有 IdentRef 还能解引用 → 此刻析构安全
    ptr::drop_in_place(slot.data_ptr()); // 真正析构对象
    let new = cur.inc_generation().with_retain(0).with_state(Free); // 代号 +1,槽变空
    // 可复用
    meta.compare_exchange(cur, new, Release, Relaxed); // 置回 Free
    slab.zone().return_slot_from_reclaim(slot); // 还槽给 Keg
}

```


6.4 延迟回收 (EBR)

Zone 中对核心对象的访问大量是按句柄查找, 以按 pid 找进程为例: 持有一个 `Weak<Process>`, 在一个 epoch guard 下把它 observe 成 `IdentRef`(顺带校验代号与 Live), 再解引用读这个进程。为了使这种频繁的读路径尽可能便宜, 它只读一下 slot 的状态字, 不增加任何引用计数。但这会导致并发问题, 因为该读路径未增加引用计数, 可能在读的时候, 该 slot 的引用计数归零了, 导致对象被释放, 若该 slot 恰好被复用, 则会读到错误内容。基于 epoch 的回收 (EBR) 就是针对这种问题设计的。txKernel 的 EBR 部分参考了 Rust 生态成熟的 crossbeam-epoch。

EBR 以一个全局 epoch 作为粗粒度的逻辑时钟。读者进入临界区时, 将当前全局 epoch 记录到自己在 CPU 离开时清零; 一个对象被删除时不立即析构, 而是记下当时的全局 epoch、挂入本 CPU 的退休链表。回收的判据是 `safe_epoch >= retired_at_epoch + 2`。加二的原因在于: 对象在 epoch E 退休时可能有读者也停在 E; 全局 epoch 从 E 推进到 E+1 以“没有 CPU 停在 E 之前”为前提, 从 E+1 推进到 E+2 以“没有 CPU 停在 E”为前提。因此当全局 epoch 达到 E+2, 所有可能看到该对象的读者必然都已离开临界区, 析构方才安全。这两个 epoch 的跨度即宽限期。

EBR 的全局状态集中在 `EpochDomain`, 其中频繁被多核访问的字段做了缓存行对齐以避免伪共享; 每个 CPU 一份 `CpuLocalEpochState`, 保存它登记的 epoch 与自己的待回收链表; 待回收结点取自每 CPU 的定长池。

```
struct EpochDomain {
    global_epoch: CachePadded<AtomicU64>,          *// 全局逻辑时钟*
    active_guards: CachePadded<AtomicUsize>,        *// 当前活动临界区计数*
    cpu_states: [CpuLocalEpochState; MAX_EPOCH_CPUS], *// 每 CPU 状态//
    possible_cpus / 平台钩子 / 退休池后备存储 ...*
}

struct CpuLocalEpochState {
    initialized: AtomicBool,                        *// 是否已加入 epoch 域*
    local_epoch: AtomicU64,                         *// 0 = 静默; 非零 = 当前所处 epoch*
    retired: UnsafeCell<RetiredList>, *// 本 CPU 待回收链表*
}

struct RetiredNode {
    ptr: *mut u8,                                   *// 待回收对象*
    reclaim_fn: unsafe fn(*mut u8), *// 对象专属回收回调*
    retired_at_epoch: u64,                          *// 退休时的全局 epoch*
    next: Option<usize>,                            *// 下一结点在定长池中的下标*
}
```

第 7 章 进程间通信

7.1 概述

进程间通信(IPC)让相互隔离的进程交换数据与信号。txKernel 实现了信号、管道、futex、System V IPC(信号量/消息队列/共享内存),以及一组基于文件描述符的事件机制(eventfd/signalfd/timerfd/epoll)。

这些机制看似各异,但每一种都只做三件事:持有一份共享状态、让调用者阻塞到某条件成立、状态变化时唤醒被阻塞者。其中真正不同的只有“状态与条件”(管道是环、信号量是计数、futex 是用户字、信号是待处理位集);而对象存储、阻塞、唤醒都来自下层通用设施——对象池管存储与生命周期,等待/通知 substrate 管阻塞与唤醒,这些已在前言详述。所以各 IPC 机制都很薄,下文每节只讲它在通用骨架之上独有的语义。

7.2 信号

信号是向进程异步投递通知的机制。它与管道、信号量、futex 不同:没有独立的池中对象,状态直接放在进程与线程的负载上,txKernel 只用一层 shim 在这些字段上做 POSIX 规定的路由。一个信号要经两步才打断目标线程——投递(放入待处理状态)、递送(返回用户态前执行处置)。

7.2.1 信号的表示

状态分两级:进程级有信号处理表 SigActionTable (64 项,记每个信号的处置)和进程待处理队列;线程级有 64 位信号掩码(SIGKILL / SIGSTOP 不可屏蔽)和线程待处理队列。信号号 1..=64,其中 1..=31 为标准信号(多次投递合并为一个待处理态)、32..=64 为实时信号。每个信号的处置记在 SigActionTable 的一项里:

```
pub struct SigActionEntry {
    disposition: SigDisposition, *// Default / Ignore / Handler(用户函数地址)*
    flags: SaFlags,                *// SA_RESTART / SA_SIGINFO / SA_ONSTACK / ...*
    sa_mask: SignalMask,           *// 运行 handler 期间额外屏蔽的信号*
    restorer: usize,               *// 用户态 restorer(sigreturn 跳板)*
}
```

返回用户态前要频繁判断“该线程有没有信号要处理”,而逐一扫描掩码和两级队列太慢。shim 因此维护一份快速视图 InterruptSummary,把答案压成三个标志、在投递和改掩码时顺带更新,使该判断保持 O(1):

```
pub struct InterruptSummary {           // 线程级,3个标志
    deliverable_signal: bool,           // 有可投递(未屏蔽)的待处理信号
    termination: bool,                  // 需要终止
    stop_requested: bool,                // 需要停止(SIGSTOP)
}
```

7.2.2 信号的投递

投递分两路。控制类信号(SIGKILL / SIGSTOP / SIGCONT)不可屏蔽、即时生效,直接改目标状态(终止/停止/继续)并唤醒。其余信号按 SigActionTable 路由:忽略则丢弃,默认则执行默认动作,用户

handler 则放入线程待处理队列、留待递送。投递后还会 fire 进程的信号端口,唤醒 `signalfd` 这类订阅者(见“事件机制”)。

7.2.3 投递点:AST 检查点

真正递送发生在返回用户态前的 AST 检查点(调度衔接见进程管理一章)。线程进入用户态前先看 `InterruptSummary`:需终止则终止;否则取一个最低号、未屏蔽的待处理信号,按处置决定——忽略则丢弃,默认则终止或停止,用户 handler 则布置信号帧、转入处理函数。

布置用户 handler 时,内核在用户栈压一个信号帧(存被中断的上下文与信号号)、把执行点改到 handler;handler 用 `sigreturn` 返回,内核据信号帧恢复现场。信号帧的保存与恢复是架构相关的,由 HAL 提供(见硬件抽象层一章)。

7.3 管道与 futex

管道与 futex 都是匿名的阻塞通道:没有名字,靠 fd 或用户地址引用,两端在等待源上挂起与唤醒。挂起与唤醒的机制是共用的,本节只看各自独有的状态与条件。

管道用一个 `PipePayload` 表示,读写两端共享。它的状态是一个环形缓冲,外加两个看似多余、实则不可省的计数——读者数与写者数:

```
pub struct PipePayload {
    ring: SpinMutex<PipeRing>,          *// 环形缓冲(VecDeque<PipeBuf>)*
    reader_count: AtomicU32,             *// 读端数*
    writer_count: AtomicU32,             *// 写端数*
    reader_wait_source: Arc<WaitSource>, *// 读者挂这里(环空时)*
    writer_wait_source: Arc<WaitSource>, *// 写者挂这里(环满时)*
}
```

读者在环空时挂到读端等待源,写者在环满时挂到写端等待源,对端写入或读取后 fire 唤醒——这部分是 `substrate` 的常规用法。管道真正特有的是关闭语义,而它无法从对象的引用计数推出,必须由这两个独立计数表达:最后一个写者关闭时,读者读到 EOF(返回 0);最后一个读者关闭时,写者得到 `EPIPE`(并由系统调用层投递 `SIGPIPE`)。计数本身由进程 fd 表在 `close/dup/fork/exec/exit` 时增减。

futex 是用户态同步原语(pthread 的 mutex、condvar 即建于其上),其特点是等待的条件在用户内存里、键也是用户地址。txKernel 用固定 256 个桶,按地址哈希分桶,等待者登记在以(地址空间, 用户地址)为键的桶上。`FUTEX_WAIT` 先校验 `*uaddr == val` 再挂到桶的等待源——“先检查后挂起”在这里是必须的,否则会错过两次检查之间的唤醒;`FUTEX_WAKE` fire 该桶、唤醒至多 n 个等待者。哈希分桶意味着不同地址可能落入同一桶而被一并唤醒,但这无害:等待者醒来会重新校验条件,不满足则继续等待。

7.4 System V IPC

System V 的信号量、消息队列、共享内存,在 txKernel 里都套用与进程对象同一套身份/资源二分(见前言):身份侧记录 key、内核分配的 id、创建者凭证、权限与 `destroyed` 标志,并指向各自的负载;三者由一张全局 key→id 表寻址,各带一份 `IpcPerm` 权限信息:

```
pub struct IpcPerm {
    key: SysvKey,          *// 用户 key(IPC_PRIVATE 则匿名新建)*
```

```

uid: u32, gid: u32,    *// 所有者*
cuid: u32, cgid: u32, *// 创建者*
mode: u16,             *// 读写权限位*
}

```

骨架共享,三者的差别就只在各自负载承载的语义:

- 信号量:负载持有信号量值数组和一个“已变化”通道; `semop` 在原子操作下改值后 fire,阻塞的 `semop` 挂在该等待源上。 `SEM_UNDO` 调整记录在进程负载中,进程退出时回滚,避免死锁。
- 消息队列:负载是一条 FIFO 消息表,带发送、接收两个通道;队列满时 `msgsnd` 挂发送源,队列空时 `msgrcv` 挂接收源,另一侧操作后 fire。
- 共享内存:负载由一个页容器(`PageContainer`)支撑,并记录所有 `attach`(虚拟地址→地址空间); `shmat` 把段映射进调用者地址空间, `shmdt` 解除,直到 `destroyed` 且 `attach` 计数归零才真正回收。

key→id 由一个全局分配器管理:传入 `IPC_PRIVATE` 则新分配一个匿名 id,传入具体 key 则查表返回已存在的 id;id 单调递增、从不复用。

7.5 事件机制

`eventfd`、`signalfd`、`timerfd`、`epoll` 把各类“事件”统一成同一种形状:一个文件描述符加“可读即就绪”。统一之后,它们便能与管道、socket 一道被 `poll` / `epoll` 一视同仁地聚合等待。

每种事件源都是一个 Zone 对象、包装成一个 fd,各自的状态不同:`eventfd` 是一个计数器,`signalfd` 持有订阅掩码和一队待读的信号,`timerfd` 记到期次数。它们都向一个等待源注册:阻塞读(`eventfd` 计数为零、`signalfd` 无信号)就挂到该等待源,事件产生(`eventfd` 写、信号投递、定时器到期)即 fire 唤醒——仍是 `substrate` 的常规用法,各机制只额外提供“何为就绪”的判断。`poll` / `epoll` 则是对一批 fd 的等待源做批量等待与就绪查询。

这样,来自不同来源的“可继续执行”都被归一成同一种 fd 就绪,上层无需为每类对象另写一套等待逻辑——这也正是整套 IPC 复用同一套 `substrate` 的集中体现。

第 8 章 设备驱动

8.1 概述

txKernel 的设备层支持:块设备 `virtio-blk`、网络设备 `virtio-net`、字符设备(串口及其终端 TTY),以及 `/dev/null`、`/dev/zero` 等伪设备。其中块 / 网设备基于 `virtio` 规范,核心复用成熟的 `virtio-drivers` crate——txKernel 只补一个 DMA 适配器,把 `virtqueue` 所需的内存接到帧分配器,RISC-V 经 MMIO 接入, LoongArch 经 PCI 接入。

其设计前提是目标平台(QEMU virt 及对应开发板)的外设在编译期即已知,不需要热插拔,也不需要运行时加载或卸载驱动。因此设备模型被简化为一张静态表——绝大多数设备是 `&'static` 常驻对象,不进对象池、无运行时生命周期;仅在运行时动态创建的实体(如 `pty`)才用对象池分配。

8.2 设备模型

按访问方式把设备分为三类,每类对应一个统一接口——块设备按块读写、字符设备按字节流读写、网络设备收发以太帧;设备号统一用 DevT (高 32 位 major、低 32 位 minor)标识。

```
pub trait BlockDeviceOps {                *// 块设备:按块读写*fn read_blocks(&self, block,
target: &mut [Frame], g) -> StepOutcome<(), NoProgress>;
    fn write_blocks(&self, block, source: &[Frame], g)    -> StepOutcome<(),
NoProgress>;
    fn barrier(&self, g) -> StepOutcome<(), NoProgress>;
}
pub trait CharDeviceOps {                *// 字符设备:按字节流读写*fn read(&self, out:
&mut [u8], g) -> StepOutcome<usize, ByteProgress>;
    fn write(&self, bytes: &[u8], g)    -> StepOutcome<usize, ByteProgress>;
}
pub trait NetDeviceOps {                *// 网络设备:收发以太帧*fn receive(&self) ->
Option<RxFrame>;
    fn transmit(&self, frame: &[u8], g) -> StepOutcome<()>;
    fn mac_addr(&self) -> EthernetAddress;
    fn mtu(&self) -> u16;
}
```

三类设备各自登记在一张静态注册表中(`BlockDeviceRegistration` / `CharDeviceBinding` / `NetDeviceRegistration`,均为 `&'static`):启动时一次性注册、运行期只读查询,没有动态增删,注册时校验 DevT 与名字不重复。唯一的例外是 TTY——pty 在运行时成对创建与销毁,因此其身份与负载用对象池分配、由 `Cap` 引用,生命周期随 open/close 变化。

8.3 设备发现

启动时,引导固件把设备树(FDT)交给内核:RISC-V 由 `OpenSBI` 经寄存器传入 DTB 指针,LoongArch 经 UEFI 系统表中的设备树配置项取得。内核借助 `fdt crate` 解析 FDT,但只取启动信息——内存布局、CPU 数、命令行与 `initrd` 起止。

设备本身的 MMIO 基址与中断号不来自设备树,而由板级代码按平台硬编码(QEMU virt 布局固定),整理进 `PlatformInfo.mmio_regions`,驱动初始化时按名字(如 `"virtio0"`)查找自己的地址。两种架构的接入方式不同:

- RISC-V 用 virtio-MMIO(按硬编码 MMIO 基址直接访问设备寄存器);
- LoongArch 用 virtio-PCI(枚举 PCIe 配置空间发现设备)。

总的来说,txKernel 以设备树读取启动信息,但未做设备树级的设备枚举。

8.4 块设备

块设备按固定大小的“块”读写,是文件系统的底层存储载体;virtio-blk 是 QEMU 提供的一块虚拟磁盘,txKernel 的根文件系统就装在它上面。

实现上,virtio-blk 把这块磁盘包成 `VirtioMmioBlock` (RISC-V)或 `VirtioPciBlock` (LoongArch),内部持有 `virtio-drivers` crate 的 `VirtIOBlk`,初始化时记下设备容量与块大小(512 字节)。它在其上实现 `BlockDeviceOps`:

- `read_blocks` / `write_blocks` 按页遍历目标帧,算出每一帧对应的起始扇区号 (`block_id + idx × 每页扇区数`),交给 `VirtIOBlk` 经 `virtqueue` 提交读/写请求,设备完成后回收;
- `barrier` 调用 `flush`,保证此前写入落盘(支撑 `fsync`)。

块设备再经一个 `bdev-fs` 暴露给文件系统、走页缓存读写(见文件系统一章)。

8.5 网络设备

网络设备就是网卡,负责收发以太网帧,是网络栈(TCP / UDP)与外界之间的出入口;`virtio-net` 是 QEMU 提供的一块虚拟网卡。

实现上,`virtio-net` 包成 `VirtioMmioNet` 或 `VirtioPciNet`,内部是 `virtio-drivers crate` 的 `VirtIONetRaw`,外加两条待完成队列 `rx_pending` / `tx_pending`。初始化时读出网卡 MAC、预填一批接收缓冲。它实现 `NetDeviceOps` :

- `receive`:轮询已完成的接收,取出 `virtio` 头之后的以太网帧负载、包成 `RxFrame` 返回,并补回一个接收缓冲;
- `transmit`:先回收已完成的发送,若发送队列已满则让出,否则拼出“`virtio` 头 + 帧”提交,并记入 `tx_pending`;
- `mac_addr` / `mtu`:返回缓存的值;
- 设备中断到来时,`ack_interrupt_and_fire` 应答中断并轮询收 / 发队列、唤醒等待者。

网络设备不接入 VFS,而由网络栈直接消费其 `receive` / `transmit`。

8.6 字符设备

字符设备里最主要的是串口和架在它上面的终端(TTY)。三者关系是:串口是收发字节的硬件,终端是给它加“终端行为”的软件层,`pty` 则是没有硬件、纯软件造出来的一对终端。

串口:目标平台上是一个 16550 兼容串口(RISC-V 在物理 `0x1000_0000`、LoongArch 在 `0x1fe0_01e0`),按字节收发——发就往发送寄存器写、收就从接收寄存器读,辅以一个状态寄存器判断可读/可写。它对外是一个字符设备(`CharDeviceOps`)。输入走中断:开启串口的接收中断、并在中断控制器(RISC-V PLIC / LoongArch 两级)放开该中断号后,有输入到来即触发中断,由 `uart_rx_irq_handler` 读出字节、放进终端输入缓冲、唤醒在等输入的线程。

终端(TTY):在串口之上提供“终端该有的行为”——行编辑(退格删字、删整行)、回显、`Ctrl-C` / `Ctrl-Z` 转成信号发给前台进程、`Ctrl-S` / `Ctrl-Q` 暂停与恢复输出。这些行为由 `termios` 配置、由行规程(`N_TTY`)实现;一个终端的全部状态都收在 `TtyPayload` 里:

```
pub struct TtyPayload {
    termios: AtomicSlot<Termios>,          *// 终端配置(是否回显、是否行缓冲、控制字符...)*
    ldisc_state: UnsafeCell<LdiscState>,    *// 行规程状态(正在编辑的当前行等)*
    input_queue: SpinMutex<TtyRing>,        *// 给进程 read 的输入缓冲*
    output_queue: SpinMutex<TtyRing>,        *// 发往串口 / 对端的输出缓冲*
    transport: TtyTransport,                 *// 接硬件串口,还是 pty 对端*
}
```

`pty`(伪终端):没有硬件,是一对互为对端的终端——一端充当“键盘 + 屏幕”、另一端交给程序(终端模拟器、`ssh` 等要用它)。它在运行时成对创建,因此用对象池分配(其余设备都是常驻静态对象)。

第 9 章 硬件抽象层

txKernel 实现了完整的硬件抽象层(Hardware Abstraction Layer,HAL),将 RISC-V64 与 LoongArch64 两种架构的差异收敛在一组面向内核的接口中。这些接口均与 CPU 架构相关,按功能可分为以下几类:

- 启动与平台信息: `BootPlatformIf`、`InitIf`、`BootInfoIf`、`PlatformInfoIf`、`AuxvIf`、`PlatformConfig` —— 固件交接、每核早期初始化、启动参数与设备树解析、页大小与地址空间布局等平台常量。
- 页表与地址空间: `PmapIf` —— 建立、修改、撤销内核与进程映射,创建与切换进程页表,扩展直映射窗口。
- 陷入、中断与信号: `TrapIf`、`IrqIf`、`SignalFrameIf` —— trap 现场的保存与恢复、陷入类型判定、外部中断控制器的申请与应答、在用户栈布置信号现场;陷入事件经内核侧回调接口 `KernelTrapSink` 路由到具体处理。
- 时钟: `TimeIf` —— 读取高精度计数器、设置定时器到期时刻。
- 多核与每核状态: `SmpIf`、`PercpuIf` —— 启动从核、核间中断(IPI)的收发、每核私有存储的安装与访问。
- 浮点与向量: `FpSimdIf` —— 惰性浮点/SIMD 上下文的保存与恢复。
- 缓存与 DMA: `CacheIf`、`DmaIf` —— 缓存维护操作、DMA 一致性内存的分配。

这些接口由统一的 `TxPlatform trait` 聚合(惰性浮点等少数按需单独实现),每个目标平台各实现一份;内核代码一律对平台类型 `P: TxPlatform` 泛型,通过 `P::方法()` 调到具体架构的实现。

平台选择上参考了 ArceOS/axHal 的静态思路:具体跑哪一套不在运行时决定,而是编译期就定死——内核对平台类型泛型,运行期没有“当前是哪个架构”的判断。在一些组件的具体(如惰性浮点保存、硬件 TLB 重填、用户指针缺页探测等)实现上,则学习了往届作品的设计方法。

9.1 启动

txKernel 的内核启动大致分为三步:固件交接、`_start` 切换地址空间、通用主线 `kernel_main`。

固件(RISC-V 为 OpenSBI,LoongArch 为 QEMU 直接引导)完成最底层初始化后,跳转到内核入口 `_start`,并传入核号与设备树/启动参数。

`_start` 接收固件参数,建立每核启动栈、清零 BSS、构建引导页表,随后开启分页并跳到高半区。其中页表很关键:内核镜像被固件装在物理低地址、却按高半区虚址链接,而开启分页的那条指令本身仍跑在低地址的 `_start` 里。因此引导页表必须同时满足两类需求——既要让正在低地址执行的 `_start` 平稳跨过“开分页”这一刻,又要为之后在高半区运行的内核铺好地址空间。RISC-V 为此手搭三类映射:

- 恒等映射(低虚址=低物理址):`csrw satp` 开分页那一刻 PC 仍在低处,必须有低地址映射撑住,否则开分页后取下一条指令立即缺页。它是过桥时脚下的踏板,跳到高半区后即弃。
- 直映射窗口(整段 RAM→固定偏移+物理址):内核运行中要随手按物理地址访存(清新分配的帧、读写页表节点、访问设备 MMIO),有此窗口则“物理地址+偏移”即得可解引用的内核虚址。
- 内核高半区别名(内核镜像物理页→高链接地址,按段 RX/R/RW):内核按高地址链接,必须有“高虚址→镜像物理页”才能执行;按段取最小权限,代码不可写、数据不可执行。

LoongArch 的处理方式与此不同,启动阶段不构建引导页表,而是借助 DMW(直接映射窗口)硬件机制建立地址空间。启动代码配置两个映射窗口:DMW0 为可缓存窗口,覆盖虚址高位为 `0x9`

的区段;DMW1 为非缓存窗口,覆盖虚址高位为 0x8 的区段。两者均为直接映射,即窗口内虚址的低位直接作为物理地址。配置完成并执行 `invtlb` 使 TLB 失效后,直接跳转至内核链接所在的高地址(0x9000000000201000,落在 DMW0 窗口内)。由于内核镜像本身即链接在 DMW0 窗口中(窗口基址 0x9000000000000000),DMW0 同时承担两个职责:既作为内核镜像的可执行别名供取指执行,又作为直映射窗口供内核按物理地址访存;DMW1 则用于对设备寄存器(MMIO)的非缓存访问。因此,RISC-V 需显式构建的恒等映射、直映射窗口、内核高半区别名三类映射,在 LoongArch 上均由 DMW 硬件窗口直接提供,启动阶段既无需构建页表,也无需用于过渡执行的恒等映射。

跳到高半区后, `_start` 经 `rust_entry` (只负责选定当前平台类型)进入通用主线 `kernel_main`。`kernel_main` 不再分架构,负责做每核准备并按依赖顺序把整个系统拉起来——substrate、完整 trap 向量、reactor、各子系统,直到 `exec /init` 进入用户态。

RISC-V 的启动代码如下:

```
_start:
    mv    s0, a0                // 核号 hartid
    mv    s1, a1                // 设备树 DTB 指针
    la    sp, __tx_boot_stack_top_load
    slli  t1, s0, 17            // 按 hartid 错开每核启动栈
    sub   sp, sp, t1
    // 清零 BSS;在 __bootstrap_root 写入恒等映射、直映射窗口、内核高半区三类映射
    srli  t0, s2, 12
    li    t1, TX_RV64_SATP_SV39
    or     a0, t0, t1
    csrw  satp, a0              // 开启 Sv39 分页
    sfence.vma
    li    t0, TX_RV64_KERNEL_VIRT_OFFSET
    add   sp, sp, t0            // sp / gp 移到高半区
    add   gp, gp, t0
    la    t1, __rust_entry_load
    add   t1, t1, t0
    jr    t1                    // 跳进高半区 rust_entry

// 板级 rust_entry:唯一作用是定下平台类型,直接进主线
pub extern "C" fn rust_entry(cpu_id: usize, firmware_arg: usize) -> ! {
    tx_kernel::kernel_main::<ActivePlatform>(cpu_id, firmware_arg)
}

// 通用主线:对平台 P 泛型,架构相关操作都经 P 调到各自实现
pub fn kernel_main<P: TxPlatform + 'static>(cpu_id: usize, firmware_arg: usize) -> ! {
    P::install_minimal_trap_vector();                // 极简 trap 桩
    let handoff = P::boot_handoff(cpu_id, firmware_arg); // 归一固件参数
    P::install_early_percpu(handoff.cpu_id);          // 装每核私有状态
    P::mark_cpu_online(handoff.cpu_id);                // 标记本核上线
    init::CoreInit::<P>::boot(handoff)                // 按依赖顺序起整个系统
}
```


9.2 页表管理

页表管理是内核侧的硬件分页层:它向上为内核与各进程提供一套统一接口 `PmapIf` (建立、修改、撤销映射,以及切换地址空间),向下把 RISC-V 与 LoongArch 两套差异极大的分页硬件收敛在接口之后——内核代码自始至终只跟 `PmapIf` 打交道,不感知架构。

整套机制建立在一个固定的地址空间布局上,虚拟地址以符号位分高、低两半:

- 低半区:各进程私有的用户空间,进程间互不可见,随 `exec`、`mmap`、`fork`、写时复制不断增删映射。
- 高半区:内核独占,并在所有进程间共享同一份映射;其中固定放两块——直映射窗口(整段物理内存按固定偏移映入,“物理地址 + 偏移”即得可解引用的内核虚址,供清零新页、读写页表节点、访问 MMIO)与内核镜像(代码/只读/数据三段按 RX/R/RW 映入)。高半区共享,使进程切换只换低半区的根、内核映射无须重建。

接口之下,两架构的实现差异显著:RISC-V 用三级 Sv39、单根 `satp`、权限正向编码、TLB 由硬件自动重填;LoongArch 用四级(`PWCL` / `PWCH` 配置)、双根 `PGDL` / `PGDH`、权限反向编码、TLB 软件重填,并以 DMW 硬件窗口替代手搭的直映射——这些差异都被 `PmapIf` 的同一组方法吸收。

9.2.1 页表项

页表中的最小单位是页表项(PTE),记录一个物理页号和一组权限/状态标志(有效、已访问、脏等)。RISC-V 和 LoongArch 的页表权限位采用相反的编码模式:

- RISC-V: 正向——置 `R` / `W` / `X` 表示“允许读/写/执行”。
- LoongArch: 反向——页默认可读、可执行,反以 `NR` / `NX` 表示“禁止读/执行”,并另用存储访问类型位携带缓存属性。

两个架构的页表权限位控制逻辑如下——RISC-V 置位即允许、LoongArch 置位即禁止:

```

*// RISC-V:正向—置位即“允许”*fn encode_leaf_pte_with_permissions(phys, perms) -> u64
{
    let mut flags = 0;
    if perms.contains(READ)    { flags |= PTE_R; }
    if perms.contains(WRITE)   { flags |= PTE_W; }
    if perms.contains(EXECUTE) { flags |= PTE_X; }
    if perms.contains(USER)    { flags |= PTE_U; }
    ((phys >> 12) << 10) | flags | PTE_V | PTE_A | PTE_D
}

*// LoongArch:反向—默认可读可执行,置位即“禁止”*fn encode_la64_leaf_pte(phys, perms) ->
u64 {
    let mut flags = LA64_PTE_V | LA64_PTE_A | LA64_PTE_PRESENT | mat; *// mat=缓存属性
    if !perms.contains(READ)    { flags |= LA64_PTE_NR; } *// 反向
    if perms.contains(WRITE)   { flags |= LA64_PTE_W | LA64_PTE_D; }
    if !perms.contains(EXECUTE) { flags |= LA64_PTE_NX; } *// 反向
    if perms.contains(USER)    { flags |= LA64_PTE_PLV_USER; }
    (phys & LA64_PTE_PFN_MASK) | flags
}

```

两边都用一个用户位区分内核页与用户页;内核镜像的代码/只读/数据三段则按链接期范围分别取 RX/R/RW,在硬件层面拿到最小权限。

9.2.2 PmapIf

txKernel 通过 PmapIf 来管理页表的建立,修改和切换等操作。它把页表操作列成一组方法——建立、提交、撤销映射,创建和切换进程页表,扩展直映射窗口——内核只管调用,不关注下层架构。

RISC-V 和 LoongArch 分别实现一份 PmapIf 实现对不同架构的统一处理。两种架构在方法的实现上差距很大

- `activate_pmap` (切地址空间):RISC-V 写一次 `satp`,LoongArch 写 `PGDL`、`ASID`、`CRMD` 一组 CSR;
- `commit_kernel_mapping` (写一条映射):RISC-V 编正向 PTE(置 R/W/X),LoongArch 编反向 PTE(清 NR/NX);
- `extend_direct_map` (扩展直映射窗口):RISC-V 要真去建一批 1GiB 大页表项,LoongArch 却直接返回成功、什么都不做——它的 DMW 硬件窗口本来就覆盖了全部物理内存,不需要页表。

内核代码对平台类型 P 泛型(TxPlatform 要求实现 PmapIf)。编译某块板子时,P 被定成那块板子的 Platform,内核里每一句 `P::reserve_kernel_mapping(...)` 在编译期就绑到对应架构那份实现上。

PmapIf 同时管两类页表:所有进程共享的内核页表、每个进程私有的页表,两者用同一组方法、只是作用的根不同。改一条映射统一拆成预留和提交两步(预留可能失败、失败就整条回滚,提交保证成功),撑页表的中间节点由一个专用页池供给、不占内核堆。

```
pub trait PmapIf {
    fn reserve_kernel_mapping(virt, phys, kind)
        -> Result<Option<PmapReservation>, PmapError>;    // 预留,可失败
    fn commit_kernel_mapping(reservation, perms);          // 提交,不失败
    fn unmap_kernel_mapping(virt, kind);                  // 撤销 + 失效 TLB
    fn extend_direct_map(phys_end) -> Result<(), PmapError>; // 扩展直映射窗口
    fn create_pmap_root() -> Result<PmapRoot, PmapError>; // 创建进程页表根
    fn activate_pmap(root: &PmapRoot);                    // 切换时激活
    // ...(进程映射另有一组带 root 参数的同名方法)
}
```

9.2.3 架构差异

两种架构的页表机制差别很大,下面分开讲。

RISC-V(Sv39)

- 三级页表:39 位虚拟地址按 9/9/9 位索引三级目录、余 12 位为页内偏移,从 `satp` 指向的根表逐级走到物理页。每级表项凭有没有 R/W/X 位来判断是否是叶节点,若没有,则继续向下遍历;若有,则是叶节点。因此,任意一级都能当叶子,因此支持 1GiB/2MiB/4KiB 三种页面,直映射窗口就用 1GiB 大页一段段铺成。
- 单根:内核与用户共用同一张根页表(高半区放内核条目、低半区放用户条目),根的物理页号连同 Sv39 模式位写进 `satp` 即开启翻译;切换地址空间只需改写 `satp`,其中带的 ASID 让切换不必全量刷新 TLB。

- TLB 由硬件重填:TLB 缺失时硬件页表遍历器自动查表回填,内核不介入。

LoongArch

- 四级页表:48 位虚拟地址按 9/9/9/9 位索引四级目录、余 12 位为页内偏移。LoongArch 的页表格式直接在启动时用 `PWCL / PWCH` 声明,是否是叶节点则 `PRESENT` 位区分,同样实现了不同大小的页面支持。
- 双根:用户半区与内核半区各用一张根—— `PGDL` (每进程一张)和 `PGDH` (常驻、所有进程共享),硬件依虚址最高位自动选用哪张;切换地址空间只换 `PGDL` 和 `ASID`, `PGDH` 不动。
- DMW 直映射窗口:配好 `DMW0` (可缓存)与 `DMW1` (非缓存)两个窗口后,虚址高位一旦匹配,硬件直接取其低位作物理地址,既不查页表、也不占 TLB;于是直映射窗口和内核镜像都落在 DMW 里,连 `extend_direct_map` 都不用建表。
- TLB 由软件重填:TLB 缺失触发专门的重填异常,内核在入口用 `lddir / ldpte / tlbfill` 逐级走页表、取项填入;为求快,入口只借专用暂存寄存器、不构造完整异常上下文。`invtlb` 可按 `ASID/地址` 精确失效。

LoongArch 切换地址空间时写的一组 CSR:

```
write_la64_csr(LA64_CSR_ASID, switch.asid); // 关联地址空间标识
write_la64_csr(LA64_CSR_PGDL, switch.pgdl); // 仅更换用户半区根
write_la64_csr(LA64_CSR_PGDH, switch.pgdh); // 内核半区根(常驻不变)
write_la64_csr(LA64_CSR_CRMD, LA64_CRMD_PG | ...); // 开启分页
la64_invtlb_all();
```

9.3 中断与异常处理

txKernel 把所有陷入内核的事件——系统调用、缺页、时钟中断、外部设备中断、核间中断(IPI)、非法指令——统一成一条 trap 路径处理,并在这条路径上一并完成线程的上下文切换。

整条路径分两层。平台侧只做四件机械的事:保存现场、把硬件陷入原因翻译成架构无关的分类、调一个内核入口、再按内核回传的 `TrapAction` 收尾。内核侧(`KernelTrapDispatcher`,架构无关)只按分类把事件路由到对应处理。两层之间只靠 `TrapAction` 这一个窄接口耦合,因此同一份内核 trap 逻辑在 RISC-V 与 LoongArch 上不加改动地复用,架构差异全收在平台侧。

需注意的是,txKernel 采用了无栈协程的设计,因此没有每线程内核栈,线程就是一个 future,因此“进用户态→陷入→处理→重入”不是简单的内核调用栈操作,而是一次 future 的让出与重新 poll。

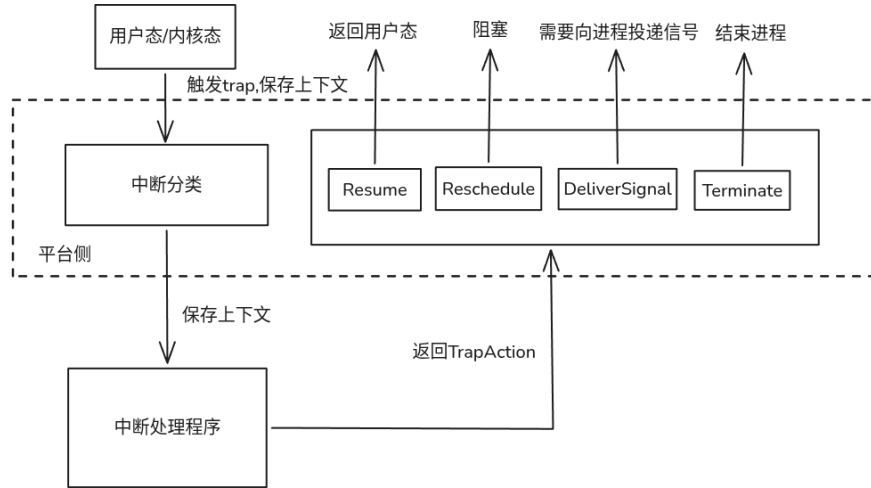


图 22 中断处理流程图

9.3.1 中断处理流程

用户程序触发 trap 后,硬件跳到预先安装的 trap 入口。入口先保存现场——切到每核 trap 栈,把被陷线程的通用寄存器与关键 CSR 存入 trap 帧。

随后判别 trap 类型,归一成统一的 `TrapClass`,但两架构的读法不同。RISC-V 读 `scause`:最高位区分中断与异常、其余位是编号,编号表示是哪种中断,一次 match 即可 (`classify_rv64_trap`:8=系统调用,12/13/15=取指/读/写缺页,5=定时器,9=外部中断,1=核间中断…);LoongArch 读 `estat` 的 `ecode` 字段先定大类,若是中断(`INT`)还需再看 `estat` 的中断状态位(`IS`)才能分出定时器/IPI/外部中断,是两步 (`classify_la64_trap`)。出错地址 RISC-V 取自 `stval`、LoongArch 取自 `badv` :

```
// RISC-V
const fn classify_rv64_trap(scause: usize) -> TrapClass {
    let is_interrupt = scause >> (usize::BITS - 1) != 0; // 最高位
    let code = scause & !(1 << (usize::BITS - 1)); // 其余位 = 编号
    match (is_interrupt, code) {
        (false, 8) => TrapClass::Syscall,
        (false, 12) => TrapClass::PageFault { write: false, instruction:
true }, // 取指缺页
        (false, 13) => TrapClass::PageFault { write: false, instruction:
false }, // 读缺页
        (false, 15) => TrapClass::PageFault { write: true, instruction:
false }, // 写缺页
        (true, 1) =>
TrapClass::InterprocessorInterrupt, // 核间中断
        (true, 5) =>
TrapClass::TimerInterrupt, // 定时器
        (true, 9) =>
TrapClass::ExternalInterrupt, // 外部中断
        (false, _) => TrapClass::UnknownSync,
        (true, _) => TrapClass::UnknownInterrupt,
    }
}
```

```

}

//LoongArch
const fn classify_la64_trap(estat: usize) -> TrapClass {
    let ecode = (estat >> LA64_ESTAT_ECODE_SHIFT) & LA64_ESTAT_ECODE_MASK; // 第一步:大类
    match ecode {
        LA64_ECODE_INT => { // 中断 → 第二步看 IS 位
            if estat & LA64_ESTAT_IS_TIMER != 0 { TrapClass::TimerInterrupt }
            else if estat & LA64_ESTAT_IS_IPI != 0
{ TrapClass::InterprocessorInterrupt }
            else if estat & LA64_ESTAT_IS_HWI_MASK != 0
{ TrapClass::ExternalInterrupt }
            else { TrapClass::UnknownInterrupt }
        }
        LA64_ECODE_SYS => TrapClass::Syscall,
        LA64_ECODE_PIS | LA64_ECODE_PME => TrapClass::PageFault { write: true,
instruction: false },
        LA64_ECODE_PIF | LA64_ECODE_PNX => TrapClass::PageFault { write: false,
instruction: true },
        LA64_ECODE_PIL | LA64_ECODE_PNR | LA64_ECODE_PPI =>
            TrapClass::PageFault { write: false, instruction: false },
        _ => TrapClass::UnknownSync, // ... ALE 对齐 / BRK 断点 / INE 非法指令
    }
}

```

两种架构的 TLB 缺失处理有较大差异:RISC-V 的 TLB 缺失由硬件页表遍历器自动重填、不产生 trap;LoongArch 的 TLB 缺失则是一条单独的重填异常,走专门的重填入口、由内核以 `lddir / ldpte / tlbfill` 走页表填 TLB,根本不进上面的分类——只有重填后页表项仍无效,才转成普通缺页 `TrapClass::PageFault` 再走分类。

分类完成后, `dispatch_trap_frame` 按 `TrapClass` 回调内核的 `KernelTrapSink` 对应方法——缺页交 `on_page_fault`、系统调用交 `on_syscall`、外部中断交 `on_external_irq` (取号后分发给设备处理函数)、定时器交 `on_timer_interrupt` 等;每个回调返回一个 `TrapAction`。

内核处理完并不直接返回,而是交回一个动作,由平台侧据此收尾。原因在于 trap 入口是一段同步代码,不能像执行器上的异步任务那样就地等待 I/O 或调度;因此处理函数把需要阻塞的工作交接给执行器,只回一个动作说明下一步:

- `Resume`:已就地处理完,恢复现场、直接返回用户态;
- `Reschedule`:需要阻塞或重新调度,退回异步执行器,就绪后再返回用户态;
- `DeliverSignal`:在用户栈布置信号现场,转入信号处理函数;
- `Terminate`:终止该进程。

这样,频繁的 trap (如 `getpid`、空转的定时器中断、未唤醒任何线程的外部中断)走 `Resume` 当场返回,完全不经过异步执行器,省去一次调度往返;只有真正需要阻塞的 trap 才付出交接代价。处理入口与动作约定在 `KernelTrapSink` 中:

```
pub enum TrapAction { Resume, Reschedule, DeliverSignal, Terminate }

pub trait KernelTrapSink<P: TxPlatform> {
    fn on_page_fault(view: TrapFrameMut<'_>, fault: FaultInfo) -> TrapAction;
    fn on_syscall(view: TrapFrameMut<'_>) -> TrapAction;
    fn on_timer_interrupt(cpu: CpuId, view: TrapFrameMut<'_>) -> TrapAction;
    fn on_external_irq(cpu: CpuId) -> TrapAction;
    fn on_ipi(cpu: CpuId) -> TrapAction;
    fn on_illegal_or_sync_fault(view: TrapFrameMut<'_>, fault: FaultInfo) ->
    TrapAction;
}
```

以外部中断为例,处理函数取号、分发、应答后,只有当中断唤醒了等待的线程时才返回 `Reschedule`,否则 `Resume`:

```
fn on_external_irq(_cpu) -> TrapAction {
    let irq = P::claim();
    let handled = P::dispatch_irq(irq);
    P::complete(irq);
    match handled {
        IrqHandled::Wake => TrapAction::Reschedule, // 唤醒了等待线程 → 退回执行器
        _                => TrapAction::Resume,     // 没唤醒谁 → 直接返回
    }
}
```

9.3.2 上下文的保存与恢复

进出内核都要保存与恢复被陷线程的通用寄存器和关键 CSR(两者是一对操作),但两种架构所用的暂存机制不同。

RISC-V 借助 `sscratch` 寄存器在用户栈与每核 trap 栈之间切换:陷入时 `sscratch` 持有 trap 栈顶,与 `sp` 一换即得到内核栈;返回时再换回。浮点寄存器按需保存——仅当 `sstatus.FS` 标记浮点被改动时才保存,省去多数 trap 的浮点开销。

```
addi sp, sp, TX_RV64_TF_SIZE # 释放 trap 帧
csrrw sp, sscratch, sp       # sp ↔ sscratch:换回用户栈
sret
```

LoongArch 没有等价的栈交换寄存器,改以多个 `KSAVE` 暂存寄存器分工: `KSAVE0` 预置 trap 栈顶、陷入时与 `sp` 交换, `KSAVE3` 暂存工作寄存器, `KSAVE1`、`KSAVE2` 保存内核线程指针类寄存器。保存完毕后,还要把内核入口地址转换为 DMW 缓存地址再跳入,以确保在缓存窗口中执行;返回用户态前则清除 `LLBCTL` 的链接位,以配合 `ll/sc` 原子序列的语义。

```
csrrw $sp, KSAVE0          # 用户 trap:换到每核 trap 栈,KSAVE0 存用户 sp
# ..... 保存 GPR 与 ESTAT/ERA/BADV/CRMD/PRMD .....
li.d $r13, 0x9000_0000_0000_0000 # 内核入口地址 → DMW 缓存地址
```



```
or    $r12, $r12, $r13
jirl  $r1, $r12, 0           # 进入内核处理
```

9.4 时钟与定时器

txKernel 通过 `TimeIf` 来管理时间与定时。它把时间操作列成一组方法——读当前时刻、安排下一次定时器中断、取消定时器、查定时器频率。

```
pub trait TimeIf {
    fn read_ns() -> u64;           // 当前单调时间(纳秒)
    fn set_deadline_ns(ns: u64);  // 安排下一次定时器中断的绝对时刻
    fn cancel_deadline();
    fn frequency_hz() -> u64;     // 定时器频率
}
```

`read_ns` 读硬件单调计数器并换算成纳秒, `set_deadline_ns` 把下一次中断时刻写进定时器。两架构的硬件机制不同:

- RISC-V: 读 `time` 计数器取时。设的是绝对时间 `mtimecmp`, 但它属 M 态、内核不能直接写, 需调用 SBI 的 `set_timer` 来设置。
- LoongArch: 读稳定计数器(stable counter)取时。定时器 TCFG 收的是倒计时, 即过多少时间再唤醒, 故先算还差多少 tick 再写; 内核在 PLV0, 可直接读写定时器 CSR, 无需固件中介。

txKernel 用 `TimerWheel` 来统一管理各种定时需求, `TimerWheel` 管理一张全局登记表, 使用 `SpinMutex` 保护, 确保并发安全。当前时钟管理机制用 `Vec` 来管理已登记的定时器, 效率不高, 后续考虑实现完整的时钟轮。

```
pub struct TimerWheel {
    state: Arc<SpinMutex<TimerWheelState>>, // 全局共享, 锁保护并发登记/触发
}
struct TimerWheelState {
    next_token: u64,           // 单调发号, 每条登记一个 token
    entries: Vec<Entry>,       // 所有已登记的定时器
}
struct Entry {
    deadline: Deadline,        // 到期时刻
    mailbox: Option<Weak<TaskMailbox>>, // 到期时唤醒的目标(线程信箱)
    token: TimerToken,          // 登记标识, 用于取消
    role: TimerGuardRole,       // 用途: sleep 本身 / 超时中止 / 委托超时
}
```

9.5 用户态内存管理

内核处理系统调用时常要读写用户态指针指向的内存(参数缓冲区、路径字符串等)。这些地址由用户传入, 可能未映射、不合法, 或合法但物理页尚未在内存。内核需处理非法指针与缺页的情况。

txKernel 是无栈协程内核, 没有每线程内核栈, 内核态一旦自己缺页, 就没有现场可供挂起、调页、再恢复, 因此 txKernel 不允许内核态缺页 (`on_page_fault` 中非用户态的缺页直接 `Terminate`)。因此

txKernel 不直接解引用用户传入的指针，而是在拷贝前就判断是否缺页。访问每一页前，先通过 `resolve_user_page_addr` 解析该页，有三种结果：

- 命中:页已映射且权限匹配,取其内核直映射地址,拷贝这一段;
- 非法:无映射或权限不符,返回 `EFAULT` ;
- 缺页:页合法但物理页未就位、要等 I/O,则让出 future(记下已拷字节数),页就绪后重新 poll、从断点继续拷贝。

确保拷贝时涉及的页均已就位，不会出现缺页中断。

```
fn copy_in(aspace, dst: &mut [u8], src: UserPtr<u8>, guard) -> StepOutcome<usize,
ByteProgress> {
    let (mut copied, total) = (0, dst.len());
    while copied < total {
        let user_addr = src.addr() + copied;
        let page_addr = user_addr & !(USER_PAGE_SIZE - 1);           // 当前字节所在
        // 页
        let within    = user_addr - page_addr;                       // 页内偏移
        let chunk     = min(total - copied, USER_PAGE_SIZE - within); // 本页待拷字节
        // 数

        match resolve_user_page_addr(aspace, page_addr, Read, guard) {
            // 命中:取该页内核直映射地址,拷这一段,推进
            Done(frame_base) => {
                copy_nonoverlapping(frame_base.add(within),
dst[copied..].as_mut_ptr(), chunk);
                copied += chunk;
            }
            // 没映射/权限不符:已拷部分则返回部分,否则 EFAULT
            Err(e) => return if copied > 0 { Done(copied) } else { Err(e) },
            // 需等 I/O 调页:让出协程,记下进度,页就绪后从断点续拷
            Blocked(t) => return yield_wait_token(ByteProgress::new(copied), t),
        }
    }
    Done(copied)
}
```

第 10 章 网络栈

10.1 概述

txKernel 的网络模块以 `smoltcp` 作为引擎库实现了 TCP,UDP,ICMP 协议，同时还实现了套接字对象模型、网络命名空间与整栈驱动循环，对上为用户程序提供与 POSIX 兼容的 BSD 套接字接口，对下驱动 `virtio-net` 网卡与本地回环等网络设备。txKernel 把 TCP/UDP 的协议状态机、序列号、窗口与校验和等细节交给 `smoltcp`，而自己实现套接字的身份与生命周期、收发缓冲、设备与命名空间抽象。

网络栈采用清晰的分层架构，自顶向下依次为套接字层、传输层、网络层、链路层与设备层，各层之间通过统一的设备抽象与套接字对象交互，层间依赖严格自上而下，下层不感知上层的存在，从而降低了模块间的耦合度，便于后续扩展与维护，整体分层结构如图 23 所示。

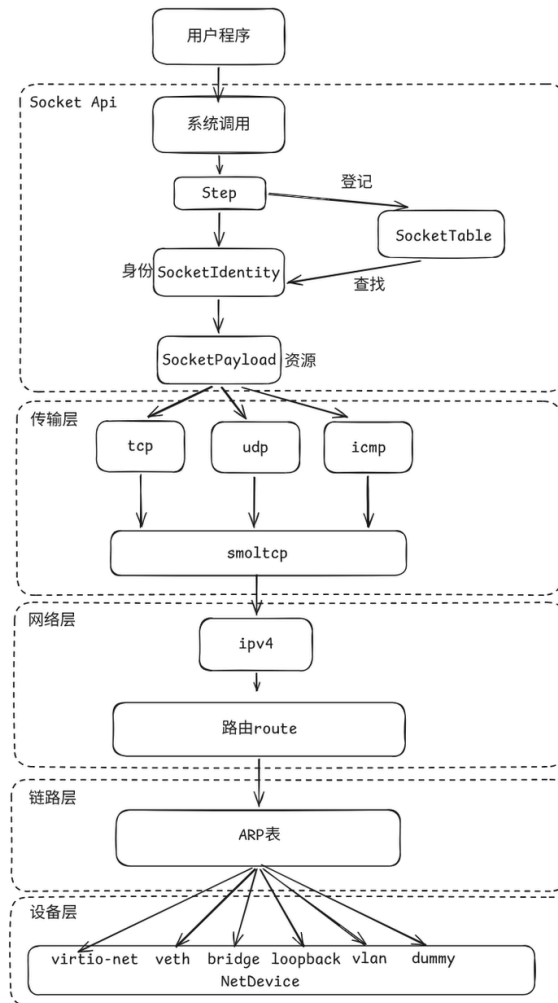


图 23 网络栈概述

10.2 Socket 模型

10.2.1 身份资源二分

网络模块中每一个套接字都被建模为两个独立的内核对象：轻量且长期存在的身份对象 `SocketIdentity`，以及笨重且可独立装卸的资源对象 `SocketPayload`。这一拆分的理由可以由前言的 **身份资源二分** 可得。

身份对象只持有就绪通知线 `readiness`、等待载体 `wait_carriers`、紧急数据端口 `urgent_port`，以及一个指向载荷的句柄 `payload`；其余一切都被收纳进载荷对象。身份以 `Cap<SocketIdentity>` 计数持有，载荷以 `PayloadCap<SocketPayload>` 计数持有，后者在锁定载荷的同时内部持有身份，保证载荷不会比身份活得更久。

```
pub struct SocketIdentity {
    pub kind: SocketKind,           // 套接字类型(TCP/UDP)
```

```

pub family: AddressFamily,           // 地址族
pub readiness: SocketReadiness,      // 就绪通知(可读/可写/可接受)
pub wait_carriers: SocketWaitCarriers, // 阻塞等待的载体句柄
pub urgent_port: RawPort,             // TCP 紧急数据(带外)事件
payload: SpinMutex<Option<PayloadCap<SocketPayload>>>, // 指向资源,可置空
}

```

资源对象按协议族持有一组互斥的 `raw_*` 套接字槽位, 同一时刻只有一个为 `Some`; 另外集中存放协议状态机标记 `protocol`、套接字选项 `options`、收发 I/O 状态 `io`、所属命名空间 `net_namespace` 与 TCP 半连接队列 `tcp_backlog` 等。

```

pub struct SocketPayload {
    family: SpinMutex<AddressFamily>,
    net_namespace: PayloadCap<NetNamespacePayload>, // 所属网络命名空间
    protocol: SpinMutex<SocketProtocol>,           // 生命周期状态机标记
    options: SpinMutex<SocketOptionSet>,
    ip_multicast: SpinMutex<Ipv4MulticastMemberships>,
    raw_tcp: Option<RawTcpSocket>,                  // 以下为按协议族互斥的槽位
    raw_udp: Option<RawUdpSocket>,
    raw_icmp: Option<RawIcmpSocket>,
    raw_unix: Option<RawUnixSocket>,
    raw_rds: Option<RawRdsSocket>,
    raw_sctp: Option<RawSctpSocket>,
    raw_packet: Option<RawPacketSocket>,
    raw_netlink_route: Option<RawNetlinkRouteSocket>,
    raw_netlink_netfilter: Option<RawNetlinkNetfilterSocket>,
    io: SpinMutex<SocketIoState>,
    unix_peer_cred: SpinMutex<Option<UnixPeerCred>>,
    tcp_backlog: SpinMutex<TcpBacklog>,
    shutdown_rd: AtomicBool,
    shutdown_wr: AtomicBool,
}

```

套接字类型由 `SocketKind` 标识, 协议状态机由 `SocketProtocol` 携带, 二者一一对应。这种“一个身份对象+一组按需填充的协议槽”的安排带来一个直接的好处: 新增一个协议族, 只需添加一个 `raw_*` 槽位与一个状态枚举分支, 身份对象与上层分派逻辑无需改动。

```

pub enum SocketKind {
    UnixDatagram, UnixStream, Tcp, Udp, Sctp, RdsSeqPacket,
    RawIcmp, NetlinkRoute, NetlinkXfrm, NetlinkNetfilter, Packet,
}

```

10.2.2 套接字表

报文到达时, 内核须依据目的地址与端口反查出应当接收它的套接字; 用户执行 `bind` 时, 内核亦须判定目标地址是否已被占用。这两类查找均由 `SocketTable` 承担。它是每个网络命名空间独有的一

组索引，将不同形态的网络键——本地端点、监听端点、连接四元组、Unix 路径——分别映射至 `Cap<SocketIdentity>`。

```
pub struct SocketTable {                                // 每 netns 一份
    tcp_bound:      Index<LocalEndpointKey, Cap<SocketIdentity>>, // 本地端点
    tcp_listeners:  Index<ListenerKey,      Cap<SocketIdentity>>, // 监听端点
    tcp_connections: Index<ConnectionKey,    Cap<SocketIdentity>>, // 四元组
    udp_bound:      Index<LocalEndpointKey, Cap<SocketIdentity>>,
    udp_connections: Index<ConnectionKey,    Cap<SocketIdentity>>,
    // sctp_* / rds_bound / raw_icmp 同构,
    unix_bound:      Index<UnixSocketPath,    Cap<SocketIdentity>>, // 路径
    unix_stream_peers: Index<UnixStreamPeerKey, Cap<SocketIdentity>>,
}
```

套接字表中存放的是身份的 `Cap`，而非套接字对象的副本。因此，同一 `SocketIdentity` 同时被两条引用路径指向：一条源自用户侧的文件描述符，另一条源自网络侧的套接字表。前者驱动用户发起的操作，后者决定入站报文的归属；二者指向同一对象，从而保证用户视图与网络视图始终一致。身份、资源、套接字表之间的关系如下：

10.3 传输层

传输层提供面向连接 TCP 协议与无连接的 UDP 协议。两者的协议状态机均由 `smoltcp` 承担，而内核在其外围维护自己的收发缓冲与命名空间索引。

10.3.1 TCP

TCP 的实现分为两层：一层是内核自管的生命周期状态机 `TcpState`，另一层是封装 `smoltcp` 协议引擎的 `RawTcpSocket`。他们的关系如图 24 图所示：

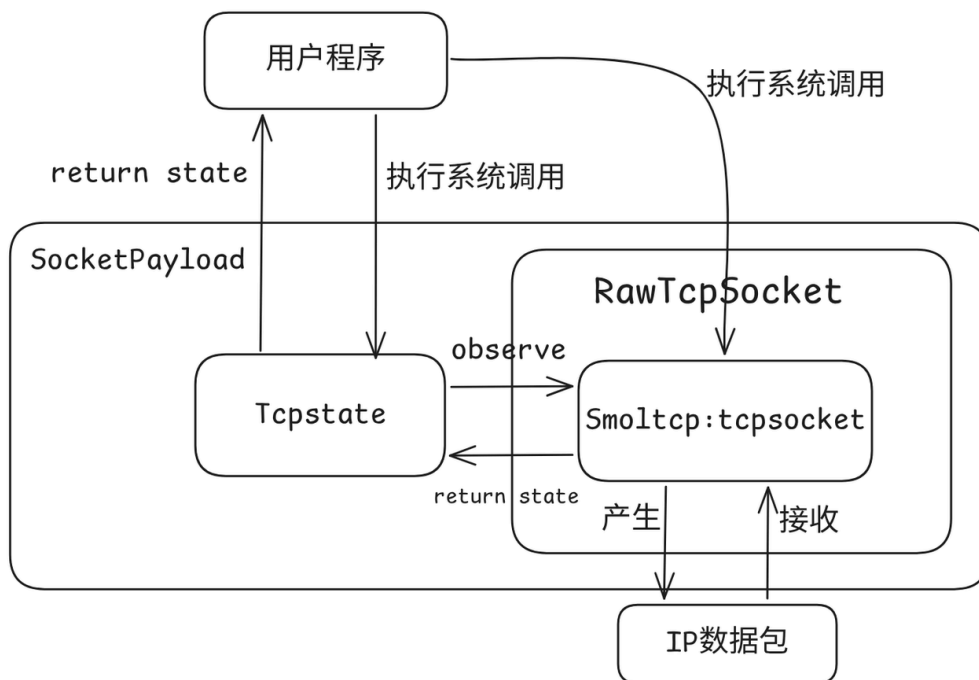


图 24 TCP 套接字的两层结构与数据流

上图展示了 TCP 套接字的两层结构与数据流。一个 TCP 套接字在 SocketPayload 内被拆成两层——左侧是内核自管的生命周期状态机 TcpState,右侧是封装 smoltcp 协议引擎的 RawTcpSocket。用户的系统调用同时作用于两层:既推进 TcpState 的状态变化,也驱动 smoltcp 执行协议动作;TcpState 通过 observe 读回 smoltcp 的真实协议状态,据此向用户返回套接字状态。报文的实际收发则在 smoltcp 与 IP 层之间进行。

TcpState 只记录套接字当前处于生命周期的哪个阶段(初始、已绑定、监听、连接中、已连接、已关闭),仅携带本地/对端端点,而不持有任何协议数据:

```
pub enum TcpState {  
    Init,  
    Bound      { local: IpEndpoint },  
    Listening   { local: IpEndpoint, backlog_limit: usize },  
    Connecting { local: IpEndpoint, remote: IpEndpoint },  
    Connected  { local: IpEndpoint, remote: IpEndpoint },  
    Closed,  
}
```

状态之间的转换由各 bind / listen / connect / accept 等执行步骤推进,如下图所示。

真正的三次握手、序列号、窗口与重传则由 RawTcpSocket 内的 smoltcp tcp::Socket 完成。以下图 25 是 smoltcp 的状态机

建立连接

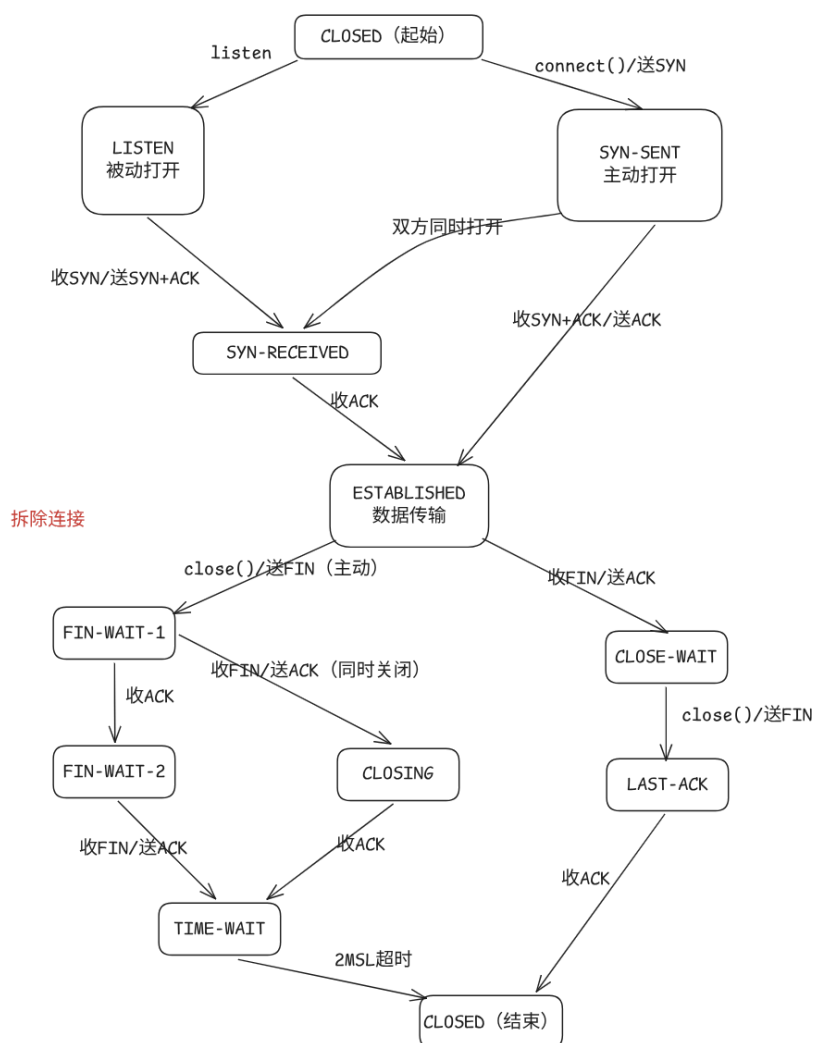


图 25 smoltcp 状态机

`RawTcpSocket` 是 TCP 协议引擎与内核缓冲的载体。它在 `smoltcp` 自带缓冲之外，另维护了自己的接收缓冲 `rx_buffer`、发送缓冲 `tx_buffer` 与 `corked_tx`：内核借此实现多段排空、`TCP_CORK` 聚合发送，并通过 `last_syn_ack` 截获 `smoltcp` 生成的报文段，自己控制重传时间间隔。

```

pub struct RawTcpSocket {
    socket: SpinMutex<Box<tcp::Socket<'static>>>, // smoltcp TCP 引擎
    protocol_state: SpinMutex<RawTcpProtocolState>, // 连接/半关闭/RST 标记
    last_syn_ack: SpinMutex<Option<SmoltcpTcpSegment>>, // 截获待发报文段
    rx_buffer: SpinMutex<VecDeque<u8>>, // 内核侧接收缓冲
    tx_buffer: SpinMutex<VecDeque<u8>>, // 内核侧发送缓冲
    corked_tx: SpinMutex<Vec<u8>>, // TCP_CORK 聚合
    recv_capacity: usize, send_capacity: usize,
}

```

图 26 以 send/recv 应用程序为例,该图展示在 loopback 下内核与 smoltcp 的调用关系。其中用 Rawtcpsocket.rxbuffer 把 smoltcp 接收区尽快抽空以释放窗口,缓解 smoltcp 压力;Rawtcpsocket.txbuffer 与 corked_tx 实现聚合发送(避免一次只发送一小段)。

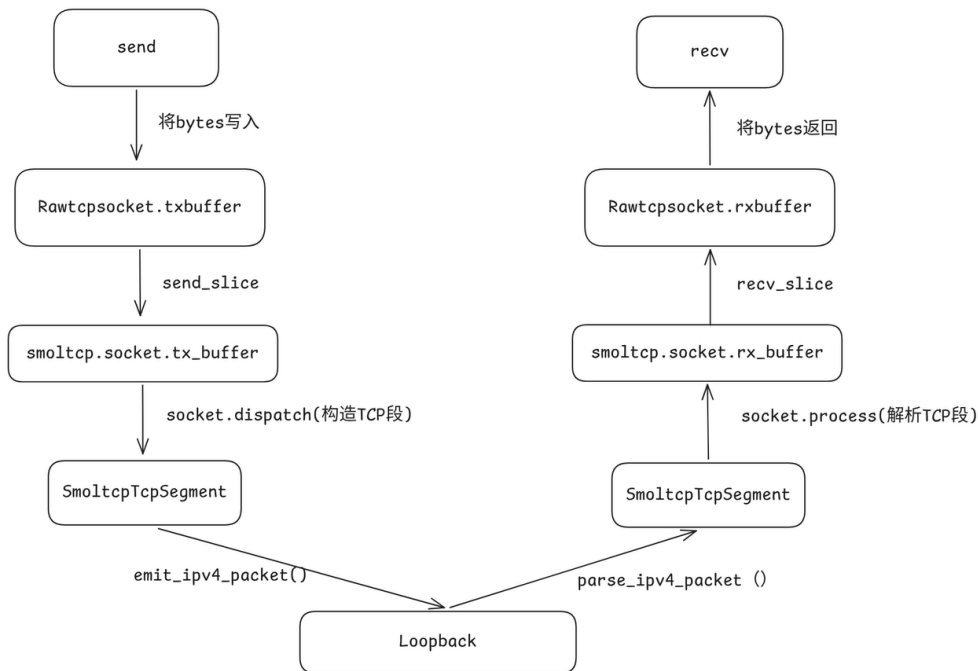


图 26 loopback 下内核与 smoltcp 的调用关系

10.3.2 UDP

UDP 套接字 `RawUdpSocket` 内部封装一个 smoltcp 的 `udp::Socket`, 并在其外维护内核侧的接收/发送数据报队列与一个用于聚合发送的暂存槽 `corked_tx`, 同时记录收发缓冲的字节容量与数据报个数上限以防止内存溢出:

```

pub struct RawUdpSocket {
    socket: SpinMutex<Box<udp::Socket<'static'>>>, // smoltcp UDP 引擎
    rx_datagrams: SpinMutex<VecDeque<UdpRxDatagram>>, // 内核侧接收队列
    tx_datagrams: SpinMutex<VecDeque<UdpTxDatagram>>, // 内核侧发送队列
    corked_tx: SpinMutex<Option<UdpTxDatagram>>, // 聚合发送暂存
    recv_capacity: usize, send_capacity: usize,
    recv_packet_capacity: usize, send_packet_capacity: usize,
}

```

发送时构造 UDP 数据报并经网络层发出;接收时根据目的端口查找已绑定的套接字, 优先匹配完整四元组、其次匹配仅绑定端口者, 命中后将数据报入队并唤醒等待的接收任务。绑定端口的索引由命名空间内的套接字表统一维护。

10.4 网络层

网络层负责 IPv4 数据包的接收、解析、路由与发送。(和总览图中的结构一样)

其中 IPv4 报文的逐字段解析、封装与校验和计算复用了成熟协议库 (smoltcp) 的 “wire” 能力实现, 而以太网封装、路由决策、分片重组与转发逻辑则由内核自行实现。

接收方向：设备收到以太网帧后，先依据以太网类型字段区分 ARP 与 IPv4；对 IPv4 报文校验首部长度与整体长度的合法性，若为分片报文则先完成重组，得到完整数据包后，再根据其协议字段（ICMP、TCP、UDP）分发给对应的传输层模块。若目的地址不属于本机，则在开启 IP 转发时按路由表选择出口接口转发（其间经过出入接口检查、netfilter hook 与 NAT 处理），未开启转发时直接丢弃。

```
match ethernet.ethertype() {
  EthernetProtocol::Ipv4 => {          // IPv4 报文
    // 校验长度合法性，必要时重组分片，得到完整 IP 包
    let packet = self.prepare_ipv4_ingress(ethernet.payload());
    // 再按协议字段（TCP/UDP/ICMP）分发给对应传输层模块
    let dispatch = demux_rx_frame_with_smoltcp(/* 完整 IP 包 */);
    self.maybe_reply_icmpv4(&dispatch, now, guard); // ICMP 回显自动应答
    dispatch
  }
  EthernetProtocol::Arp => self.process_arp(ethernet.payload(), now, guard),
  _ => PacketDispatch::Unsupported
}
```

发送方向：对于 TCP、UDP 与 ICMP，完整的 IP 数据包（含首部与校验和）均借助协议库（smoltcp）构造完毕；网络层在此基础上查找路由以确定输出设备与下一跳地址，再经 ARP 解析得到下一跳 MAC，最后交由链路层完成以太网封装与发送。内核唯一需要自行计算 IP 首部校验和的场合是分片与重组——此时需对拆分或重新拼装后的报文重新填写首部校验和。

```
pub fn dispatch_ip_at(&self, packet: &[u8], now: Instant,
                     guard: &Guard) -> PacketTxResult {
  // packet 已是完整 IP 包（首部+校验和由 smoltcp 在传输层封好）
  let ipv4 = Ipv4Packet::new_checked(packet)?; // 仅校验长度合法性
  // ① 查路由，取下一跳（无路由 → EADDRNOTAVAIL）
  let next_hop = decide_ipv4_route(self.common, ipv4.dst_addr().into())
    .next_hop()?;
  // ② ARP 解析下一跳 MAC（未命中则登记 pending，回复后补发）
  let dst_mac = match self.resolve_or_request(next_hop, now) {
    ArpResolution::Resolved { mac } => mac,
    ArpResolution::Pending { next_hop } =>
      return PacketTxResult::PendingResolution { next_hop },
    ArpResolution::Failed { errno, .. } =>
      return PacketTxResult::Failed { errno },
  };
  // ③ 封以太网帧头并交设备发送
  self.transmit_ipv4_packet(dst_mac, packet, guard)
}
```

路由表按网络命名空间维护，每个命名空间持有一组路由条目，条目记录目标网络、前缀长度、网关、输出接口与来源地址等信息，查找采用最长前缀匹配：

```
struct NetNamespaceRouteEntry {
  dst: Ipv4Address,          // 目标网络
```

```

prefix_len: u8,                // 前缀长度(用于最长前缀匹配)
gateway: Option<Ipv4Address>,  // 下一跳网关
oif_name: Option<&'static str>, // 输出接口
preferred_src: Option<Ipv4Address>,
table: u8, protocol: u8, scope: u8, route_type: u8,
}

```

txKernel 还实现了 ICMP，实现了 Echo 请求与回复以支持 Ping。与 TCP/UDP 借助 smoltcp 的套接字不同，ICMP 回显不经 smoltcp 套接字，而是直接借助 smoltcp 的“wire”能力自行构造应答报文：收到 Echo Request 时交换源与目的地址、改写报文类型、重算校验和，再经 IP 层回送。

```

pub fn build_icmpv4_echo_reply(p: &Icmpv4EchoPacket) -> LoopbackIpPacket {
    build_icmpv4_echo_packet(p, /*request=*/ false) // 与 request 共用构造器
}

// 先建 ICMP 报文(选 Reply 变体、算 ICMP 校验和),
// 再借 smoltcp 封 IPv4 头(next_header = Icmp, hop_limit = 64)
fn build_icmpv4_echo_packet(p: &Icmpv4EchoPacket, req: bool)
    -> LoopbackIpPacket {
    let icmp = build_icmpv4_echo_message(p, req);
    let ip = IpRepr::Ipv4(Ipv4Repr {
        src_addr: to_smoltcp_ipv4(p.src),
        dst_addr: to_smoltcp_ipv4(p.dst),
        next_header: IpProtocol::Icmp,
        hop_limit: 64,
        payload_len: icmp.len(),
    });
    let hlen = ip.header_len();
    let mut bytes = vec![0u8; hlen + icmp.len()];
    ip.emit(&mut bytes[..hlen], &ChecksumCapabilities::default());
    bytes[hlen..].copy_from_slice(&icmp); // IP 头 + ICMP 报文
    LoopbackIpPacket::new(bytes)
}

```

10.5 链路层

链路层主要处理以太网帧的封装与解析，以及地址解析协议 ARP。EtherIface 把一个实现了 NetDeviceOps 的设备包装为一个三层网络接口，在其内部维护 ARP 缓存、待解析表、邻居发现表与 IPv4 分片重组表（另含承载本接口 IP 与路由配置的 common）：当设备接收到一帧时，该接口先检查帧长度合法性、确认目的 MAC 是本机或广播/组播，再提取以太网类型，据此把数据包分发给 IP 模块或 ARP 模块。

```

pub struct EtherIface {
    pub netdev: &'static NetDeviceRegistration,
    pub common: IfaceCommon, // 本接口 IP / 路由等公共配置
    pub ether_addr: EthernetAddress,
}

```



```

arp_table: SpinMutex<BTreeMap<Ipv4Address, ArpEntry>>,      // IP→MAC 缓存
pending_arp: SpinMutex<BTreeMap<Ipv4Address, ArpPendingEntry>>, // 待解析
ndisc_table: SpinMutex<BTreeMap<Ipv6Address, NdiscEntry>>, // IPv6 邻居
ipv4_fragments: SpinMutex<BTreeMap<Ipv4FragmentKey, Ipv4ReassemblyEntry>>,
next_ipv4_ident: AtomicU64, // 分片用的 IP 标识号
pub name: &'static str,
pub stats: NetStats,
pub arp_stats: ArpStats,
}

```

ARP 模块维护一张 IP 到 MAC 地址的映射缓存，并关联对应的网络设备。当上层需要发送 IP 数据包但尚未知道目的 MAC 时，链路层先查找缓存：若命中则直接在数据包前封装以太网帧头并发出；若未命中，则在 `pending_arp` 中登记该报文的目的地 IP 地址（原报文暂不发出），由驱动循环据此构造 ARP 请求广播出去，待回复到达后再补发原报文。缓存表项设有 300 秒的老化时间，过期后重新解析。如果在这其中收到 ARP 请求数据包的话，则将其源 MAC 缓存（learn 机制），过程如图 27 所示：

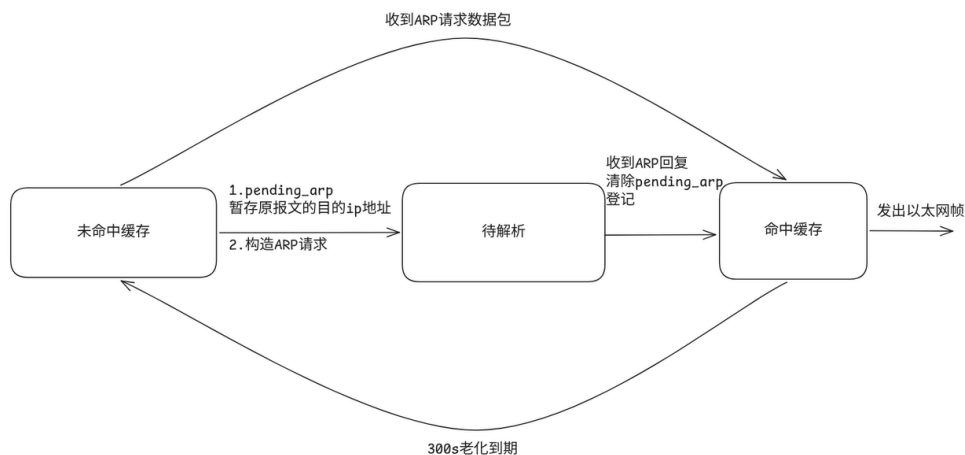


图 27 ARP 映射缓存机制

收到 ARP 数据包时，模块解析其操作码与地址字段：若为请求，先记录请求方的地址映射，再在目标 IP 为本机时回送 ARP 应答；若为回复，则更新缓存并清除该地址的待解析登记，其余在 `pending_arp` 表中被搁置的报文随后由驱动循环补发 ARP 请求。过程如图 28 所示

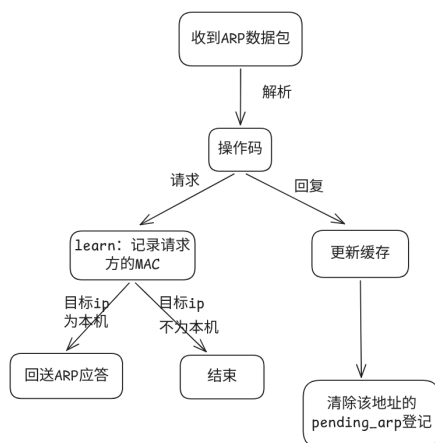


图 28 收到 ARP 数据包的解析过程

10.6 设备

设备层是网络模块的最底层，负责抽象各类虚拟网络设备，并向上层提供统一的发送、接收与中断接口。该层通过 `NetDeviceOps` trait 规范所有设备的行为，其中包括接收一帧 `receive`、发送一帧 `transmit`、查询发送就绪 `tx_readiness`、获取 MAC 地址与 MTU，以及网桥端口管理与中断应答等可选接口：

```
pub trait NetDeviceOps: Send + Sync + 'static {
    fn receive(&self) -> Option<RxFrame>;
    fn transmit(&self, frame: &[u8], guard: &Guard<'_>) -> StepOutcome<()>;
    fn tx_readiness(&self, guard: &Guard<'_>) -> PacketTxReadiness { .. }
    fn mac_addr(&self) -> EthernetAddress;
    fn mtu(&self) -> u16;
    fn device_kind(&self) -> NetDeviceKind { NetDeviceKind::Ethernet }
    // 网桥端口、中断应答等可选接口, 默认实现略
    fn ack_interrupt_and_fire(&self) -> NetDeviceIrqOutcome { .. }
}
```

目前实现该 trait 的设备共 6 种，对应 `NetDeviceKind` 的六个变体：loopback、virtio-net (Ethernet)、veth、bridge、vlan 与 dummy。其中

- virtio-net 是接入宿主机网络的真实网卡，其收发队列各设 256 个描述符 (`VirtioNetQueueConfig::new(256, 256)`)；
- veth 为成对的虚拟以太网设备，一端发出的帧直接进入对端的接收队列；
- bridge 维护学习型 MAC 转发表与端口集合，按目的 MAC 转发或泛洪；vlan 仅建模 VLAN 接口的元数据（创建、启停、列举、删除），并未实现真正的打标签数据面，其 `transmit` 与 dummy 一样是空操作；
- dummy 则用于丢弃测试。回环设备较为特殊：`LoopbackDevice` 同样实现了 `NetDeviceOps`（`device_kind` 为 `Loopback`），但采用 NOARP、IP 级语义——`transmit` 入队的是裸 IP 包（不加以太网帧头、不做 ARP 解析），`receive` 在同一接口上原样取回，恰如 Linux 的 `lo`；

- `LoopbackIface` 持有一个 `LoopbackDevice` 并把队列操作委派给它，使回环数据面也走统一的 `NetDeviceOps` 接口，而非自成一套旁路队列。这样既复用了统一的设备抽象，又避免为回环强行套上以太网封装与邻居解析。

10.7 异步网络 I/O 模型

txKernel 的网络栈以无栈协程 (Rust 的 `async/await`) 组织。所有可能阻塞的网络操作 (`connect`、`accept`、`recv`、`send`) 都不会阻塞内核线程：当操作无法立即完成时，执行步骤返回 `Yield`，连同它要等待的载体 (`wait carrier`) 与关心的事件一并交还调度器，协程随即被挂起；待底层事件发生、对应载体被触发后，调度器重新调度该协程，从挂起点继续执行。执行步骤的返回结果除完成 `Done` 与出错 `Err` 外，正是这个让出态 `Yield`：

```
pub enum StepOutcome<T, P> {
    Continue { progress: P },           // 取得进展,继续推进
    Yield    { progress: P, shape: YieldShape }, // 让出,等待某载体
    Done(T),                             // 完成
    Err(Errno),                          // 出错
}
```

例如阻塞式 `connect` 在握手尚未完成时，构造一个等待“发送缓冲可用或连接断开”的等待 token 并让出，而不是自旋或占用线程：

```
let wait = WaitToken::new(
    socket.wait_carriers.send,
    SendWireSet::SPACE.bits() | SendWireSet::BROKEN.bits(),
);
yield_on_token(wait) // 返回 StepOutcome::Yield{ shape: OnWaitSource{..} }
```

整栈的协议推进与收发由一个独占的协作式驱动循环 `net_delegate_task_loop` 统一驱动。它本身就是一个 `async` 函数：循环等待 `POLL/TICK` 唤醒，每次被唤醒后在 `epoch::guard` 下执行一次 `net_delegate_step_once`（依次完成收包、回环、发送、ARP 解析与定时器推进），并据返回的下一个截止时刻安排下一次唤醒，随后休眠。整个过程状态如图 29 所示：

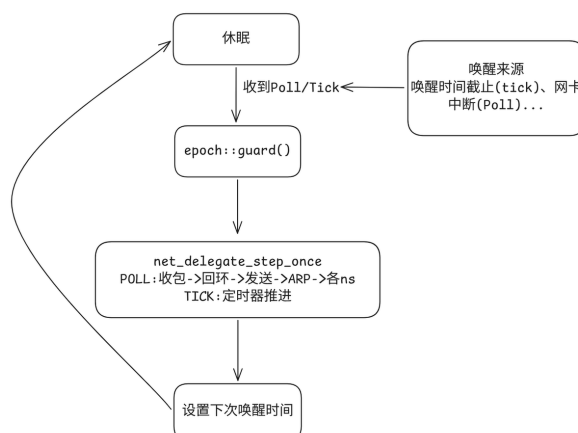


图 29 网络驱动循环状态变化过程

```

pub async fn net_delegate_task_loop_with_deadline_hook(
    driver: &dyn NetDelegateDriver, config: NetDelegateTaskConfig,
    mut on_deadline: impl FnMut(Option<Instant>),
) -> NetDelegateTaskReport {
    let mut report = NetDelegateTaskReport::default();
    loop {
        // 等待 POLL/TICK 唤醒; 令牌无效或非 Ready 即退出
        let token = net_delegate_wait_token(); // POLL | TICK
        let Some(wait) = wait_source::wait_on_token(token)
        else { break };
        if wait.await != WaitOutcome::Ready { break; }
        let guard = tx_substrate::epoch::guard();
        let outcome = net_delegate_step_once(driver, &guard); // 收包/回环/发送/ARP/定
时
        on_deadline(outcome.next_deadline); // 安排下一次唤醒
    }
    report
}

```

这一无栈协程模型与单一驱动循环相配合，使得成百上千个套接字的阻塞等待都不占用内核线程，也把协议推进的时机收敛到一处，便于推理并发正确性与控制调度开销。

第 11 章 开发流程与团队分工

团队分工：

- **杨岩琰（队长）**：负责 reactor 设计，线程进程设计及 VFS 设计。
- **刘佳硕**：负责硬件抽象层设计、RISC-V 与龙芯设计，TTY 设计。
- **孟书培**：负责网络栈设计、网络外设驱动设计。

开发流程：

- **1—2 月**：假期，学习 Rust，完成 rCore 实验，学习 BSD 架构。基于 BSD 内存管理架构和 Chronix 的异步无栈协程设计了第一版内核，用 AI 编写了能跑通 BusyBox 的原型。在推进到 procfs 时发现 BSD 的 vmObject 架构并不能很好地支持伪文件系统，遂放弃。
- **3 月**：暂停进度，在第一版内核的基础上重新设计内核的架构。现在的异步模型、系统调用架构都是在 3 月设计的。
- **4 月**：打地基，启动、HAL、pmap、page allocator、EBR/Zone、reactor、trap、progress/CI 工具链。
- **5 月**：搭内核功能面，VM/VFS/PageBacked、process/thread/signal/tty、exec/ELF、ext4、fork/clone/wait4、OSComp basic/busybox 初步跑通。
- **6 月**：冲真实 Linux 兼容和可用性，LTP/OSComp 网络、SCTP、netperf/iperf、glibc/musl 差异、Alpine procfs/tty/sqlite/tcc、ext4 持久化、tx-observe 性能归因，整理 AI 代码。

应该说, AI 给了我们更大的学习和试错空间: 快速修改代码和学习现有接口在以前都是难以想象的。AI 开发也帮我们快速暴露了大规模项目中的沟通、知识管理、项目协作问题, 以及开发中才能遇到的细节问题。这些都是课内小规模试验难以学到的知识。在第一版内核中暴露出的问题 (VFS 设计、对象分离、完整 future 实现等) 都在第二版的设计文档里被解决, 从而保证了三个月内核的高速开发没有因为架构问题发生大的重构。

第 12 章 AI 使用说明

本作品由学生和 AI 共同完成。AI 承担了主要的编码工作, 但每个子系统的设计是由参赛队员完成的, 设计文档保留在 `/docs/design` 下。AI 编码完成后队员验证并重构代码。AI 使用的进展记录在 `progress.md` 内。

我们将 AI 用于下面场景:

- **知识学习与设计。** 在比赛初始阶段, 我们借助 AI 读往届优秀作品与一些开源的内核项目, 用 AI 拆解往届作品的代码, 给出往届作品中的优秀设计以及开源项目中值得借鉴的地方。
 - **想法验证与原型搭建。** 在假期学习阶段, 我们用 AI 参考 BSD 搭建了第一版内核, 阅读和修改原型给我们提供了相比教材和具有提供框架的实验所不可比的一手知识和雷点。
 - **内核调试。** AI 对反汇编工具链有良好的知识, 在裸机开发环境帮助我们节省了大量时间。我们为 AI 准备了一系列 `xtask` 命令, 封装了常见的反汇编、`qemu` 内存 dump 与栈追踪、`scause` decode 工具, 并准备了相关的 `debug skill` 辅助 AI 调试。
 - **性能调优。** 我们用 AI 构建了 `tx-observe` 框架, 在系统调用关键节点输出带时间戳和层级关系 `log`, 导出为 `parquet` 格式供 AI 用 SQL 分析热点路径, 避免 AI 低效读取或通过打印调试浪费 `token`。
- 和 AI 开发时遇到的问题和反思如下:
- **AI 开发时代更加强调项目的知识管理。** AI 生成代码的速度远大于人阅读的速度, 没有架构知识很快就会跟不上 AI 生成代码的思路, 所谓 `code is cheap, show me the talk`。
 - **对 AI 指令不明确容易导致 AI “走捷径”,** 只做局部修改通过测例或不完全修改。我们主要通过两个思路克服: 一是善用 `linter` 等代码检查工具, 拒绝编译含有旧接口的代码; 二是跟上代码库的知识, 避免对 AI 提出模糊的指令。
 - **围绕 coding 的工具链直接决定了 AI 的能力上限。** AI 开发下, 工具链 (如 `cloc`、`rg`, 以及自己实现的 `xtask`) 的质量至关重要。AI 输出上下文有限, 不可能装入代码库的所有知识。以性能调优为例, 代码量大后 AI 不可能跟踪整个调用流程, 而 `flamegraph` 等追踪框架能快速帮 AI 定位关键路径, 能极大节约 AI 开发时间。
 - **Harness 工程同样重要。** 过多的 `skills`、MCP 注入会污染和浪费 Agent 的上下文。应当追求知识和技能对 Agent 的渐进式披露 (`progressive disclosure`), 给设计文档和 `skills` 建立简单的索引, 让 AI 按需读取, 而不是盲目探索或全部注入。
 - **开发者必须对知识负责。** 代码是项目知识的实现。当 AI 推进到用户知识的边界后, 开发速度会自然放缓。我们队员在撰写这份设计文档时发现了许多代码中不熟悉的地方, 阅读这些代码指明了我们进一步开发的方向。

第 13 章 未来展望

1. **Linux ABI 覆盖还不完整。** 我们已经有大量 syscall 和 OSComp/LTP 证明, 但还差成体系的尾部: waitid / clone3 / setns / unshare / pidfd_getfd、 sendmsg / recvmsg / sendmmsg / recvmmsg、 xattr、 openat2 / execveat / 现代 mount API、 timer_create / getitimer / setitimer、 process_vm_*、 memfd_create、 seccomp / bpf / perf / ptrace / keyring / landlock 等。这里不是简单补表, 很多需要真正的策略和对象模型。
2. **真实发行版兼容还没闭环。** Alpine 已经能跑 shell、BusyBox vi、SQLite、TCC 演示、procfs/net/TTY 一批用户态程序, 但还不是完整 distro 体验。GCC 大 rootfs、动态 loader / 大文件读取、复杂临时文件 / pipe / fork / wait 组合、OpenRC/daemon 类 workload 还需要继续扫。
3. **文件系统离完整语义还有距离。** tmpfs/devfs/procfs/ext4 已经很有用, SQLite ext4 持久化也有证明。但完整内核还需要: xattr、完整 mount namespace 和现代 mount API、crash recovery / fsync durability、更多 ext4 语义、fanotify/inotify 真实事件队列、quota/handle/open-by-handle 等。现在更像“能支撑关键用户态路径”, 不是“完整通用文件系统栈”。
4. **进程、线程、信号有待完善。** fork/clone/wait、pthread、futex、signal 都已经可用, 但 glibc pthread cancel/join、robust-list、clear_child_tid、queued RT signals、waitid/rusage、rseq、复杂 signal restart 语义还没完全 Linux 等价。这里会直接影响 glibc/libctest、语言运行时和高并发程序。
5. **网络还有高级协议和边角。** 我们已经推进到 netperf/iperf、SCTP、netns/veth、procfs/sysfs、iptables/nft shim 等很强的阶段。但完整内核还要补: 更深的 netfilter/nftables、IPsec、multicast 全量、CAN、NFS/RPC、AF_UNIX 文件系统语义、完整 route/IPv6 策略、socket message-vector API 和长期稳定批量运行。
6. **SMP 和性能还不是生产级。** QEMU SMP、reactor、remote wake、pmap shutdown、observe 分析都已经基础。但 progress 里最新 pthread hot path 仍显示 sys_clone、sys_munmap、sys_futex、sys_rt_sigprocmask、sys_mmap、sys_exit 是主要成本桶。完整内核需要把这些从“可运行”推进到“可扩展”: 更低锁持有时间、更好的 pmap/VM teardown、remote wake/shutdown 策略、调度公平性和多核默认用户态。
7. **安全和权限模型还偏浅。** 现在有 DAC、setuid、部分 capability 行为、usersns/netns 相关能力, 但完整 Linux 还包括 seccomp、LSM/Landlock、keyring、audit、capget/capset 深语义、ptrace 权限、namespace 隔离完整性。这些目前不是主战场。
8. **硬件和驱动生态很窄。** 主力证明面是 RV64 QEMU; LA64 有进展但不是同等成熟; 真实 K210/M1 Dock 等硬件路径仍是缺口。驱动上已有串口、virtio block/net 等核心路径, 但离完整内核的 PCI、块层、电源、时钟、输入、图形、存储设备生态还很远。

第 14 章 参考设计

- ArceOS (axhal) —— 对其 axhal 进行修改实现了我们的硬件抽象层。
- StarryOS (rsxt4) —— 抛弃其内部缓存, 提取同步逻辑对其做了异步适配。
- Asterinas (仓库) —— 参考其网络栈设计。

- 《FreeBSD 操作系统设计与实现（第二版）》与 FreeBSD 内核代码 —— 最初的学习资源，提供了第一版内核架构参考，以及第二版内核的消息总线设计。
- **Chronix**（仓库） —— 无栈异步协程设计。
- **Linux**（官网） —— 大量参考，系统调用功能的金标准。

